MACHINE LEARNING FROM SCRATCH — MODULE 5

MATHEMATICS FOR MACHINE LEARNING

From Beginner Foundations to Advanced ML Mathematical Mastery

A Comprehensive Professional Reference Handbook

Kajola Gbenga Data Scientist & AI Engineer

How to Use This Module

Every major concept follows the same pattern:

```
FORMULA → NUMERICAL EXAMPLE → NUMPY IMPLEMENTATION → ML APPLICATION
```

You do not need a mathematics background. You do need to work through the numerical examples by hand at least once — reading mathematics is not the same as doing it. All code assumes `import numpy as np`.

Table of Contents

Foundations (1–16) Why maths matters · Notation · Arithmetic · Algebra · Linear equations · Simultaneous equations · Functions · ML functions · Exponents · Logarithms · Sigmoid · Softmax · Sequences · Summation · Coordinate geometry · Distance

Calculus (17–32) Limits · Derivatives · Differentiation rules · ML derivatives · Partial derivatives · Gradient · Chain rule · Second derivatives · Multivariable calculus · Directional derivatives · Hessian · Optimization · Gradient descent · GD variants · Learning rate · Convexity

Linear Algebra (33–54) Introduction · Scalars · Vectors · Vector operations · Dot product · Norms · Matrices · Matrix operations · Matrix multiplication · Transpose · Identity · Inverse · Linear systems · Determinants · Rank · Linear independence · Basis & vector spaces · Eigenvalues · Eigenvectors · PCA · Covariance · Correlation

Probability & Statistics (55–70) Foundations · Conditional probability · Bayes · Random variables · Expectation · Variance · Standard deviation · Distributions · Normal · Z-score · CLT · Law of large numbers · Sampling · Estimation · Likelihood · MLE

ML Mathematics (71–95) Loss functions · Regularization · Bias-variance · Linear regression · Logistic regression · KNN · Naive Bayes · Decision trees · Entropy · Information gain · Gini · Clustering · Gradient optimization · Numerical stability · Scaling · High-dimensional distance · Cosine similarity · Projections · GD from scratch · Neural networks · Backpropagation · Transformers · Embeddings · Reinforcement learning · Time series

Practice & Reference (96–110) NumPy computation · Visualization · Exercises · Coding challenges · Interview prep · Troubleshooting · Formula sheet · Maths→ML map · Capstone · Assessment · Concept map · Careers · Memorize vs understand · Cheat sheet · Module 6 preparation

1. Why Mathematics Matters in Machine Learning

1.1 The Honest Answer

You can train a model without mathematics. `model.fit(X, y)` works. But you cannot *debug* it, *choose* between approaches, *know when it is lying to you*, or *invent anything new* without understanding the machinery underneath.

Concretely, mathematics is what lets you answer: Why did my loss become `NaN`? Why does this feature ruin my model? Why is my learning rate wrong? Why does regularization work? Why is my matrix singular? Why does KNN fail in high dimensions? Every one of those is a mathematical question wearing an engineering costume.

1.2 The Five Mathematical Pillars

Branch	What it gives you	Where you meet it
Algebra	Manipulating symbols and equations	Every formula you will ever read
Calculus	How things change; how to find minima	Gradient descent, backpropagation
Linear algebra	Operating on many numbers at once	Feature matrices, neural networks, embeddings, PCA
Probability	Reasoning under uncertainty	Naive Bayes, likelihood, generative models
Statistics	Learning from samples	Estimation, evaluation, inference
Optimization	Finding the best parameters	*Training itself is optimization*

1.3 The Structure of All Machine Learning

```
        MODEL  +  PARAMETERS  +  DATA  +  LOSS FUNCTION  +
OPTIMIZATION
                                    ‖
                                    ▼
                            MACHINE LEARNING
```

Every supervised algorithm in this curriculum is an instance of this template:

Component	Linear Regression	Logistic Regression	Neural Network
Model f	$X\beta$	$\sigma(X\beta)$	nested layers of $f(Wx+b)$
Parameters θ	β	β	all W, b
Loss	MSE	Log loss	Cross-entropy
Optimization	Normal equation or GD	Gradient descent	GD + backpropagation

Recognizing this template is the single most valuable structural insight in the module. Once you see it, a new algorithm stops being "a new thing to memorize" and becomes "which four slots did they fill differently?"

1.4 The Master Equation

```
y = f(X, θ)
```

Symbol	Name	Meaning	Concrete example
X	Input / features / design matrix	What we know	Customer age, income, tenure
y	Output / target / label	What we predict	Will they churn?
f	Model / hypothesis function	The *form* of the relationship	A line, a tree, a network
θ	Parameters (theta)	The numbers *learned* from data	Regression coefficients, network weights

Read it aloud: *"The output is some function of the inputs, controlled by parameters."*

Learning means finding the θ that makes f(X, θ) closest to the true y. That is the entire game. Sections 28–32 are about how we perform that search.

1.5 A Caution About Ambition

You do not need to become a mathematician. You need enough mathematics to reason about ML systems. This module deliberately teaches *the mathematics that pays rent* — every section justifies its existence by pointing at an algorithm.

---

2. Mathematical Notation

2.1 The Symbol Table

Symbol	Read as	Meaning	Example
Σ	"sum over"	Add up a sequence	$\sum_{i=1}^{n} x_i = x_1+x_2+\ldots+x_n$
Π	"product over"	Multiply a sequence	$\prod_{i=1}^{n} x_i = x_1 \cdot x_2 \cdot \ldots \cdot x_n$
∈	"is an element of"	Membership	$x \in \mathbb{R}$ ("x is a real number")
∀	"for all"	Universal quantifier	$\forall i, x_i > 0$
∃	"there exists"	Existential quantifier	$\exists x$ such that $f(x)=0$
≈	"approximately equals"	Near equality	$\pi \approx 3.14159$
≤ ≥	"less/greater than or	Inequality	$0 \le p \le 1$

Symbol	Read as	Meaning	Example
	equal”		
∝	“proportional to”	Scales with	P(A
∞	“infinity”	Unbounded	lim x→∞
ℝ	“the real numbers”	Set of all reals	x ∈ ℝ
$\mathbb{R}^{\mathbf{n}}$	“R-n”	n-dimensional real space	x ∈ $\mathbb{R}^3$ is a 3-vector
argmin	“argument that minimizes”	The *input* giving the smallest output	θ* = argmin_θ L(θ)
:=	“is defined as” / “is assigned”	Definition or update	θ := θ − α∇J
|x|	“absolute value of x”	Magnitude, sign removed	|−3| = 3
‖x‖	“norm of x”	Vector length	$\|x\|_2 = \sqrt{(\Sigma x_i^2)}$
∂	“partial d”	Partial derivative	∂f/∂x
∇	“nabla” / “del” / “gradient”	Vector of all partial derivatives	∇f
∫	“integral of”	Continuous summation	∫f(x)dx

2.2 Conventional Letters in ML

Convention	Meaning
Lowercase italic *x*, *y*	Scalars (single numbers)
Lowercase bold **x**, **y**	Vectors
Uppercase bold **X**, **W**	Matrices
n	Number of samples (rows)
m or *d* or *p*	Number of features (columns)
i	Index over samples (i = 1…n)
j	Index over features (j = 1…m)
θ, β, **W**	Parameters
α (alpha)	Learning rate
λ (lambda)	Regularization strength
ŷ (“y-hat”)	Predicted value (the hat means *estimated*)
x̄ (“x-bar”)	Sample mean
μ (mu)	Population mean
σ (sigma)	Standard deviation; σ^2 is variance
ε (epsilon)	Error term, or a tiny number
η (eta)	Sometimes used for learning rate

A note on the hat. ŷ vs y is not decoration. **y** is truth; **ŷ** is your model's guess. Confusing them makes every loss-function formula incomprehensible, since loss is precisely a measure of the gap between them.

2.3 Summation Notation Read Slowly

$$\sum_{i=1}^{n} x_i$$

Three parts: *i=1* (start), *n* (stop), x_i (what to add). It means: set i to 1, take x_1; set i to 2, take x_2; keep going to n; add everything.

```
x = np.array([3, 7, 2, 8])
print(x.sum())          # 20  — this is Σxᵢ
print(np.prod(x))       # 336 — this is Πxᵢ
```

Every Σ in this module is a loop you could write in Python. That reframing removes most of the intimidation.

---

3. Arithmetic Foundations

3.1 The Pieces You Need

Concept	ML relevance
Integers / decimals	Counts vs continuous measurements; dtype matters in NumPy
Fractions	Probabilities are fractions of a whole
Percentages	Accuracy, growth rates — a percentage is a fraction × 100
Ratios	Class imbalance (1:99), odds, engineered features like debt-to-income
Negative numbers	Errors, gradients, coefficients pointing the opposite way
Order of operations	Misreading a formula is a real and costly bug
Powers	Squared errors, exponential functions
Roots	RMSE, Euclidean distance, standard deviation

3.2 Order of Operations — PEMDAS

Parentheses → **E**xponents → **M**ultiplication/**D**ivision (left to right) → **A**ddition/**S**ubtraction (left to right).

Consider MSE: $(1/n)\Sigma(y_i - \hat{y}_i)^2$. The parentheses force the subtraction *first*, then the square, then the sum, then the division. Compute it in any other order and you get a different (wrong) number.

```
y_true = np.array([10, 20, 30])
y_pred = np.array([12, 18, 33])

# Step by step — exactly following the order of operations
diff    = y_true - y_pred        # [-2,  2, -3]   innermost
parentheses
squared = diff ** 2              # [ 4,  4,  9]   exponent
total   = squared.sum()          # 17             summation
mse     = total / len(y_true)    # 5.667          division
print(mse)                       # 5.666666666666667
```

3.3 Powers and Roots

Rule	Example
$x^2 = x \cdot x$	$5^2 = 25$
$x^0 = 1$ (any $x \neq 0$)	$7^0 = 1$
x^(1/2) = √x	9^0.5 = 3
x^(−1) = 1/x	4^(−1) = 0.25

```
print(np.sqrt(16))      # 4.0
print(16 ** 0.5)        # 4.0  — same thing
print(2 ** -1)          # 0.5
```

ML connection. RMSE = √MSE exists purely to undo the squaring, returning the error to the *original units* of y. If MSE is 25 "squared naira", RMSE is 5 naira — a number a business person can actually interpret.

3.4 Ratios and Class Imbalance

A fraud dataset with 500 fraud cases in 100,000 transactions has a positive rate of 500/100000 = 0.005 = **0.5%**, an imbalance ratio of **1:199**.

```
n_fraud, n_total = 500, 100_000
print(f"Rate: {n_fraud/n_total:.3%}")                # Rate: 0.500%
print(f"Ratio 1:{(n_total-n_fraud)/n_fraud:.0f}")    # Ratio 1:199
```

That single arithmetic calculation determines your metric choice, your splitting strategy and your class weighting.

---

4. Algebra Foundations

4.1 Vocabulary

In the expression **$3x^2 + 5x - 7$**:

Term	Which part	Role
Variable	x	An unknown or changing quantity

Term	Which part	Role
Coefficient	3, 5	The number multiplying a variable
Constant	−7	A fixed number, no variable attached
Term	$3x^2$, 5x, −7	Pieces separated by + or −
Expression	the whole thing	A combination with no equals sign
Equation	$3x^2 + 5x - 7 = 0$	Two expressions set equal

4.2 Simplifying, Expanding, Factorizing

```
Simplify:   3x + 5x − 2x          = 6x            (combine like terms)
Expand:     (x + 3)(x + 2)        = x² + 5x + 6   (multiply out)
Factorize:  x² + 5x + 6           = (x + 3)(x + 2) (the reverse)
```

Why this matters in ML. Deriving the gradient of a loss function is almost entirely expansion and simplification. If you cannot expand $(y - X\beta)^T(y - X\beta)$, you cannot derive the normal equation (Section 74).

4.3 Solving Equations

The governing principle: **whatever you do to one side, do to the other.**

```
      3x + 5 = 20
3x + 5 − 5 = 20 − 5     (subtract 5 from both sides)
        3x = 15
      3x/3 = 15/3        (divide both sides by 3)
         x = 5
```

4.4 Inequalities

Same rules, **with one trap**: multiplying or dividing by a negative number *flips* the inequality.

```
−2x > 6
  x < −3        (divided by −2, so > became <)
```

ML relevance. Probability constraints ($0 \le p \le 1$), constrained optimization, and decision thresholds (predict fraud if $p \ge 0.7$) are all inequalities.

4.5 An ML-Flavoured Worked Example

A linear model predicts churn score: **score = 0.8·support_calls − 0.3·tenure + 2**. At what number of support calls does a customer with 10 months tenure cross a score of 5?

```
0.8s − 0.3(10) + 2 = 5
0.8s − 3 + 2       = 5
0.8s − 1           = 5
```

```
0.8s                = 6
s                   = 7.5
```

So 8 support calls (the first integer above 7.5). That calculation is exactly what a **decision boundary** is (Section 15) — solved with nothing but algebra.

---

5. Linear Equations

5.1 The Line

```
y = mx + b          (school notation)
ŷ = β₀ + β₁x        (statistics / ML notation)
```

These are **the same equation**. Only the labels differ:

School	ML	Name	Meaning
m	β_1	Slope / coefficient	How much ŷ changes when x increases by 1
b	β_0	Intercept / bias	The value of ŷ when x = 0

5.2 Interpreting Slope and Intercept in Business Terms

A house-price model: **price = 5,000,000 + 250,000 × bedrooms**

- **β_0 = 5,000,000**: the baseline — what the model predicts for a zero-bedroom property. Often not physically meaningful, but mathematically necessary.
- **β_1 = 250,000**: each additional bedroom adds ₦250,000 to the predicted price.

This interpretability is exactly why linear regression survives in regulated industries despite being 200 years old. A credit officer can explain a decline decision by pointing at coefficients.

5.3 Computing Slope From Two Points

```
m = (y₂ − y₁) / (x₂ − x₁)     "rise over run"
```

Given (2, 8) and (5, 20):

```
m = (20 − 8) / (5 − 2) = 12 / 3 = 4
b: 8 = 4(2) + b  →  b = 0
Line: y = 4x
```

```python
x1, y1, x2, y2 = 2, 8, 5, 20
m = (y2 - y1) / (x2 - x1)
b = y1 - m * x1
print(f"y = {m}x + {b}")      # y = 4.0x + 0.0
```

5.4 Multiple Features

Real models have many inputs:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_m x_m$$

Which in vector form (Section 37) collapses to the elegant $\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\beta}$. That compression is *why* linear algebra matters: a formula with a hundred terms becomes two symbols.

---

6. Simultaneous Equations

6.1 The Problem

Two equations, two unknowns:

```
2x + 3y = 12
4x −  y = 10
```

6.2 Method 1 — Substitution

```
From (2):  y = 4x − 10
Sub into (1):  2x + 3(4x − 10) = 12
               2x + 12x − 30   = 12
               14x             = 42
               x               = 3
Then:          y = 4(3) − 10 = 2
Solution: (x, y) = (3, 2)
```

6.3 Method 2 — Elimination

```
(1):        2x + 3y = 12
(2) × 3:   12x − 3y = 30      (multiply so y-terms cancel)
Add:       14x      = 42  →  x = 3,  then y = 2
```

6.4 Graphical Interpretation

Each equation is a line. The solution is **where they intersect**. This tells you immediately when things go wrong:

Geometry	Algebra	ML meaning
Lines cross once	Unique solution	Well-posed problem
Lines are parallel	No solution	Inconsistent system
Lines are identical	Infinitely many solutions	**Redundant information — this is multicollinearity** (Section 47)

6.5 Matrix Representation — The Bridge to Linear Algebra

```
[ 2   3 ] [ x ]   [ 12 ]
[ 4  −1 ] [ y ] = [ 10 ]
     A       x   =    b
```

```
A = np.array([[2, 3], [4, -1]])
b = np.array([12, 10])
```

```
solution = np.linalg.solve(A, b)
print(solution)              # [3. 2.]
```

This is the single most important connection in the module. Solving `Ax = b` *is* fitting a linear regression (Section 74). Sections 39–45 develop the machinery; you have just seen why it exists.

---

7. Functions

7.1 Definition

A **function** is a rule assigning **exactly one output to each input**.

```
f: X → Y          "f maps from X to Y"
f(x) = 2x + 1     the rule
```

Term	Meaning	Example for $f(x)=\sqrt{x}$
Domain	All valid inputs	$x \geq 0$
Range	All possible outputs	$f(x) \geq 0$
Input	The argument	x
Output	The result	$f(x)$

Why domain matters in ML. $\log(x)$ requires $x > 0$. Log loss computes $\log(\hat{p})$. If your model ever outputs exactly 0, you get $\log(0) = -\infty$ and your training explodes. Every real implementation therefore clips probabilities into $[\varepsilon, 1-\varepsilon]$ (Section 84). That production bug is a *domain* violation.

7.2 Composition

```
(g ∘ f)(x) = g(f(x))        "apply f, then g"
```

Given $f(x) = 2x$ and $g(x) = x + 3$:

```
g(f(x)) = g(2x) = 2x + 3
f(g(x)) = f(x+3) = 2x + 6      ← note: NOT the same
```

Composition is the mathematical definition of a neural network. A 3-layer network is literally $f_3(f_2(f_1(x)))$. And the chain rule (Section 23) — which is how we differentiate compositions — is therefore the mathematical definition of backpropagation.

7.3 Inverse Functions

f^{-1} undoes f: $f^{-1}(f(x)) = x$.

Function	Inverse
$f(x) = 2x$	$f^{-1}(x) = x/2$
$f(x) = e^x$	$f^{-1}(x) = \ln(x)$
$f(x) = \sigma(x)$ (sigmoid)	$f^{-1}(x) = \ln(x/(1-x))$ — the **logit**

The logit is exactly how logistic regression converts a probability back into a linear score.

---

8. Important ML Functions

8.1 The Catalogue

Function	Formula	Range	Where it appears
Linear	$f(x) = ax + b$	$(-\infty, \infty)$	Linear regression, the pre-activation of every neuron
Polynomial	$f(x) = ax^2 + bx + c$	varies	Polynomial features, loss surfaces
Exponential	$f(x) = e^x$	$(0, \infty)$	Softmax, probability, decay schedules
Logarithmic	$f(x) = \ln(x)$	$(-\infty, \infty)$ for $x>0$	Log loss, entropy, likelihood
Sigmoid	$\sigma(x) = 1/(1+e^{-x})$	$(0, 1)$	Binary classification output
Softmax	$e^{xi}/\Sigma e^{xj}$	(0,1), sums to 1	Multi-class output
ReLU	$\max(0, x)$	$[0, \infty)$	Hidden layers in deep networks
Leaky ReLU	$\max(\alpha x, x)$, $\alpha \approx 0.01$	$(-\infty, \infty)$	Fixes ReLU's "dying neuron" problem
tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	$(-1, 1)$	Older networks, RNNs

8.2 Implementations

```
def linear(x, a=2, b=1):   return a * x + b
def sigmoid(x):            return 1 / (1 + np.exp(-x))
def relu(x):               return np.maximum(0, x)
def leaky_relu(x, a=0.01): return np.where(x > 0, x, a * x)
def tanh(x):               return np.tanh(x)

x = np.array([-2, -1, 0, 1, 2])
print("sigmoid   ", sigmoid(x).round(3))    # [0.119 0.269 0.5   0.731
0.881]
print("relu      ", relu(x))                # [0 0 0 1 2]
print("leaky_relu", leaky_relu(x))          # [-0.02 -0.01  0.    1.
2.  ]
print("tanh      ", tanh(x).round(3))       # [-0.964 -0.762  0.
0.762  0.964]
```

8.3 Why Activation Functions Exist At All

Stack two linear layers: $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$ — **still linear.** A hundred linear layers collapse into one. Without a non-linear activation between layers, depth buys you nothing.

That is the entire justification for ReLU, sigmoid and tanh. They inject the non-linearity that makes depth meaningful.

8.4 Why ReLU Won

Property	Sigmoid	ReLU
Computation	requires exp()	a comparison
Derivative	≤ 0.25, → 0 at extremes	exactly 1 for x > 0
Gradient through many layers	shrinks toward 0 (**vanishing gradients**)	passes through undiminished

Multiply 0.25 by itself ten times: 0.00000095. That is the vanishing gradient problem, and it is why deep networks were nearly untrainable before ReLU. ReLU's derivative of 1 means gradients survive depth (Section 101).

9. Exponents

9.1 The Laws

Law	Formula	Example
Product	$x^a \cdot x^b = x^{(a+b)}$	$2^3 \cdot 2^2 = 2^5 = 32$
Quotient	$x^a / x^b = x^{(a-b)}$	$2^5 / 2^2 = 2^3 = 8$
Power of power	$(x^a)^b = x^{(ab)}$	$(2^3)^2 = 2^6 = 64$
Zero	$x^0 = 1$	$7^0 = 1$
Negative	$x^{-a} = 1/x^a$	$2^{-3} = 1/8$
Fractional	$x^{(a/b)} = \sqrt[b]{(x^a)}$	$8^{(2/3)} = 4$

9.2 The Number e

e ≈ 2.71828, the base of natural exponential growth. It is not arbitrary: e^x is the unique function that is *its own derivative*.

$d/dx\ e^x = e^x$

That property is why e appears everywhere in calculus-based ML — it makes derivatives of sigmoid, softmax and log-likelihood tractable.

9.3 Growth and Decay

```
# Exponential decay — a learning-rate schedule
alpha_0, decay = 0.1, 0.95
for epoch in [0, 10, 50, 100]:
    print(f"epoch {epoch:3d}: lr = {alpha_0 * decay**epoch:.5f}")
# epoch   0: lr = 0.10000
# epoch  10: lr = 0.05987
# epoch  50: lr = 0.00769
# epoch 100: lr = 0.00059
```

Large steps early to move fast, tiny steps later to settle precisely — the standard schedule pattern (Section 31).

9.4 The Overflow Trap

```
print(np.exp(10))      # 22026.46
print(np.exp(100))     # 2.688e+43
print(np.exp(1000))    # inf  ← OVERFLOW
```

`float64` maxes out around 1.8e308, i.e. exp(~709). Any score above that produces `inf`, and `inf/inf = nan`, and your training silently dies. This is precisely why stable softmax subtracts the maximum first (Sections 12, 84).

---

10. Logarithms

10.1 Definition

A logarithm answers: **"what power do I raise the base to, to get this number?"**

```
log_b(x) = y    ⇔    b^y = x

log₁₀(1000) = 3        because 10³ = 1000
log₂(8)     = 3        because 2³  = 8
ln(e)       = 1        because e¹  = e    (ln = log base e, "natural
log")
```

10.2 The Log Rules — And Why ML Cares

Rule	Formula	Why it matters in ML
Product	log(ab) = log(a) + log(b)	**Converts products into sums** — the key to likelihood
Quotient	log(a/b) = log(a) − log(b)	Odds ratios, log-ratios
Power	$\log(a^b) = b \cdot \log(a)$	Pulls exponents down where they can be differentiated
Base change	log_b(x) = ln(x)/ln(b)	Converting between entropy units (bits vs nats)
Identity	$\ln(e^x) = x$, e^(ln x) = x	Undoing exponentials in derivations

10.3 The Product-to-Sum Trick — Why Likelihood Uses Logs

Likelihood multiplies probabilities across n samples:

$$L(\theta) = P(x_1|\theta) \cdot P(x_2|\theta) \cdot \ldots \cdot P(x_n|\theta) = \prod_i P(x_i|\theta)$$

With n = 1000 probabilities each ~0.5, that product is 0.5^1000 ≈ 10^{-301} — which underflows to exactly **0.0** in floating point. All information destroyed.

Taking logs:

$$\ln L(\theta) = \sum_i \ln P(x_i|\theta)$$

A *sum* of ~1000 negative numbers. Perfectly representable. And because ln is **monotonically increasing**, whatever θ maximizes L also maximizes ln L — so nothing is lost.

```python
p = np.full(1000, 0.5)
print(np.prod(p))            # 0.0                ← destroyed
print(np.sum(np.log(p)))     # -693.147...        ← preserved
```

This is the entire reason ML optimizes log-likelihood rather than likelihood. It is a numerical necessity, not a mathematical preference.

10.4 Negative Log-Likelihood

Optimizers minimize by convention, so we flip the sign:

```
maximize ln L(θ)    ⇔    minimize −ln L(θ)
```

−ln L is exactly what log loss / cross-entropy is (Section 71). The connection is not a coincidence — it is a definition.

10.5 Log Behaviour Worth Internalizing

```python
for v in [0.001, 0.01, 0.1, 0.5, 0.9, 0.99, 1.0]:
    print(f"log({v:5}) = {np.log(v):8.3f}")
# log(0.001) =  -6.908     ← confident AND wrong = huge penalty
# log(  0.5) =  -0.693
# log(  0.9) =  -0.105
# log(  1.0) =   0.000     ← perfectly right = zero penalty
```

In log loss, predicting 0.001 when the truth is 1 costs 6.9; predicting 0.9 costs 0.105. **Log loss punishes confident wrongness ferociously** — that asymmetry is a deliberate design property, and it is why log loss produces well-calibrated probabilities while accuracy does not.

---

11. Sigmoid Function

11.1 The Formula

$$\sigma(x) = 1 / (1 + e^{-x})$$

11.2 Why It Exists

A linear model outputs any real number: $z = \beta_0 + \beta_1 x_1 + \ldots \in (-\infty, \infty)$. But a probability must lie in $[0, 1]$. Sigmoid is the **squashing function** that maps the unbounded real line onto $(0, 1)$.

```
z = −5  → σ = 0.007      z = 0 → σ = 0.5      z = +5 → σ = 0.993
```

11.3 Properties

Property	Value	Consequence
Range	$(0, 1)$	Interpretable as probability — never exactly 0 or 1
$\sigma(0)$	0.5	$z = 0$ is the decision boundary
Symmetry	$\sigma(-x) = 1 - \sigma(x)$	$P(\text{class } 0) = 1 - P(\text{class } 1)$
Monotonic	always increasing	Higher score → higher probability, never reverses
Derivative	$\sigma'(x) = \sigma(x)(1 - \sigma(x))$	Remarkably simple — see below

11.4 Deriving the Derivative

This derivation is worth doing once because the result is used in every logistic regression and every sigmoid-activated network.

```
σ(x) = (1 + e⁻ˣ)⁻¹

Using the chain rule (Section 23):
σ'(x) = −1·(1 + e⁻ˣ)⁻² · d/dx(1 + e⁻ˣ)
      = −(1 + e⁻ˣ)⁻² · (−e⁻ˣ)
      = e⁻ˣ / (1 + e⁻ˣ)²

Now split it cleverly:
      = [1/(1 + e⁻ˣ)] · [e⁻ˣ/(1 + e⁻ˣ)]
      =     σ(x)      ·  [ (1 + e⁻ˣ − 1)/(1 + e⁻ˣ) ]
      =     σ(x)      ·  [ 1 − σ(x) ]

∴  σ'(x) = σ(x)(1 − σ(x))
```

Why this is beautiful and practical: to get the gradient you only need the value you *already computed* in the forward pass. No re-evaluation of exponentials. Every deep learning framework exploits this.

11.5 Implementation — Naive vs Stable

```python
def sigmoid_naive(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_stable(x):
    """Numerically stable for large-magnitude inputs."""
    return np.where(x >= 0,
                    1 / (1 + np.exp(-x)),                # x≥0: exp(-x) is small
                    np.exp(x) / (1 + np.exp(x)))         # x<0: exp(x) is small

def sigmoid_derivative(x):
    s = sigmoid_stable(x)
    return s * (1 - s)

x = np.array([-1000, -5, 0, 5, 1000])
print(sigmoid_stable(x))       # [0.    0.007 0.5   0.993 1.   ]
print(sigmoid_derivative(np.array([-5,0,5])).round(4))  # [0.0066 0.25 0.0066]
```

`sigmoid_naive(-1000)` computes `np.exp(1000)` → `inf` → warning and a wrong result. The stable version branches so the exponential argument is always ≤ 0.

11.6 The Vanishing Gradient, Quantified

The maximum of σ'(x) is **0.25** (at x = 0). In a deep network, backpropagation multiplies these derivatives layer by layer:

$$\text{10 layers: } 0.25^{10} \approx 0.00000095$$

The gradient reaching early layers is essentially zero, so those layers stop learning. This is the *mathematical* explanation of why sigmoid was abandoned for hidden layers in favour of ReLU (Section 8.4) — though it remains correct and standard for the **output** layer of a binary classifier, where there is no further backpropagation through it.

---

12. Softmax Function

12.1 The Formula

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

12.2 What It Does

Takes a vector of arbitrary real "scores" (logits) and returns a **probability distribution**: all values in (0,1), summing to exactly 1.

Worked example. Scores z = [2.0, 1.0, 0.1] for classes [Fraud, Suspicious, Normal]:

```
Step 1 — exponentiate:
  e^2.0 = 7.389
  e^1.0 = 2.718
  e^0.1 = 1.105

Step 2 — sum: 7.389 + 2.718 + 1.105 = 11.212

Step 3 — divide each by the sum:
  7.389/11.212 = 0.659
  2.718/11.212 = 0.242
  1.105/11.212 = 0.099
                 ─────
                 1.000  ✓
```

12.3 Why Exponentiate At All?

Three reasons: it makes every value **positive** (probabilities cannot be negative); it is **monotonic** (the largest score stays largest); and it **amplifies differences** — a score gap of 1.0 becomes a probability ratio of e ≈ 2.7, making the model's preference decisive rather than mushy.

12.4 Numerical Stability — The Max Subtraction Trick

```
z = np.array([1000, 1001, 1002])
print(np.exp(z))     # [inf inf inf]  → inf/inf = nan. Training destroyed.
```

The fix, and why it is mathematically free:

$$\frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{e^{z_i} \cdot e^{-C}}{\sum_j e^{z_j} \cdot e^{-C}} = \frac{e^{z_i - C}}{\sum_j e^{z_j - C}}$$

Multiplying numerator and denominator by the same constant e^{-C} changes nothing. Choosing $\mathbf{C = \max(z)}$ makes the largest exponent exactly 0, so $e^0 = 1$ and everything else is smaller. Overflow becomes impossible.

```
def softmax(z):
    """Numerically stable softmax."""
    z_shift = z - np.max(z)           # the crucial line
    exp_z   = np.exp(z_shift)
    return exp_z / np.sum(exp_z)

print(softmax(np.array([2.0, 1.0, 0.1])).round(3))   # [0.659 0.242 0.099]
print(softmax(np.array([1000., 1001., 1002.])).round(3))  # [0.09 0.245 0.665]
```

Both work. The second would be `[nan nan nan]` without the shift. **Every production softmax implementation in every framework contains this line** — you have just derived why.

12.5 Softmax vs Sigmoid

	Sigmoid	Softmax
Output	One probability	A distribution over k classes
Sums to 1?	N/A	Yes, always
Use	Binary classification; multi-**label**	Multi-**class** (mutually exclusive)

Softmax with k=2 is mathematically equivalent to sigmoid — sigmoid is the special case.

---

13. Sequences and Series

13.1 Arithmetic Sequences

Constant **difference** d: a, a+d, a+2d, ...

$$\text{nth term: } a_n = a_1 + (n-1)d$$
$$\text{Sum: } S_n = \frac{n}{2}(a_1 + a_n)$$

ML link: a linear learning-rate decay schedule is an arithmetic sequence.

13.2 Geometric Sequences

Constant **ratio** r: a, ar, ar^2, ...

```
nth term:  aₙ = a₁·r^(n−1)
Sum of infinite series (|r| < 1):  S = a₁/(1 − r)
```

ML links: exponential learning-rate decay (Section 9.3) is geometric. So is the **discounted return** in reinforcement learning (Section 94): $G = r_1 + \gamma r_2 + \gamma^2 r_3 + \ldots$ — a geometric series whose convergence for $\gamma < 1$ is precisely why discounting keeps infinite-horizon returns finite.

```python
gamma, rewards = 0.9, np.array([10, 10, 10, 10, 10])
discounts = gamma ** np.arange(len(rewards))
print(discounts.round(3))                    # [1.    0.9   0.81
0.729 0.656]
print(f"Discounted return: {(rewards * discounts).sum():.2f}")   #
40.95
print(f"Infinite horizon:  {10/(1-gamma):.2f}")                  #
100.00
```

13.3 Convergence

A sequence **converges** if it approaches a fixed value. Gradient descent produces a sequence of parameter values $\theta_0, \theta_1, \theta_2, \ldots$ and we want it to converge to θ^*.

```python
# Convergence check used in real training loops
losses = [10.0, 5.2, 3.1, 2.4, 2.1, 2.02, 2.005, 2.001]
tol = 1e-2
for i in range(1, len(losses)):
    if abs(losses[i] - losses[i-1]) < tol:
        print(f"Converged at iteration {i} (loss={losses[i]})")
        break
# Converged at iteration 5 (loss=2.02)
```

---

14. Summation Notation in Depth

14.1 The Statistics Ladder

These four formulas build on each other. Understanding them unlocks most of statistics.

Mean — add everything, divide by count:

$$\bar{x} = (1/n) \Sigma_{i=1}^{n} x_i$$

Deviations — how far each point is from the mean. *Always sums to zero*, which is why we cannot use it directly:

$$\Sigma_i (x_i - \bar{x}) = 0 \quad \text{always}$$

Variance — square the deviations first (killing the cancellation), then average:

$$\sigma^2 = (1/n)\ \Sigma_i\ (x_i - \bar{x})^2$$

Mean Squared Error — identical structure, but comparing prediction to truth instead of point to mean:

$$MSE = (1/n)\ \Sigma_i\ (y_i - \hat{y}_i)^2$$

Look at variance and MSE side by side. They are the same operation. Variance measures spread around the mean; MSE measures spread around the prediction. Once you see that, MSE stops being a formula to memorize.

14.2 Full Numerical Verification

```
x = np.array([2, 4, 4, 4, 5, 5, 7, 9])

mean = x.sum() / len(x)                          # 5.0
dev  = x - mean                                  # [-3 -1 -1 -1  0  0  2
4]
print("Deviations sum to:", dev.sum())           # 0  ← always
var  = (dev ** 2).sum() / len(x)                 # 4.0
std  = np.sqrt(var)                              # 2.0

print(mean, var, std)                            # 5.0 4.0 2.0
print(x.mean(), x.var(), x.std())                # identical — NumPy
confirms
```

14.3 Double Summation

$\Sigma_{i=1}^{n}\ \Sigma_{j=1}^{m}\ x_{ij}$ "sum over all rows and all columns"

```
X = np.array([[1, 2, 3],
              [4, 5, 6]])
print(X.sum())             # 21  — both indices
print(X.sum(axis=0))       # [5 7 9]   — sum over i (down columns)
print(X.sum(axis=1))       # [ 6 15]   — sum over j (across rows)
```

`axis=0` collapses rows; `axis=1` collapses columns. Getting this wrong is among the most common NumPy bugs in ML code.

---

15. Coordinate Geometry

15.1 From Points to Hyperplanes

Space	Object that divides it	Equation
1D (line)	A point	$x = c$
2D (plane)	A line	$ax + by = c$
3D	A plane	$ax + by + cz = d$
nD	A **hyperplane**	$w_1x_1 + \ldots + w_nx_n + b = 0$,

Space	Object that divides it	Equation
		i.e. $\mathbf{w^Sx + b = 0}$

A hyperplane is just "a flat thing one dimension below the space it lives in." In 2D you can picture it; in 300D you cannot, but the algebra is identical — which is the entire point of doing it algebraically.

15.2 The Decision Boundary

For a linear classifier:

```
wᵀx + b > 0   →  predict class 1
wᵀx + b < 0   →  predict class 0
wᵀx + b = 0   →  the decision boundary itself
```

```python
w = np.array([0.8, -0.3])     # weights for [support_calls, tenure]
b = 2.0

def classify(x):
    score = w @ x + b
    return score, int(score > 0)

print(classify(np.array([8, 10])))    # (5.4, 1)  → predicted churn
print(classify(np.array([1, 40])))    # (-9.2, 0) → predicted retain
```

This is exactly the algebra from Section 4.5, now in vector form. **Logistic regression, linear SVM and a single neuron all draw this same hyperplane** — they differ only in how they choose w and b.

15.3 Distance From a Point to a Hyperplane

```
distance = |wᵀx + b| / ||w||₂
```

The numerator is the raw score; dividing by the weight norm converts it to true geometric distance. **This formula is the heart of the Support Vector Machine**, which maximizes the distance from the boundary to the nearest points (the margin).

---

16. Distance Metrics

16.1 The Three Core Metrics

Euclidean (L2) — straight-line distance:

```
d(x,y) = √( Σᵢ (xᵢ − yᵢ)² )
```

Manhattan (L1) — city-block distance, only axis-aligned moves:

```
d(x,y) = Σᵢ |xᵢ − yᵢ|
```

Minkowski — the generalization:

```
d(x,y) = ( Σᵢ |xᵢ − yᵢ|^p )^(1/p)
```

p=1 gives Manhattan; p=2 gives Euclidean; p→∞ gives Chebyshev (the max coordinate difference).

16.2 Worked Example

x = (1, 2), y = (4, 6):

```
Euclidean: √((4−1)² + (6−2)²) = √(9 + 16) = √25 = 5
Manhattan: |4−1| + |6−2|      = 3 + 4      = 7
```

```
x = np.array([1, 2]); y = np.array([4, 6])

euclidean = np.sqrt(((x - y) ** 2).sum())     # 5.0
manhattan = np.abs(x - y).sum()               # 7
minkowski = (np.abs(x - y) ** 3).sum() ** (1/3)   # p=3 → 4.497

print(euclidean, manhattan, round(minkowski, 3))
print(np.linalg.norm(x - y))       # 5.0  — the idiomatic way
print(np.linalg.norm(x - y, 1))    # 7.0  — L1
```

16.3 The Critical ML Warning — Scale Dominates Distance

```
# Two customers: [age (years), income (naira)]
a = np.array([25, 2_000_000])
b = np.array([60, 2_000_100])

print(np.linalg.norm(a - b))     # 106.07
```

The age difference is **35 years** — enormous. The income difference is ₦100 — trivial. Yet the distance is 106, almost entirely driven by income. Age contributed 35 to a total dominated by the raw magnitude of the income scale.

Consequence: KNN and K-Means are meaningless on unscaled features. Whichever feature happens to have the largest numeric range silently becomes the only feature that matters. Standardization (Section 85) is not optional preprocessing here — it is a correctness requirement.

```
# After standardization both features contribute fairly
def standardize(X):
    return (X - X.mean(axis=0)) / X.std(axis=0)

X = np.array([[25, 2_000_000], [60, 2_000_100], [40, 2_000_050]],
dtype=float)
print(standardize(X).round(2))
```

16.4 Choosing a Metric

Metric	Best when
Euclidean	Continuous features, similar scales, low-to-moderate dimension

Metric	Best when
Manhattan	High dimensions (more robust — Section 86), or features measured on grids
Cosine	Text/embeddings where *direction* matters more than magnitude (Section 87)

17. Limits

17.1 The Idea

```
lim  f(x) = L
x→a
```

"As x gets arbitrarily close to a, f(x) gets arbitrarily close to L." Note: this says nothing about f(a) itself — the function need not even be defined there.

17.2 Why That Subtlety Matters

Consider $f(x) = (x^2 - 1)/(x - 1)$. At $x = 1$ this is $0/0$ — **undefined**. But factor the numerator:

```
(x² − 1)/(x − 1) = (x−1)(x+1)/(x−1) = x + 1    for x ≠ 1
```

So as $x \to 1$, $f(x) \to 2$, even though $f(1)$ does not exist.

```python
for h in [0.1, 0.01, 0.001, 0.0001]:
    x = 1 + h
    print(f"x={x:.4f}  f(x)={(x**2 - 1)/(x - 1):.6f}")
# x=1.1000  f(x)=2.100000
# x=1.0100  f(x)=2.010000
# x=1.0010  f(x)=2.001000
# x=1.0001  f(x)=2.000100   → clearly approaching 2
```

17.3 Why ML Needs Limits

The derivative is defined as a limit of a 0/0 expression. Without the limit concept, "instantaneous rate of change" is literally undefined — you cannot compute change over zero elapsed distance. Limits are the machinery that makes it rigorous. Everything in Sections 18–32 rests on this.

Limits also give us **continuity** (no sudden jumps), which matters because gradient descent assumes it can take small steps and get small changes.

---

18. Derivatives

18.1 The Question

Slope between two points is easy: rise/run. But what is the slope **at a single point**? Run = 0, so rise/run = 0/0.

The answer: take two points, compute the slope, then slide them infinitely close together.

18.2 The Definition From First Principles

$$f'(x) = \lim_{h \to 0} \frac{f(x + h) - f(x)}{h}$$

Part	Meaning
f(x + h) − f(x)	the **rise** — change in output
h	the **run** — change in input
lim h→0	shrink the gap to nothing

18.3 Deriving f'(x) for f(x) = x² By Hand

```
f'(x) = lim [ (x+h)² − x² ] / h
        h→0
      = lim [ x² + 2xh + h² − x² ] / h          (expand)
        h→0
      = lim [ 2xh + h² ] / h                     (x² cancels)
        h→0
      = lim [ h(2x + h) ] / h                    (factor h)
        h→0
      = lim (2x + h)                             (cancel h — legal since
h≠0)
        h→0
      = 2x                                       (now let h→0)

∴  d/dx x² = 2x
```

Every term in that derivation is algebra from Section 4. **Calculus is algebra plus one limit.**

18.4 Numerical Verification

```python
def numerical_derivative(f, x, h=1e-7):
    """Approximate f'(x) using the limit definition with tiny h."""
    return (f(x + h) - f(x)) / h

f = lambda x: x ** 2
for x in [1, 2, 3, 5]:
    print(f"x={x}: numerical={numerical_derivative(f, x):.4f}
exact={2*x}")
# x=1: numerical=2.0000  exact=2
# x=2: numerical=4.0000  exact=4
# x=3: numerical=6.0000  exact=6
# x=5: numerical=10.0000 exact=10
```

A practical warning on h. Too large and the approximation is poor; too small and floating-point subtraction cancellation destroys precision (Section 84). Around 1e-7 balances both for float64. The *central difference* `(f(x+h) − f(x−h))/(2h)` is more accurate and is what gradient-checking code uses.

18.5 Reading a Derivative

f'(x)	Meaning	Gradient descent behaviour
Positive	Function is increasing	Move **left** (subtract) to decrease
Negative	Function is decreasing	Move **right** (add) to decrease

f'(x)	Meaning	Gradient descent behaviour
Zero	Flat — minimum, maximum or saddle	Stop; nowhere to go locally
Large magnitude	Steep	Big step
Small magnitude	Shallow	Small step

That table is literally what the update rule $\theta := \theta - \alpha \cdot f'(\theta)$ implements. The minus sign is what turns "which way is uphill" into "go downhill."

---

19. Differentiation Rules

19.1 The Rule Table

Rule	Formula	Example
Constant	$d/dx(c) = 0$	$d/dx(7) = 0$
Power	$d/dx(x^n) = n \cdot x^{(n-1)}$	$d/dx(x^5) = 5x^4$
Constant multiple	$d/dx(c \cdot f) = c \cdot f'$	$d/dx(3x^2) = 6x$
Sum	$(f+g)' = f' + g'$	$d/dx(x^2+x^3) = 2x+3x^2$
Difference	$(f-g)' = f' - g'$	$d/dx(x^2-x) = 2x-1$
Product	$(fg)' = f'g + fg'$	see below
Quotient	$(f/g)' = (f'g - fg')/g^2$	see below
Chain	$(f(g(x)))' = f'(g(x)) \cdot g'(x)$	Section 23

19.2 Worked Examples

Power + sum + constant multiple:

$$f(x) = 3x^4 - 2x^2 + 5x - 7$$
$$f'(x) = 12x^3 - 4x + 5 - 0 = 12x^3 - 4x + 5$$

Product rule: $f(x) = x^2 \cdot e^x$

$$f' = (2x)(e^x) + (x^2)(e^x) = e^x(2x + x^2)$$

Quotient rule: $f(x) = (x^2 + 1)/(x - 3)$

$$f' = [\,(2x)(x-3) - (x^2+1)(1)\,] \,/\, (x-3)^2$$
$$= [\,2x^2 - 6x - x^2 - 1\,] \,/\, (x-3)^2$$
$$= (x^2 - 6x - 1)/(x-3)^2$$

19.3 The Most Important One for ML

The **MSE derivative** — you will meet this in every regression derivation:

$$L = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2, \qquad \text{where } \hat{y}_i = \theta x_i$$

```
dL/dθ = (1/n) Σ 2(yᵢ − θxᵢ)·(−xᵢ)      [chain rule]
      = −(2/n) Σ xᵢ(yᵢ − ŷᵢ)
```

Read it in words: **the gradient is the average of (feature × error)**. If errors correlate positively with a feature, that feature's coefficient must change. That interpretation is worth more than the symbols.

---

20. Derivatives of Important ML Functions

20.1 The Reference Table

f(x)	f'(x)	ML significance
x^2	$2x$	MSE loss
x^3	$3x^2$	Polynomial features
e^x	e^x	Self-derivative — why e is everywhere
$\ln(x)$	$1/x$	Log loss, entropy derivatives
$\sigma(x)$	$\sigma(x)(1-\sigma(x))$	Logistic regression, sigmoid layers
$\tanh(x)$	$1 - \tanh^2(x)$	RNN activations
ReLU(x)	1 if x>0, 0 if x<0 (undefined at 0)	Deep network hidden layers
Leaky ReLU	1 if x>0, α if x<0	Prevents dead neurons

20.2 Deriving ln(x)

Since $e^y = x$ when $y = \ln(x)$, differentiate both sides with respect to x:

```
eʸ · dy/dx = 1
dy/dx = 1/eʸ = 1/x
```

20.3 Deriving tanh

```
tanh(x) = (eˣ − e⁻ˣ)/(eˣ + e⁻ˣ)
tanh'(x) = 1 − tanh²(x)
```

Maximum value 1 (at x=0) — four times larger than sigmoid's 0.25, which is why tanh suffers less from vanishing gradients, though not enough to beat ReLU.

20.4 ReLU's Kink

```
ReLU(x) = max(0, x)
ReLU'(x) = 1 for x>0, 0 for x<0, **undefined at exactly x=0**
```

In practice frameworks define ReLU'(0) := 0. The probability of hitting exactly 0.0 in floating point is negligible, so this pragmatic choice causes no problems — a good example of engineering practice diverging harmlessly from strict mathematics.

20.5 Implementations

```
def d_sigmoid(x):
    s = 1/(1 + np.exp(-x)); return s * (1 - s)

def d_tanh(x):
    return 1 - np.tanh(x) ** 2

def d_relu(x):
    return (x > 0).astype(float)

x = np.array([-2., -0.5, 0., 0.5, 2.])
print("σ' ", d_sigmoid(x).round(4))   # [0.105 0.2350 0.25  0.235
0.105]
print("tanh'", d_tanh(x).round(4))    # [0.071 0.7864 1.    0.786
0.071]
print("relu'", d_relu(x))             # [0. 0. 0. 1. 1.]
```

Notice the maximum of each: sigmoid 0.25, tanh 1.0, ReLU 1.0. **That single column of numbers explains the historical evolution of activation functions.**

---

21. Partial Derivatives

21.1 The Idea

For a function of several variables, a **partial derivative** measures the rate of change with respect to *one* variable while **holding all others constant**.

```
∂f/∂x      "partial f with respect to x"
```

The curly ∂ (rather than d) signals that other variables exist and are being frozen.

21.2 Worked Example

```
f(x, y) = x²y + 3xy² + 2x

∂f/∂x:  treat y as a constant number
      = 2xy + 3y² + 2

∂f/∂y:  treat x as a constant number
      = x² + 6xy + 0 = x² + 6xy
```

Check at (x,y) = (2,3):

```
∂f/∂x = 2(2)(3) + 3(9) + 2 = 12 + 27 + 2 = 41
∂f/∂y = 4 + 6(2)(3) = 4 + 36 = 40

def f(x, y): return x**2 * y + 3*x*y**2 + 2*x

def partial(f, x, y, wrt, h=1e-6):
    if wrt == 'x': return (f(x+h, y) - f(x-h, y)) / (2*h)
```

```
    else:           return (f(x, y+h) - f(x, y-h)) / (2*h)

print(round(partial(f, 2, 3, 'x'), 4))   # 41.0
print(round(partial(f, 2, 3, 'y'), 4))   # 40.0
```

21.3 Why ML Requires Partial Derivatives

Every model has **many** parameters. A linear model with 50 features has 51 parameters. A small neural network has thousands. GPT-scale models have billions.

To improve the loss you must know: *how does the loss change if I nudge **this specific** parameter?* That is precisely a partial derivative — and you need one per parameter.

21.4 Linear Regression Partials — The Central Example

```
Model:  ŷᵢ = β₀ + β₁xᵢ
Loss:   L = (1/n) Σ (yᵢ − β₀ − β₁xᵢ)²

∂L/∂β₀ = −(2/n) Σ (yᵢ − ŷᵢ)
∂L/∂β₁ = −(2/n) Σ xᵢ(yᵢ − ŷᵢ)
```

In words: the intercept's gradient is the **average error**; the slope's gradient is the **average error weighted by the feature**. Both are intuitive — if errors are systematically positive, raise the intercept; if errors are large where x is large, adjust the slope.

```
x = np.array([1., 2., 3., 4., 5.])
y = np.array([2., 4., 5., 4., 5.])
b0, b1 = 0.0, 0.0
n = len(x)

y_hat = b0 + b1 * x
error = y - y_hat
grad_b0 = -(2/n) * error.sum()
grad_b1 = -(2/n) * (x * error).sum()
print(f"∂L/∂β₀ = {grad_b0:.3f}   ∂L/∂β₁ = {grad_b1:.3f}")
# ∂L/∂β₀ = -8.000   ∂L/∂β₁ = -26.800
```

Both gradients are negative → both parameters should *increase*. Gradient descent (Section 29) automates exactly this.

---

22. The Gradient

22.1 Definition

The **gradient** collects every partial derivative into a vector:

```
∇f = [ ∂f/∂x₁,  ∂f/∂x₂,  ...,  ∂f/∂xₙ ]ᵀ
```

Symbol ∇ is "nabla" or "del". **∇f is a vector; f is a scalar.**

22.2 The Two Properties That Matter

1. **Direction:** ∇f points in the direction of **steepest increase** of f.
2. **Magnitude:** ‖∇f‖ is how steep that increase is.

Therefore **−∇f points downhill fastest** — which is the entire justification for the minus sign in gradient descent.

22.3 The Hill Analogy

You are on a foggy hillside and want to reach the bottom. You cannot see, but you can feel the slope under your feet in every direction. The gradient is the direction of steepest *uphill*. So you step in the exact opposite direction. Repeat. That is gradient descent, and it is why the algorithm is *local* — you can only feel where you are standing, which is why non-convex surfaces (Section 32) can trap you in a local valley.

22.4 Worked Example

```
f(x,y) = x² + y²          (a bowl centred at the origin)
∇f = [2x, 2y]

At (3, 4):  ∇f = [6, 8],  ‖∇f‖ = √(36+64) = 10
```

Steepest ascent is direction [6,8] (i.e. away from the origin — correct, the bowl rises outward). Steepest descent is [−6,−8], pointing back at the origin, which is indeed the minimum.

```python
def f(v):     return v[0]**2 + v[1]**2
def grad_f(v): return np.array([2*v[0], 2*v[1]])

v = np.array([3., 4.])
g = grad_f(v)
print(g, np.linalg.norm(g))       # [6. 8.] 10.0

# Take one gradient descent step
alpha = 0.1
v_new = v - alpha * g
print(v_new, f(v), f(v_new))      # [2.4 3.2]  25.0  16.0  ← loss
decreased
```

22.5 Gradient at a Minimum

At a minimum, ∇f = **0** (the zero vector) — the surface is locally flat. This gives us both the *stopping criterion* for gradient descent and the *analytical condition* for solving optimization problems directly (set the gradient to zero and solve — exactly how the normal equation is derived in Section 74).

---

23. The Chain Rule

23.1 The Rule

For composite functions **f(g(x))**:

$$d/dx\ f(g(x)) = f'(g(x)) \cdot g'(x)$$

Or in Leibniz notation, which makes the "cancellation" intuition obvious:

$$dy/dx = (dy/du) \cdot (du/dx)$$

23.2 Worked Examples

Example 1: $y = (3x + 1)^5$

$$\text{Let } u = 3x + 1, \text{ so } y = u^5$$
$$dy/du = 5u^4 \ ; \ \ du/dx = 3$$
$$dy/dx = 5(3x+1)^4 \cdot 3 = 15(3x+1)^4$$

Example 2: $y = e^{(x^2)}$

$$u = x^2, \ y = e^u$$
$$dy/dx = e^{(x^2)} \cdot 2x$$

Example 3 (the ML one): $y = \sigma(wx + b)$, derivative with respect to w

$$u = wx + b \ ; \ \ y = \sigma(u)$$
$$dy/dw = \sigma'(u) \cdot x = \sigma(u)(1 - \sigma(u)) \cdot x$$

23.3 Chaining Through Multiple Layers

$$L = \text{loss}(a_3), \ \ a_3 = f(z_3), \ \ z_3 = W_3 a_2, \ \ a_2 = f(z_2), \ \ z_2 = W_2 a_1, \ \ldots$$

$$\partial L/\partial W_2 = (\partial L/\partial a_3)\cdot(\partial a_3/\partial z_3)\cdot(\partial z_3/\partial a_2)\cdot(\partial a_2/\partial z_2)\cdot(\partial z_2/\partial W_2)$$

This product of local derivatives, computed right to left, IS backpropagation (Section 91). There is no additional algorithm — backpropagation is the chain rule applied systematically with intermediate results cached.

23.4 A Tiny Computational Graph

$$x \longrightarrow[\times w]\longrightarrow z \longrightarrow[\sigma]\longrightarrow a \longrightarrow[\text{loss vs } y]\longrightarrow L$$

$$\text{Forward:} \quad z = wx \ ; \ \ a = \sigma(z) \ ; \ \ L = (a - y)^2$$

$$\text{Backward:} \quad \partial L/\partial a = 2(a - y)$$
$$\partial a/\partial z = a(1 - a)$$
$$\partial z/\partial w = x$$
$$\text{---------------------------------}$$
$$\partial L/\partial w = 2(a - y) \cdot a(1 - a) \cdot x$$

$$x, \ w, \ y = 2.0, \ 0.5, \ 1.0$$

```
z = w * x                          # 1.0
a = 1 / (1 + np.exp(-z))           # 0.7311
L = (a - y) ** 2                   # 0.0724

dL_da = 2 * (a - y)                # -0.5379
da_dz = a * (1 - a)                # 0.1966
dz_dw = x                          # 2.0
dL_dw = dL_da * da_dz * dz_dw      # -0.2115

print(f"L={L:.4f}  dL/dw={dL_dw:.4f}")
w_new = w - 0.5 * dL_dw            # gradient step
z2 = w_new*x; a2 = 1/(1+np.exp(-z2))
print(f"New loss: {(a2-y)**2:.4f}")   # lower than 0.0724 ✓
```

Why the vanishing gradient is now obvious. Each layer multiplies by another $\sigma' \leq 0.25$. Ten layers → factor of 0.25^{10}. The chain rule *is* the mechanism of the problem, not merely related to it.

---

24. Second Derivatives

24.1 Definition

The derivative of the derivative: $f''(x) = d/dx[f'(x)]$. It measures **curvature** — how fast the slope itself is changing.

24.2 Interpretation

$f''(x)$	Shape	Meaning
> 0	Convex ($\cup$, "cup")	Curving upward; a critical point here is a **minimum**
< 0	Concave ($\cap$, "cap")	Curving downward; a critical point here is a **maximum**
$= 0$	Inflection	Curvature changes sign

24.3 The Second Derivative Test

```
If f'(x₀) = 0 and f''(x₀) > 0  →  x₀ is a local minimum
If f'(x₀) = 0 and f''(x₀) < 0  →  x₀ is a local maximum
If f'(x₀) = 0 and f''(x₀) = 0  →  inconclusive
```

Example: $f(x) = x^3 - 3x^2$

```
f'(x) = 3x² − 6x = 3x(x−2)  →  critical points x=0, x=2
f''(x) = 6x − 6
f''(0) = −6 < 0  → local maximum at x=0
f''(2) = +6 > 0  → local minimum at x=2
```

```
f   = lambda x: x**3 - 3*x**2
fp  = lambda x: 3*x**2 - 6*x
```

```python
fpp = lambda x: 6*x - 6
for x in [0, 2]:
    kind = "minimum" if fpp(x) > 0 else "maximum"
    print(f"x={x}: f'={fp(x)}, f''={fpp(x)} → {kind}")
```

24.4 Why ML Cares

- **Convexity** (Section 32) is defined by f'' ≥ 0 everywhere — and convex problems have a *single* global minimum, which is why linear and logistic regression are so reliable.
- **Second-order optimizers** (Newton's method) use curvature to choose step sizes intelligently, converging in far fewer iterations than plain gradient descent.
- **The Hessian** (Section 27) is the multivariable generalization, and its eigenvalues distinguish minima from saddle points — which matters enormously in deep learning, where saddle points vastly outnumber local minima.

---

25. Multivariable Calculus

25.1 The Objects

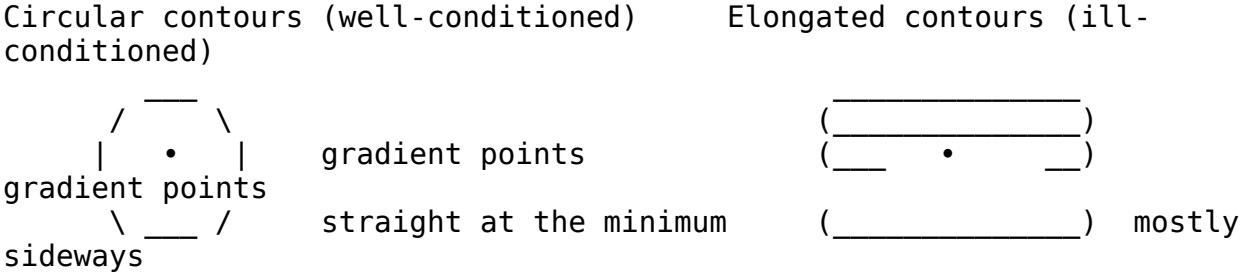
Object	Input → Output	ML example
Scalar function of a scalar	$\mathbb{R} \to \mathbb{R}$	A single-parameter loss
Scalar function of a vector	$\mathbb{R}^n \to \mathbb{R}$	**The loss function $L(\boldsymbol{\theta})$** — the central object
Vector function of a vector	$\mathbb{R}^n \to \mathbb{R}^m$	A neural network layer

Almost all of ML optimization concerns the middle row: many parameters in, one loss number out. That asymmetry is why the *gradient* (a vector of partials) is the natural tool.

25.2 Level Curves and Surfaces

A **level curve** connects all points with the same function value — a contour line on a map.

For $L(\theta_0, \theta_1) = \theta_0^2 + \theta_1^2$, the level curves are **circles**. For $L = \theta_0^2 + 10\theta_1^2$, they are **stretched ellipses**.



This picture explains why feature scaling matters for gradient descent. Unscaled features produce elongated valleys; the gradient points across the valley rather than along it, so GD zigzags and converges slowly. Standardization makes contours round and convergence fast (Section 85).

25.3 Gradient in Multiple Dimensions

```
def L(theta):              # elongated bowl — poorly conditioned
    return theta[0]**2 + 10*theta[1]**2
def grad_L(theta):
    return np.array([2*theta[0], 20*theta[1]])

theta = np.array([5., 5.])
print(grad_L(theta))       # [10. 100.]  ← wildly different magnitudes
```

A gradient of [10, 100] means one learning rate cannot suit both coordinates: large enough for θ_0 makes θ_1 explode; small enough for θ_1 makes θ_0 crawl. **That is the ill-conditioning problem, and adaptive optimizers like Adam (Section 83) exist to solve it.**

---

26. Directional Derivatives

26.1 Definition

The rate of change of f in an arbitrary **unit** direction **u**:

```
D_u f = ∇f · u          (a dot product)
```

26.2 Why the Gradient Is the Steepest Direction — The Proof

Since $\nabla f \cdot u = \|\nabla f\| \cdot \|u\| \cdot \cos(\varphi)$ and $\|u\| = 1$:

```
D_u f = ||∇f|| · cos(φ)
```

This is maximized when $\cos(\varphi) = 1$, i.e. $\boldsymbol{\varphi = 0}$, i.e. **u points the same way as ∇f**. And it is minimized when $\cos(\varphi) = -1$, i.e. $u = -\nabla f$.

That two-line argument is the complete mathematical justification for gradient descent. Every other direction gives less improvement per unit step.

26.3 Numerical Check

```
grad = np.array([3., 4.])                    # ||grad|| = 5

directions = {
    "along gradient":   np.array([3., 4.]) / 5,
    "opposite":         np.array([-3., -4.]) / 5,
    "perpendicular":    np.array([-4., 3.]) / 5,
    "45° off":          np.array([1., 0.]),
}
for name, u in directions.items():
    print(f"{name:16s}: D_u f = {grad @ u:+.3f}")
# along gradient  : D_u f = +5.000   ← maximum possible
# opposite        : D_u f = -5.000   ← minimum possible (best descent)
# perpendicular   : D_u f = +0.000   ← moving along a level curve
# 45° off         : D_u f = +3.000
```

Perpendicular to the gradient gives exactly zero change — you are walking along a contour line, confirming the geometry of Section 25.2.

---

27. The Hessian Matrix

27.1 Definition

The matrix of all **second-order** partial derivatives:

$$H = \begin{bmatrix} \partial^2 f/\partial x^2 & \partial^2 f/\partial x \partial y \\ \partial^2 f/\partial y \partial x & \partial^2 f/\partial y^2 \end{bmatrix}$$

For nice (twice continuously differentiable) functions, $\partial^2 f/\partial x \partial y = \partial^2 f/\partial y \partial x$, so **H is symmetric**.

27.2 Worked Example

```
f(x,y) = x² + 3xy + 2y²

First:   ∂f/∂x = 2x + 3y        ∂f/∂y = 3x + 4y
Second:  ∂²f/∂x² = 2            ∂²f/∂x∂y = 3
         ∂²f/∂y∂x = 3           ∂²f/∂y² = 4

H = [[2, 3],
     [3, 4]]
```

27.3 Classifying Critical Points With Eigenvalues

At a point where $\nabla f = 0$, the Hessian's **eigenvalues** (Section 50) tell you what kind of point it is:

Eigenvalues of H	Point type	Picture
All positive	**Local minimum** (H is positive definite)	Bowl
All negative	**Local maximum**	Dome
Mixed signs	**Saddle point**	Pringle — up one way, down another
Some zero	Degenerate, inconclusive	Flat direction

```
H = np.array([[2., 3.],
              [3., 4.]])
eigvals = np.linalg.eigvals(H)
print(eigvals.round(3))        # [-0.162  6.162]  ← mixed signs

if np.all(eigvals > 0):    print("minimum")
elif np.all(eigvals < 0):  print("maximum")
else:                      print("saddle point")   # ← this
```

27.4 Why Saddle Points Dominate Deep Learning

In n dimensions, a critical point is a minimum only if **all n** eigenvalues are positive. If signs were random, that has probability 2^{-n}. For n = 1,000,000 parameters, minima are astronomically rarer than saddle points.

This changed how the field thinks. For years people believed deep networks failed by getting stuck in bad local minima. The modern understanding is that saddle points — where the gradient vanishes but escape directions exist — are the real obstacle, and that momentum-based optimizers (Section 83) are effective partly because they carry enough velocity to roll off saddles.

27.5 Why We Rarely Compute the Hessian

For n parameters, H has $\mathbf{n^2}$ entries. With 1 million parameters that is 10^{12} numbers — roughly 8 terabytes in float64. Inverting it is worse still, at $O(n^3)$.

This is why deep learning uses **first-order** methods (gradient descent and its variants) despite second-order methods converging in fewer iterations: the per-iteration cost is prohibitive. Quasi-Newton methods (L-BFGS) compromise by *approximating* curvature from gradient history, and are practical for small-to-medium problems like logistic regression — which is exactly why scikit-learn's `LogisticRegression` uses `lbfgs` as its default solver.

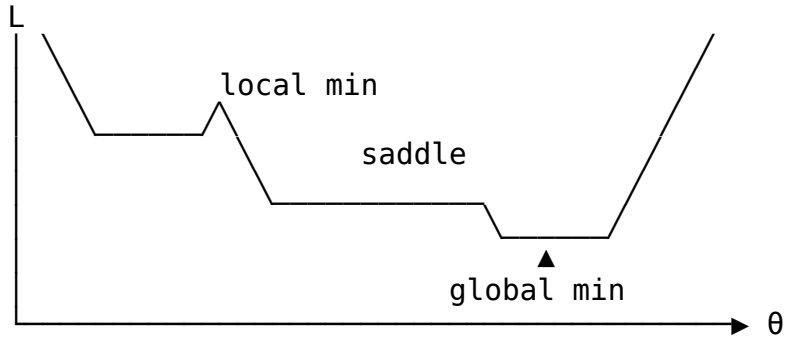
28. Optimization

28.1 The Vocabulary

Term	Meaning
Objective function	The quantity being optimized (general term)
Loss function	Error on a *single* example
Cost function	Loss averaged over the *dataset*
Minimization	Finding inputs that make the objective smallest
Maximization	Finding inputs that make it largest — note max f = min(−f)
Local minimum	Lower than all *nearby* points
Global minimum	The lowest point *anywhere*
Saddle point	Gradient is zero, but neither min nor max

In practice "loss" and "cost" are used interchangeably; the distinction only matters when reading careful texts.

28.2 The Landscape



Gradient descent finds *a* minimum — which one depends on where you start. For convex problems (Section 32) there is only one, so this concern evaporates.

28.3 Analytical vs Iterative Optimization

Analytical (closed-form): set ∇L = 0 and solve algebraically. Exact, instant — but only possible for simple models. Linear regression's normal equation (Section 74) is the canonical example.

Iterative: start somewhere, repeatedly step downhill. Works for *anything* differentiable, which is why it powers all of deep learning.

```
# Analytical: minimize f(x) = x² − 4x + 7
# f'(x) = 2x − 4 = 0  →  x = 2
f = lambda x: x**2 - 4*x + 7
print(f(2))         # 3.0  ← the minimum value
```

```python
# Iterative: same answer, found by stepping
x, alpha = 0.0, 0.1
for _ in range(50):
    x = x - alpha * (2*x - 4)
print(round(x, 6))  # 1.999996  ← converged to 2
```

Both give x = 2. The iterative method needed 50 steps to approximate what algebra gave exactly — but it will keep working when the algebra becomes impossible.

---

29. Gradient Descent

29.1 The Update Rule

$\theta := \theta - \alpha \nabla J(\theta)$

Symbol	Name	Role
θ	Parameters	What we are adjusting
:=	Assignment	"replace θ with this new value"
α	Learning rate	Step size (a hyperparameter *you* choose)
$\nabla J(\theta)$	Gradient of the cost	Direction of steepest *increase*
minus sign	—	Turns ascent into **descent**

Read aloud: *"New parameters equal old parameters minus a small step in the uphill direction."*

29.2 Fully Worked Single-Parameter Example

Minimize $J(\theta) = \theta^2$, starting at $\theta = 10$, with $\alpha = 0.1$. The gradient is $J'(\theta) = 2\theta$.

Iter	θ	$J(\theta)$	$\nabla J = 2\theta$	Update: $\theta - 0.1 \cdot \nabla J$
0	10.000	100.000	20.000	10 − 2.0 = 8.000
1	8.000	64.000	16.000	8 − 1.6 = 6.400
2	6.400	40.960	12.800	6.4 − 1.28 = 5.120
3	5.120	26.214	10.240	4.096
4	4.096	16.777	8.192	3.277
10	1.074	1.153	2.147	0.859
30	0.012	0.000	0.025	0.010

Notice: as θ approaches 0, the gradient shrinks, so the steps shrink automatically. **Gradient descent self-moderates** — it slows down as it nears the minimum without being told to.

```
def gradient_descent_1d(grad_fn, theta_init, alpha=0.1, n_iter=30):
    theta = theta_init
    history = [theta]
    for _ in range(n_iter):
        theta = theta - alpha * grad_fn(theta)
        history.append(theta)
    return theta, np.array(history)

theta_final, hist = gradient_descent_1d(lambda t: 2*t, 10.0)
print(f"Final θ: {theta_final:.6f}")     # Final θ: 0.012379
print(hist[:5].round(3))                  # [10.     8.     6.4
5.12   4.096]
```

29.3 Multi-Parameter Linear Regression From Scratch

```
def gradient_descent_linreg(X, y, alpha=0.01, n_iter=1000):
    """
    X : (n, m) feature matrix, WITHOUT an intercept column
    y : (n,)   target vector
    Returns intercept b, weights w, and loss history.
    """
    n, m = X.shape
    w = np.zeros(m)          # initialize weights at zero
    b = 0.0                  # initialize intercept at zero
    history = []

    for _ in range(n_iter):
        y_hat = X @ w + b                    # forward pass:
predictions
        error = y_hat - y                    # note the sign
convention
        loss  = (error ** 2).mean()          # MSE
        history.append(loss)

        grad_w = (2/n) * (X.T @ error)       # ∂L/∂w — feature-
weighted error
        grad_b = (2/n) * error.sum()         # ∂L/∂b — average error

        w -= alpha * grad_w                  # the update rule
        b -= alpha * grad_b

    return b, w, np.array(history)

# Synthetic data: true relationship y = 3 + 2x₁ − 1x₂ + noise
rng = np.random.default_rng(42)
X = rng.normal(size=(200, 2))
y = 3 + 2*X[:, 0] - 1*X[:, 1] + rng.normal(scale=0.1, size=200)
```

```
b, w, hist = gradient_descent_linreg(X, y, alpha=0.05, n_iter=500)
print(f"Intercept: {b:.3f}  (true 3)")      # Intercept: 3.001  (true 3)
print(f"Weights:   {w.round(3)}  (true [2, -1])")   # Weights: [ 1.995 -1.002]
print(f"Loss: {hist[0]:.3f} → {hist[-1]:.5f}")      # Loss: 14.512 → 0.00985
```

Line-by-line notes on the critical parts:

- `X @ w + b` — the model, vectorized. One matrix operation replaces a loop over 200 samples.
- `error = y_hat - y` — using (prediction − truth) rather than (truth − prediction) removes a minus sign from the gradient formulas. Both conventions appear in textbooks; be consistent.
- `X.T @ error` — this single expression computes $\Sigma_i x_{ij} \text{error}_i$ for *every* feature j simultaneously. It is the vectorized form of the partial derivative from Section 21.4, and it is why linear algebra is indispensable.
- `w -= alpha * grad_w` — the update, applied to all weights at once.

29.4 Convergence Monitoring

```
print(hist[[0, 10, 50, 100, 300, 499]].round(5))
# [14.51203  4.7351   0.06934  0.01167  0.00985  0.00985]
```

The loss falls steeply, then flattens. **A flat tail means convergence** — further iterations change nothing. Real training loops use this to stop early:

```
tol = 1e-8
for i in range(1, len(hist)):
    if abs(hist[i] - hist[i-1]) < tol:
        print(f"Converged at iteration {i}")
        break
```

---

30. Batch, Stochastic and Mini-Batch Gradient Descent

30.1 The Three Variants

Variant	Samples per update	Updates per epoch
Batch GD	All n	1
Stochastic GD (SGD)	1	n
Mini-batch GD	b (e.g. 32, 64, 256)	n/b

30.2 Comparison

Property	Batch	SGD	Mini-batch
Gradient accuracy	Exact	Very noisy	Good approximation
Speed per update	Slow	Very fast	Fast

Property	Batch	SGD	Mini-batch
Memory	Entire dataset	One sample	One batch
Convergence path	Smooth, direct	Erratic, bouncing	Moderately smooth
Escapes saddles/shallow minima	Poorly	Well (noise helps)	Well
Hardware efficiency	Good	**Poor** (no parallelism)	**Best** (GPU-friendly)
Practical usage	Small data only	Rare in pure form	**The standard everywhere**

30.3 Implementation

```
def minibatch_gd(X, y, alpha=0.05, batch_size=32, epochs=50, seed=0):
    n, m = X.shape
    w, b = np.zeros(m), 0.0
    rng = np.random.default_rng(seed)
    losses = []

    for epoch in range(epochs):
        idx = rng.permutation(n)             # shuffle each epoch — 
important
        for start in range(0, n, batch_size):
            batch = idx[start:start + batch_size]
            Xb, yb = X[batch], y[batch]

            error  = (Xb @ w + b) - yb
            grad_w = (2/len(batch)) * (Xb.T @ error)
            grad_b = (2/len(batch)) * error.sum()
            w -= alpha * grad_w
            b -= alpha * grad_b

        losses.append((((X @ w + b) - y) ** 2).mean())
    return b, w, np.array(losses)

b, w, losses = minibatch_gd(X, y, epochs=30)
print(f"b={b:.3f}, w={w.round(3)}")       # b=3.001, w=[ 1.995 -1.003]
print(f"Loss: {losses[0]:.4f} → {losses[-1]:.5f}")
```

Why shuffle every epoch? Without it, the model sees batches in the same order repeatedly, which can introduce systematic bias — particularly if the data was sorted by class or by time.

30.4 Why Mini-Batch Won

Pure SGD updates once per sample, which cannot exploit vectorized hardware — a GPU capable of thousands of parallel operations sits idle processing one row. Batch GD uses hardware well but updates too rarely on large data. Mini-batch hits the sweet spot: enough samples to fill the

hardware, enough updates to converge quickly, and enough gradient noise to escape poor regions.

Note on terminology: in modern usage, "SGD" almost always means *mini-batch* SGD. `torch.optim.SGD` and Keras's `SGD` both operate on batches.

---

31. Learning Rate

31.1 The Most Consequential Hyperparameter

α	Behaviour	Symptom in the loss curve
Too small	Tiny steps; correct but glacially slow	Loss decreases, barely
Good	Steady, efficient descent	Smooth decline, flattening
Too large	Overshoots the minimum, oscillates	Loss bounces up and down
Far too large	Diverges to infinity	Loss → `inf` or `nan`

31.2 Numerical Demonstration

Minimizing $J(\theta) = \theta^2$ from $\theta = 10$, gradient 2θ:

```
for alpha in [0.01, 0.1, 0.9, 1.01]:
    theta = 10.0
    path = [theta]
    for _ in range(20):
        theta = theta - alpha * 2 * theta
        path.append(theta)
    status = "DIVERGED" if abs(theta) > 1e6 else f"θ={theta:.4f}"
    print(f"α={alpha:<5}: {status}")

# α=0.01 : θ=6.6761      ← too small: barely moved in 20 steps
# α=0.1  : θ=0.0115      ← good: converged nicely
# α=0.9  : θ=0.0115      ← oscillating but converging (sign flips each
step)
# α=1.01 : DIVERGED      ← exploded
```

The threshold is mathematical, not mystical. For $J = \theta^2$, the update is $\theta(1 - 2\alpha)$. This shrinks only if $|1 - 2\alpha| < 1$, i.e. $\mathbf{0 < \alpha < 1}$. At $\alpha = 1.01$ the multiplier is -1.02, so magnitude grows by 2% each step — exponential divergence.

31.3 Learning Rate Schedules

Schedule	Formula	Rationale
Constant	α	Simple; often adequate
Step decay	$\alpha \times \gamma$ every k epochs	Big steps early, fine steps later

Schedule	Formula	Rationale
Exponential decay	$\alpha_0 \cdot e^{(-kt)}$	Smooth version of the above
1/t decay	$\alpha_0/(1 + kt)$	Classic, satisfies theoretical convergence conditions
Cosine annealing	$\alpha_0 \cdot \frac{1}{2}(1 + \cos(\pi t/T))$	Popular in deep learning; smooth descent to near zero
Warmup + decay	Rise then fall	Standard for training Transformers

```
epochs = np.arange(0, 100)
step  = 0.1 * (0.5 ** (epochs // 30))
expo  = 0.1 * np.exp(-0.03 * epochs)
cos   = 0.1 * 0.5 * (1 + np.cos(np.pi * epochs / 100))
for e in [0, 30, 60, 99]:
    print(f"epoch {e:3d}: step={step[e]:.5f}  exp={expo[e]:.5f}
cos={cos[e]:.5f}")
```

31.4 Practical Guidance

Start with **0.01** or **0.001**. Plot the loss curve. If it descends too slowly, multiply α by 3; if it oscillates or explodes, divide by 3. Tune on a **logarithmic scale** (0.0001, 0.001, 0.01, 0.1) rather than linearly — the behaviour changes by order of magnitude, not by increments.

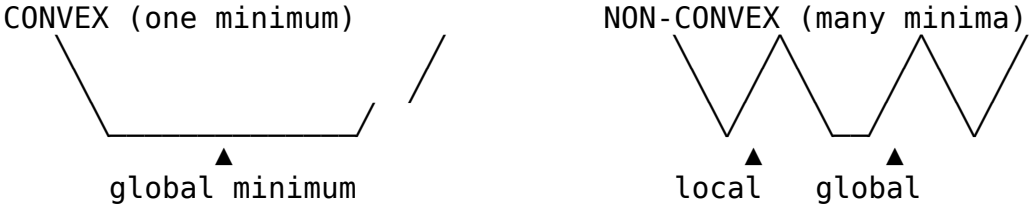
Scaled features make learning rates transferable. On unscaled data the right α differs wildly between features (Section 25.3); after standardization one α works for all.

---

32. Convexity

32.1 Definition

A function is **convex** if the straight line between any two points on its graph never dips below the graph.



Formally: $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$ for all $\lambda \in [0,1]$.

Equivalently, for twice-differentiable functions: **$f''(x) \geq 0$ everywhere** (or, in multiple dimensions, the Hessian is positive semi-definite).

32.2 Why Convexity Is Wonderful

Property	Consequence for ML
Any local minimum is the **global** minimum	Gradient descent cannot get trapped

Property	Consequence for ML
The solution is unique (for strict convexity)	Reproducible — same data, same answer
Initialization does not matter	No need for random restarts
Convergence is guaranteed	With a suitable learning rate

32.3 Convexity of ML Loss Functions

Model + loss	Convex?	Implication
Linear regression + MSE	**Yes**	Closed-form solution exists; GD always finds it
Logistic regression + log loss	**Yes**	Reliable, reproducible convergence
Linear SVM + hinge loss	**Yes**	Global optimum guaranteed
Ridge / Lasso	**Yes**	The penalty preserves convexity
Neural networks	**No**	Many minima; results depend on initialization and seed
K-Means objective	**No**	Different initializations give different clusters — hence `k-means++` and `n_init`

This table explains a practical mystery. Why does training the same neural network twice give different results, while logistic regression gives identical results every time? Convexity. It is also why deep learning cares so much about initialization schemes and random seeds, while classical convex models do not.

32.4 Checking Convexity Numerically

```
def is_convex_1d(f, lo, hi, n=1000):
    """Check f'' >= 0 across a range using finite differences."""
    x = np.linspace(lo, hi, n)
    h = x[1] - x[0]
    second = (f(x[2:]) - 2*f(x[1:-1]) + f(x[:-2])) / h**2
    return np.all(second >= -1e-6)      # small tolerance for float
error

print(is_convex_1d(lambda x: x**2, -5, 5))              # True  —
convex
print(is_convex_1d(lambda x: x**4 - 3*x**2, -3, 3))     # False —
non-convex
print(is_convex_1d(lambda x: np.exp(x), -3, 3))         # True
print(is_convex_1d(lambda x: np.log(1 + np.exp(x)), -5, 5))  # True —
softplus
```

32.5 The Deep Learning Paradox

Neural network losses are non-convex with astronomically many minima — yet deep learning works spectacularly. The modern explanation: in very high dimensions, **most local minima are nearly as good as the global one**, and the truly bad critical points are saddle points which stochastic gradient noise escapes (Sections 27.4, 30.2). Non-convexity is a theoretical concern that turns out to be practically survivable — which is itself a useful lesson about the gap between theory and empirical practice.

33–35. Linear Algebra: Scalars, Vectors, and Why It All Matters

33.1 The Object Hierarchy

Object	Dimensions	Notation	NumPy shape	ML example
Scalar	0	x	`()`	Learning rate $\alpha = 0.01$
Vector	1	$\mathbf{x}$	`(n,)`	One customer's features
Matrix	2	$\mathbf{X}$	`(n, m)`	The whole dataset
Tensor	3+	$\mathscr{X}$	`(n, h, w, c)`	A batch of colour images

```
scalar = np.array(5.0)
vector = np.array([1., 2., 3.])
matrix = np.array([[1., 2.], [3., 4.], [5., 6.]])
tensor = np.zeros((32, 28, 28, 3))       # 32 RGB images, 28×28

for name, obj in [("scalar", scalar), ("vector", vector),
                  ("matrix", matrix), ("tensor", tensor)]:
    print(f"{name:7s} shape={str(obj.shape):16s} ndim={obj.ndim}")
```

33.2 Why Linear Algebra Is Fundamental to ML

Three reasons, each decisive:

1. **Data is naturally a matrix.** n rows × m features. Every dataset you have ever loaded into Pandas is a matrix.
2. **Models are transformations.** A neural network layer is literally **Wx + b** — a matrix multiplication plus a vector addition.
3. **Speed.** Vectorized operations run in optimized C/Fortran (BLAS/LAPACK) and exploit SIMD instructions and GPUs. Loops in Python do not.

```
import time
n = 1_000_000
a, b = np.random.rand(n), np.random.rand(n)

t0 = time.perf_counter()
loop_result = sum(a[i] * b[i] for i in range(n))
t_loop = time.perf_counter() - t0

t0 = time.perf_counter()
vec_result = a @ b
t_vec = time.perf_counter() - t0

print(f"Loop:       {t_loop:.4f}s")
print(f"Vectorized: {t_vec:.6f}s")
```

```
print(f"Speedup:    {t_loop/t_vec:.0f}×")
# Typically 100–500× faster
```

This speedup is not a convenience — it is what makes modern ML possible at all. Training GPT-scale models with Python loops would take geological time.

34.1 Scalars

A single number. Examples in ML: learning rate α, regularization strength λ, bias term b, a loss value, a single feature value, a probability.

35.1 Vectors

An ordered list of numbers. Written as a **column** by convention:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n$$

$x \in \mathbb{R}^n$ reads: "x is a vector of n real numbers."

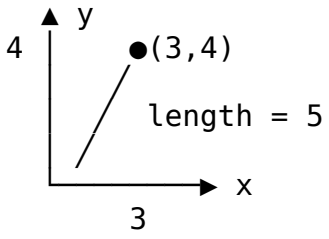
35.2 What Vectors Represent in ML

Interpretation	Example
Feature vector	One customer: [34, 850000, 18, 2] = [age, income, tenure, calls]
Parameter vector	Model weights: [0.3, −1.2, 0.8]
Embedding	A word/product/user as 300 learned numbers (Section 93)
Gradient	The direction to move each parameter
A point in space	Geometric view — the basis of distance and similarity

35.3 The Two Views of a Vector

```
ALGEBRAIC: an ordered list
   [3, 4]
```

```
GEOMETRIC: an arrow in space
```



Both views are essential: the algebraic view lets you compute, the geometric view lets you *understand* why cosine similarity measures semantic closeness (Section 87).

35.4 Row vs Column Vectors in NumPy

```
v  = np.array([1., 2., 3.])          # shape (3,)   — 1-D, neither row nor column
```

```
col = v.reshape(-1, 1)                # shape (3,1)  — explicit column
row = v.reshape(1, -1)                # shape (1,3)  — explicit row

print(v.shape, col.shape, row.shape)   # (3,) (3, 1) (1, 3)
print(v @ v)                           # 14.0 — dot product of 1-D
arrays
print(col.T @ col)                     # [[14.]] — a 1×1 matrix, not a
scalar
```

A common source of bugs: NumPy's `(3,)` is *not* the same as `(3,1)`. Broadcasting will sometimes silently produce a `(3,3)` matrix when you expected a `(3,)` vector. Always print `.shape` when debugging.

---

36. Vector Operations

36.1 The Operations

Operation	Formula	Requirement
Addition	$(x+y)_i = x_i + y_i$	Same shape
Subtraction	$(x-y)_i = x_i - y_i$	Same shape
Scalar multiplication	$(cx)_i = c \cdot x_i$	—
Element-wise (Hadamard) product	$(x \odot y)_i = x_i y_i$	Same shape
Dot product	$x \cdot y = \Sigma\, x_i y_i$	Same length → returns a **scalar**

36.2 Numerical Examples

```
x = np.array([1., 2., 3.])
y = np.array([4., 5., 6.])

print(x + y)        # [5. 7. 9.]
print(x - y)        # [-3. -3. -3.]
print(3 * x)        # [3. 6. 9.]
print(x * y)        # [ 4. 10. 18.]   ← element-wise, NOT dot product
print(x @ y)        # 32.0            ← dot product: 1·4 + 2·5 + 3·6
print(np.dot(x, y)) # 32.0            ← same
```

The single most common NumPy error in ML code is confusing `x * y` (element-wise) with `x @ y` (dot product). They are different operations returning different shapes. * broadcasts; @ contracts.

36.3 Geometric Meaning

- **Addition** places arrows tip-to-tail.
- **Scalar multiplication** stretches ($|c|>1$), shrinks ($|c|<1$) or flips ($c<0$) the arrow.
- **Subtraction** $x - y$ gives the arrow *from* y *to* x — which is exactly why distance is $\|x - y\|$ (Section 16).

37. The Dot Product

37.1 Two Equivalent Definitions

```
ALGEBRAIC:   x · y = Σᵢ xᵢyᵢ = x₁y₁ + x₂y₂ + ... + xₙyₙ

GEOMETRIC:   x · y = ||x|| ||y|| cos(φ)      where φ is the angle between
them
```

The fact that these are the same is one of the most useful results in applied mathematics — it links arithmetic to geometry.

37.2 What the Sign Tells You

x·y	Angle	Meaning
Large positive	φ ≈ 0°	Vectors point the **same** way
Zero	φ = 90°	**Orthogonal** — no relationship
Large negative	φ ≈ 180°	Vectors point **opposite** ways

```python
a = np.array([1., 0.])
for name, b in [("same",   np.array([1., 0.])),
                ("45°",    np.array([1., 1.])/np.sqrt(2)),
                ("orthog", np.array([0., 1.])),
                ("opposite",np.array([-1., 0.]))]:
    print(f"{name:9s}: dot={a @ b:+.3f}")
# same     : dot=+1.000
# 45°      : dot=+0.707
# orthog   : dot=+0.000
# opposite : dot=-1.000
```

37.3 Why the Dot Product Is Everywhere in ML

Linear regression / logistic regression:

```
ŷ = wᵀx + b        ← a dot product between weights and features
```

A single neuron:

```
z = wᵀx + b, then a = f(z)
```

Similarity in embeddings: the dot product (normalized, giving cosine similarity — Section 87) measures semantic closeness between words, documents, users and products.

Attention in Transformers: attention scores are QK^T — a matrix of dot products measuring how much each token should attend to each other token (Section 92).

One operation, four foundational applications. Understanding the dot product deeply pays back more than almost any other single concept in this module.

37.4 Worked ML Example

```
# A fraud model's weights and one transaction's features
w = np.array([0.5, -0.2, 1.8, 0.3])   # [amount_z, tenure_z,
night_flag, foreign_flag]
b = -1.0
x = np.array([2.1, -0.5, 1.0, 1.0])   # a large, late-night, foreign
transaction

z = w @ x + b
p = 1 / (1 + np.exp(-z))
print(f"score z = {z:.3f}, P(fraud) = {p:.3f}")
# score z = 2.255, P(fraud) = 0.905
```

Trace the contributions: the night flag contributed 1.8, the amount 1.05, foreign 0.3, tenure +0.1, minus the bias 1.0. **That decomposition is exactly how you explain a linear model's decision to a regulator.**

---

38. Vector Norms

38.1 The Three Norms

```
L1 (Manhattan):   ‖x‖₁ = Σ |xᵢ|
L2 (Euclidean):   ‖x‖₂ = √(Σ xᵢ²)
L∞ (Max):         ‖x‖∞ = maxᵢ |xᵢ|
```

A **norm** is a measure of "size" or "length" that is always ≥ 0, is 0 only for the zero vector, and satisfies the triangle inequality.

38.2 Worked Example

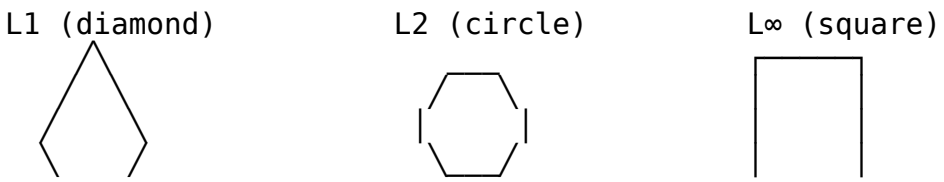
For x = [3, −4, 5]:

```
‖x‖₁ = 3 + 4 + 5 = 12
‖x‖₂ = √(9 + 16 + 25) = √50 ≈ 7.071
‖x‖∞ = max(3, 4, 5) = 5
```

```
x = np.array([3., -4., 5.])
print(np.linalg.norm(x, 1))       # 12.0
print(np.linalg.norm(x))          # 7.0711  (L2 is the default)
print(np.linalg.norm(x, np.inf))  # 5.0
```

38.3 The Geometry — And Why L1 Creates Sparsity

The "unit ball" ‖x‖ = 1 has a different shape for each norm:





The L1 diamond has sharp corners on the axes. When an optimizer minimizes loss subject to an L1 constraint, the solution tends to land on a corner — and corners are points where *some coordinates are exactly zero*. The L2 circle has no corners, so solutions land anywhere on the smooth boundary with all coordinates small but non-zero.

This geometric fact is the complete explanation of why Lasso (L1) performs feature selection while Ridge (L2) does not (Section 72). It is rare for a picture to explain a machine learning behaviour so precisely.

38.4 ML Applications of Norms

Use	Norm
Ridge regularization penalty	$\|\beta\|_2^2$
Lasso regularization penalty	$\|\beta\|_1$
Euclidean distance in KNN/K-Means	$\|x - y\|_2$
Gradient clipping in deep learning	rescale if $\|\nabla\|_2 >$ threshold
Normalizing embeddings for cosine similarity	$x / \|x\|_2$

```
# Gradient clipping — prevents exploding gradients (Section 101)
def clip_gradient(g, max_norm=1.0):
    norm = np.linalg.norm(g)
    return g * (max_norm / norm) if norm > max_norm else g

g = np.array([30., 40.])          # norm 50 — exploding
print(clip_gradient(g))           # [0.6 0.8] — direction preserved,
norm = 1
```

Note that clipping preserves the *direction* and only caps the *magnitude* — the optimizer still moves the right way, just not catastrophically far.

39–43. Matrices: Notation Through Identity

39.1 Matrix Notation

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} \quad \in \mathbb{R}^{2\times 3} \qquad \text{"2 rows, 3 columns"}$$

a_{ij} is the entry in row i, column j. **Rows first, always.**

39.2 The ML Dataset as a Matrix

$$\begin{array}{c} \\ c_1 \\ c_2 \end{array} \begin{array}{c} \begin{array}{cccc} \text{age} & \text{income} & \text{tenure} & \text{calls} \end{array} \\ \begin{bmatrix} 34 & 850000 & 18 & 2 \\ 27 & 420000 & 6 & 7 \end{bmatrix} \end{array} \quad X \in \mathbb{R}^{n\times m}$$

```
  c₃ │  55  1200000     42       0  │
  c₄ └  41   680000     24       3  ┘
```

n rows = samples. m columns = features. This convention is universal in ML and is why `X.shape` is `(n_samples, n_features)` in every scikit-learn function.

```
X = np.array([[  34,  850000, 18, 2],
              [  27,  420000,  6, 7],
              [  55, 1200000, 42, 0],
              [  41,  680000, 24, 3]], dtype=float)
print(X.shape)          # (4, 4)  → 4 customers, 4 features
print(X[0])             # first customer (a row)
print(X[:, 1])          # income column across all customers
```

40.1 Matrix Operations

```
A = np.array([[1., 2.], [3., 4.]])
B = np.array([[5., 6.], [7., 8.]])

print(A + B)      # [[ 6.  8.] [10. 12.]]   element-wise
print(A - B)      # [[-4. -4.] [-4. -4.]]
print(2 * A)      # [[2. 4.] [6. 8.]]
print(A * B)      # [[ 5. 12.] [21. 32.]]   ← ELEMENT-WISE (Hadamard)
print(A @ B)      # [[19. 22.] [43. 50.]]   ← MATRIX MULTIPLICATION
```

40.2 * vs @ — The Critical Distinction

Operator	Name	Requirement	Result shape
`A * B`	Element-wise / Hadamard	Same shape (or broadcastable)	Same as inputs
`A @ B`	Matrix multiplication	`A.shape[1] == B.shape[0]`	`(A.rows, B.cols)`

These produce completely different numbers. In the example above, `A*B` gave `[[5,12], [21,32]]` while `A@B` gave `[[19,22],[43,50]]`. **Using the wrong one is the most common silent bug in hand-written ML code** — silent because both often run without error.

41.1 Matrix Multiplication Derived

`C = A @ B` where $c_{ij} = \Sigma_k \, a_{ik} \cdot b_{kj}$

In words: **entry (i,j) of the result is the dot product of row i of A with column j of B.**

Full worked example:

```
A = [1 2]    B = [5 6]
    [3 4]        [7 8]

c₁₁ = row1(A)·col1(B) = (1)(5) + (2)(7) = 5 + 14 = 19
c₁₂ = row1(A)·col2(B) = (1)(6) + (2)(8) = 6 + 16 = 22
c₂₁ = row2(A)·col1(B) = (3)(5) + (4)(7) = 15 + 28 = 43
c₂₂ = row2(A)·col2(B) = (3)(6) + (4)(8) = 18 + 32 = 50
```

```
C = [19 22]
    [43 50]
```

41.2 Dimensional Compatibility

```
(m × n) @ (n × p) = (m × p)
      ↑       ↑
   these must match — they are "contracted away"
```

```
A = np.random.rand(3, 4)
B = np.random.rand(4, 5)
print((A @ B).shape)      # (3, 5)  ✓ inner dims 4 and 4 match

C = np.random.rand(3, 5)
try:
    A @ C
except ValueError as e:
    print("Error:", e)    # matmul: ... size 4 is different from 3
```

Shape errors are the most frequent runtime failure in ML code. The habit that fixes it: write the shapes as a comment above every matrix operation.

41.3 Matrix Multiplication Is Not Commutative

```
print(A2 := np.array([[1.,2.],[3.,4.]]) @ np.array([[5.,6.],[7.,8.]]))
print(np.array([[5.,6.],[7.,8.]]) @ np.array([[1.,2.],[3.,4.]]))
# [[19. 22.] [43. 50.]]   vs   [[23. 34.] [31. 46.]]  — different!
```

AB ≠ BA. This is why the order of layers in a neural network matters, and why you cannot casually rearrange matrix expressions in derivations.

41.4 The ML Applications

Linear regression on a whole dataset at once:

```
X = np.random.rand(1000, 5)        # 1000 samples, 5 features
beta = np.random.rand(5)           # 5 coefficients
y_hat = X @ beta                   # 1000 predictions in ONE operation
print(y_hat.shape)                 # (1000,)
```

A neural network layer:

```
batch = np.random.rand(32, 784)    # 32 images, 784 pixels each
W1 = np.random.rand(784, 128)      # layer 1 weights
b1 = np.random.rand(128)
h1 = np.maximum(0, batch @ W1 + b1)   # ReLU(XW + b)
print(h1.shape)                    # (32, 128)
```

Transformer attention (Section 92): QK^T is a matrix multiplication producing every pairwise token similarity simultaneously.

42.1 Transpose

Flip rows and columns: $(A^S)_{ij} = A_{ji}$

```
A = np.array([[1., 2., 3.],
              [4., 5., 6.]])         # (2, 3)
print(A.T)                            # (3, 2)
# [[1. 4.]
#  [2. 5.]
#  [3. 6.]]
print(A.T.shape)                      # (3, 2)
```

Properties: $(A^T)^T = A$, $(A+B)^T = A^T+B^T$, and crucially $(AB)^S = B^S A^S$ (note the reversal).

Where you meet it: `X.T @ X` in the normal equation (Section 74); `X.T @ error` in the gradient (Section 29.3); QK^T in attention.

43.1 The Identity Matrix

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

1s on the diagonal, 0s elsewhere

AI = IA = A — the identity is to matrices what 1 is to numbers.

```
I = np.eye(3)
A = np.random.rand(3, 3)
print(np.allclose(A @ I, A))      # True
```

Its importance: the **inverse** is defined by $AA^{-1} = I$ (Section 44), and the identity appears explicitly in Ridge regression's solution $(X^SX + \lambda I)^{-1}X^Sy$, where adding λI is precisely what makes an otherwise-singular matrix invertible (Section 72).

44. Inverse Matrices

44.1 Definition

A^{-1} is the matrix satisfying $\mathbf{AA^{-1} = A^{-1}A = I}$. It is the matrix analogue of a reciprocal.

44.2 The 2×2 Formula

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \qquad A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

The quantity **ad − bc** is the **determinant** (Section 46). If it is zero, you are dividing by zero — **the inverse does not exist**.

44.3 When the Inverse Fails to Exist

A matrix is **singular** (non-invertible) when:

- det(A) = 0
- Rows or columns are **linearly dependent** (Section 48)
- The matrix is not square
- Rank < n (Section 47)

In ML this happens constantly, for concrete reasons:

Cause	Example
Duplicate features	`price_naira` and `price_thousands` (perfectly correlated)
Redundant encoding	One-hot encoding *all* k categories without dropping one (the "dummy variable trap")
More features than samples	10,000 genes measured on 100 patients
Perfectly correlated engineered features	`total = part_a + part_b` alongside both parts

44.4 Why You Should Rarely Compute an Inverse Explicitly

This is one of the most practically valuable lessons in the entire module.

```
A = np.array([[4., 7.], [2., 6.]])
b = np.array([1., 2.])

x_inv   = np.linalg.inv(A) @ b        # DON'T do this
x_solve = np.linalg.solve(A, b)       # DO this
print(x_inv, x_solve)                 # both ≈ [-0.8  0.6]
```

Three reasons `solve` is better:

1. **Numerical accuracy.** Computing the inverse then multiplying introduces two rounding stages; `solve` uses LU decomposition in one stage, accumulating less error.

2. **Speed.** Inversion is roughly 2–3× the work of solving directly.
3. **Robustness.** `solve` degrades more gracefully on near-singular matrices and gives clearer errors.

```
# Demonstration on an ill-conditioned matrix
H = np.array([[1., 1/2, 1/3], [1/2, 1/3, 1/4], [1/3, 1/4, 1/5]])  # Hilbert 3×3
b = np.array([1., 2., 3.])
x1 = np.linalg.inv(H) @ b
x2 = np.linalg.solve(H, b)
print("Residual via inv:  ", np.abs(H @ x1 - b).max())
print("Residual via solve:", np.abs(H @ x2 - b).max())
# solve typically gives a smaller residual
print("Condition number:", np.linalg.cond(H).round(1))   # ~524 — already poor
```

The condition number measures how much a matrix amplifies input errors. Above ~1e10 is dangerous; above ~1e15 the result is essentially noise in float64.

---

45. Linear Systems

45.1 The Canonical Problem

```
Ax = b
```

Given A and b, find x. This single equation form covers an enormous range of problems — including, as Section 74 shows, fitting a linear regression.

45.2 Three Possible Outcomes

Case	Geometry	Detection
Unique solution	Lines/planes meet at one point	det(A) ≠ 0, full rank
No solution	Parallel, never meeting	Inconsistent system
Infinitely many	Identical lines/planes	Rank deficient

45.3 Solving in NumPy

```
# 2x + 3y +  z = 10
#  x -  y + 2z =  5
# 3x + 2y -  z =  4
A = np.array([[2., 3., 1.],
              [1., -1., 2.],
              [3., 2., -1.]])
b = np.array([10., 5., 4.])

x = np.linalg.solve(A, b)
print(x.round(4))                    # [0.7143 2.4286 1.1429]
print(np.allclose(A @ x, b))         # True — verification
```

Always verify with `np.allclose(A @ x, b)`. It costs one line and catches silent failures.

45.4 Overdetermined Systems — The ML Case

In machine learning, we almost always have **more equations than unknowns** (more samples than features). Such a system usually has *no exact solution* — no line passes through all 1000 points.

The answer is **least squares**: find the x minimizing $\|Ax - b\|_2$. This is exactly what linear regression does.

```
rng = np.random.default_rng(0)
A = rng.normal(size=(100, 3))            # 100 equations
b = A @ np.array([2., -1., 0.5]) + rng.normal(scale=0.1, size=100)

x, residuals, rank, sv = np.linalg.lstsq(A, b, rcond=None)
print(x.round(3))      # [ 2.001 -0.997  0.503]  ← recovered the true coefficients
print("rank:", rank)   # 3 — full rank, well-posed
```

`np.linalg.lstsq` is the **numerically stable, professionally correct** way to fit linear regression. It uses SVD internally and handles rank-deficient matrices gracefully rather than crashing — which is exactly why scikit-learn's `LinearRegression` calls it rather than implementing the normal equation with an explicit inverse.

---

46. Determinants

46.1 Definition and Computation

2×2:

```
det [a b]  =  ad − bc
    [c d]
```

3×3 (cofactor expansion along the first row):

```
det = a(ei − fh) − b(di − fg) + c(dh − eg)
```

46.2 Geometric Meaning

The determinant is the factor by which the matrix scales area (2D) or volume (3D).

det(A)	Geometric effect
2	Doubles area
1	Preserves area (a rotation, say)
0	**Collapses to zero area** — flattens the space
Negative	Scales *and* flips orientation

det = 0 means the transformation squashes space into a lower dimension. Information is destroyed, and destroyed information cannot be recovered — which is precisely why a zero determinant means non-invertible.

```
A = np.array([[3., 0.], [0., 2.]])
print(np.linalg.det(A))          # 6.0 — scales area by 6

B = np.array([[1., 2.], [2., 4.]])    # row 2 = 2 × row 1
print(np.linalg.det(B))          # 0.0 — singular, collapses to a line
```

46.3 Properties

Property	Formula
Invertible ⟺	$\det \neq 0$
Product	$\det(AB) = \det(A)\cdot\det(B)$
Transpose	$\det(A^T) = \det(A)$
Inverse	$\det(A^{-1}) = 1/\det(A)$
Identity	$\det(I) = 1$

46.4 A Practical Caution

Do not test singularity with `det(A) == 0`. Floating point almost never produces exactly zero, and determinants of large matrices overflow or underflow badly. **Use the condition number or rank instead:**

```
A = np.array([[1., 2.], [2., 4.0000001]])
print(np.linalg.det(A))          # 1e-07 — "not zero" but practically
singular
print(np.linalg.cond(A))         # ~5e7  — clearly ill-conditioned
print(np.linalg.matrix_rank(A))  # 2, but numerically fragile
```

---

47. Rank

47.1 Definition

The **rank** of a matrix is the number of linearly independent rows (equivalently, columns). It measures how much *genuine, non-redundant* information the matrix contains.

```
rank(A) ≤ min(rows, columns)
```

Full rank means rank equals that maximum. **Rank deficient** means less.

47.2 Examples

```
A = np.array([[1., 2.], [3., 4.]])
print(np.linalg.matrix_rank(A))     # 2 — full rank

B = np.array([[1., 2.], [2., 4.]])  # row 2 = 2 × row 1
print(np.linalg.matrix_rank(B))     # 1 — rank deficient
```

```
C = np.array([[1., 2., 3.],
              [4., 5., 6.],
              [5., 7., 9.]])        # row 3 = row 1 + row 2
print(np.linalg.matrix_rank(C))     # 2 — the third row adds nothing
```

47.3 Rank and Multicollinearity — The ML Connection

Multicollinearity is when features are linearly related. Perfect multicollinearity makes X^TX singular and the normal equation unsolvable.

```
# A realistic scenario: a redundant engineered feature
n = 100
rng = np.random.default_rng(1)
age     = rng.integers(20, 70, n).astype(float)
income  = rng.normal(500_000, 100_000, n)
income_k = income / 1000                    # ← same information,
different units!

X_bad = np.column_stack([np.ones(n), age, income, income_k])
print("rank:", np.linalg.matrix_rank(X_bad), "of", X_bad.shape[1])   #
rank: 3 of 4
print("cond:", f"{np.linalg.cond(X_bad.T @ X_bad):.2e}")             #
astronomically large

try:
    np.linalg.inv(X_bad.T @ X_bad)
    print("inverted (but the result is numerical garbage)")
except np.linalg.LinAlgError as e:
    print("Singular matrix:", e)
```

Symptoms of multicollinearity in a fitted model: wildly large coefficients with opposite signs; coefficients that change dramatically when you add or remove a single sample; huge standard errors; and a model that predicts adequately but whose coefficients are uninterpretable nonsense.

Remedies: drop redundant features; combine them; use PCA (Section 52); or apply **Ridge regularization**, which adds λI to X^TX and thereby guarantees invertibility (Section 72). That last connection — regularization as a *numerical* fix as well as a statistical one — is worth remembering.

---

48. Linear Independence

48.1 Definition

Vectors $v_1, \ldots, v_k$ are **linearly independent** if the only solution to

$$c_1v_1 + c_2v_2 + \ldots + c_kv_k = 0$$

is $c_1 = c_2 = \ldots = c_k = 0$.

If any non-trivial combination gives zero, they are **dependent** — meaning at least one vector can be written in terms of the others, and therefore adds no new information.

48.2 Examples

```
# Independent: neither is a multiple of the other
v1, v2 = np.array([1., 0.]), np.array([0., 1.])
print(np.linalg.matrix_rank(np.column_stack([v1, v2])))    # 2 ✓

# Dependent: v4 = 2·v3
v3, v4 = np.array([1., 2.]), np.array([2., 4.])
print(np.linalg.matrix_rank(np.column_stack([v3, v4])))    # 1 ✗

# Dependent: v7 = v5 + v6
v5, v6, v7 = np.array([1.,0.,0.]), np.array([0.,1.,0.]),
np.array([1.,1.,0.])
print(np.linalg.matrix_rank(np.column_stack([v5,v6,v7])))  # 2 ✗
```

48.3 The Dummy Variable Trap

This is the most common accidental linear dependence in real ML work:

```
# One-hot encoding a 3-category feature, keeping ALL columns
onehot_all = np.array([[1.,0.,0.],
                       [0.,1.,0.],
                       [0.,0.,1.],
                       [1.,0.,0.]])
X = np.column_stack([np.ones(4), onehot_all])    # intercept + all 3
dummies

print("rank:", np.linalg.matrix_rank(X), "of", X.shape[1])   # rank: 3
of 4
# Because col1 + col2 + col3 = the intercept column, exactly.
```

The three dummy columns always sum to 1, which is exactly the intercept column — perfect dependence. **The fix is `drop_first=True`** in `pd.get_dummies` or `drop='first'` in scikit-learn's `OneHotEncoder`. You lose no information (the dropped category is the reference level) and you restore full rank.

---

49. Basis and Vector Spaces

49.1 Definitions

Term	Meaning
Vector space	A set of vectors closed under addition and scalar multiplication
Span	All vectors reachable as linear combinations

Term	Meaning
	of a given set
Basis	A *minimal* set that spans the space — independent AND spanning
Dimension	The number of vectors in a basis

49.2 The Standard Basis

In $\mathbb{R}^3$: e_1=[1,0,0], e_2=[0,1,0], e_3=[0,0,1]. Any vector decomposes uniquely:

$[3, 5, 2] = 3e_1 + 5e_2 + 2e_3$

49.3 Alternative Bases — The Idea Behind PCA

The standard basis is not the only one, nor always the best. A different basis describes the same data with different coordinates.

```
v = np.array([3., 4.])

# Standard basis coordinates: just (3, 4)
# A rotated basis:
u1 = np.array([1., 1.]) / np.sqrt(2)
u2 = np.array([-1., 1.]) / np.sqrt(2)

c1, c2 = v @ u1, v @ u2          # project onto the new basis
print(c1.round(4), c2.round(4))  # 4.9497  0.7071
print((c1*u1 + c2*u2).round(4))  # [3. 4.]  — reconstructed exactly
```

Notice: in this rotated basis, the first coordinate (4.95) carries far more of the vector's magnitude than the second (0.71). If we discarded the second coordinate we would lose little.

That is precisely PCA (Section 52): find the basis where the first few coordinates capture almost all the variance, then discard the rest.

49.4 Feature Space

Your dataset's m features define an m-dimensional feature space, and each sample is a point in it. Multicollinearity (Section 47) means your data actually lives in a *lower-dimensional subspace* than m — which is both a numerical problem and an opportunity for dimensionality reduction.

50–51. Eigenvalues and Eigenvectors

50.1 The Defining Equation

$Av = \lambda v$

Symbol	Name	Meaning
A	Matrix	A linear transformation

Symbol	Name	Meaning
v	Eigenvector	A special direction ($v \neq 0$)
λ	Eigenvalue	A scalar

In words: an eigenvector is a direction that the transformation **does not rotate** — it only stretches or shrinks it, by the factor λ.

50.2 The Intuition

Most vectors get both rotated and rescaled by a matrix. Eigenvectors are the exceptional directions that stay on their own line.

```
Generic vector:                    Eigenvector:
   v ──► Av                           v ──► Av
   ↗        ↖  (rotated)              ↗      ↗   (same direction,
longer)
```

50.3 Finding Them By Hand

```
Av = λv  ⇒  (A − λI)v = 0

For a non-zero v to exist, (A − λI) must be singular:
        det(A − λI) = 0        ← the "characteristic equation"
```

Worked example:

```
A = [4  1]
    [2  3]

det([4−λ    1  ]) = (4−λ)(3−λ) − 2 = λ² − 7λ + 10 = (λ−5)(λ−2) = 0
   ([ 2   3−λ ])

λ₁ = 5, λ₂ = 2

For λ=5:  (A − 5I)v = [−1  1][v₁] = 0  →  v₁ = v₂  →  v = [1, 1]
                      [ 2 −2][v₂]

For λ=2:  (A − 2I)v = [ 2  1][v₁] = 0  →  2v₁ = −v₂ →  v = [1, −2]
                      [ 2  1][v₂]
```

```python
A = np.array([[4., 1.], [2., 3.]])
eigvals, eigvecs = np.linalg.eig(A)
print(eigvals.round(4))          # [5. 2.]
print(eigvecs.round(4))
# [[ 0.7071 -0.4472]
#  [ 0.7071  0.8944]]     ← columns are eigenvectors (normalized to
length 1)
```

```
# Verify Av = λv for the first pair
v = eigvecs[:, 0]
print(np.allclose(A @ v, eigvals[0] * v))    # True ✓
```

NumPy returns **unit-length** eigenvectors, so [0.707, 0.707] is the normalized version of [1, 1] — same direction, length 1.

51.1 Properties

Property	Statement
Count	An n×n matrix has n eigenvalues (counting multiplicity)
Trace	$\Sigma\lambda_i$ = sum of diagonal entries
Determinant	$\Pi\lambda_i$ = det(A)
Symmetric matrices	Always have **real** eigenvalues and **orthogonal** eigenvectors
Singular $\Longleftrightarrow$	At least one $\lambda = 0$

The symmetric-matrix property is why PCA works so cleanly — covariance matrices are always symmetric, guaranteeing real eigenvalues and perpendicular principal components.

51.2 Where Eigen-Decomposition Appears in ML

Application	Role
PCA	Eigenvectors of the covariance matrix = principal components; eigenvalues = variance captured
Hessian analysis	Eigenvalue signs classify minima vs saddle points (Section 27.3)
Spectral clustering	Eigenvectors of a graph Laplacian reveal cluster structure
PageRank	The ranking is the dominant eigenvector of the link matrix
Stability analysis	Largest eigenvalue governs whether an iterative process converges

52. PCA Mathematics

52.1 The Goal

Find a new set of axes (a new basis, Section 49.3) such that the first few axes capture as much of the data's variance as possible — then keep only those, discarding the rest.

52.2 The Algorithm Derived

Step 1 — Centre the data. Subtract the mean of each feature.

```
X_centered = X − mean(X)
```

Why: variance is measured *about the mean*. Without centering, the first component would point toward the data's mean location rather than its direction of spread.

Step 2 — Compute the covariance matrix.

```
C = (1/(n−1)) · X_centeredᵀ X_centered        C ∈ ℝ^(m×m), symmetric
```

Why: C encodes how features vary together — exactly the structure we want to exploit.

Step 3 — Eigendecompose C.

```
C vᵢ = λᵢ vᵢ
```

Why: the eigenvector with the largest eigenvalue is the direction of maximum variance. (This is provable: maximizing v^TCv subject to $\|v\|=1$ yields exactly the top eigenvector.)

Step 4 — Sort by eigenvalue, descending. Largest variance first.

Step 5 — Project.

```
X_reduced = X_centered @ V_k        where V_k holds the top k
eigenvectors
```

Step 6 — Report explained variance.

```
explained_variance_ratio_i = λᵢ / Σλⱼ
```

52.3 Full NumPy Implementation

```python
def pca(X, n_components=2):
    """PCA from scratch. Returns projected data, components, explained
variance."""
    # Step 1: centre
    X_mean = X.mean(axis=0)
    Xc = X - X_mean

    # Step 2: covariance matrix (features × features)
    C = (Xc.T @ Xc) / (len(X) - 1)

    # Step 3: eigendecomposition — eigh for symmetric matrices
(faster, more stable)
    eigvals, eigvecs = np.linalg.eigh(C)

    # Step 4: sort descending (eigh returns ascending)
    order   = np.argsort(eigvals)[::-1]
    eigvals = eigvals[order]
    eigvecs = eigvecs[:, order]

    # Step 5: project onto the top k components
    components = eigvecs[:, :n_components]
    X_reduced  = Xc @ components
```

```
    # Step 6: explained variance
    evr = eigvals / eigvals.sum()

    return X_reduced, components, evr

# Correlated synthetic data
rng = np.random.default_rng(42)
x1 = rng.normal(0, 1, 300)
x2 = 2*x1 + rng.normal(0, 0.3, 300)      # strongly correlated with x1
x3 = rng.normal(0, 0.5, 300)             # independent noise
X = np.column_stack([x1, x2, x3])

X_red, comps, evr = pca(X, n_components=2)
print("Explained variance ratio:", evr.round(4))
# [0.9316 0.0654 0.003 ]
print(f"First 2 components retain {evr[:2].sum():.1%} of variance")
# First 2 components retain 99.7% of variance
print("Reduced shape:", X_red.shape)     # (300, 2) — from 3 columns
to 2
```

Reading the output: the first component alone captures 93% of the variance, because x1 and x2 are nearly the same information. PCA discovered that redundancy automatically and compressed 3 dimensions into 2 with 0.3% information loss.

52.4 Why PCA Reduces Dimensionality Without Destroying Information

Real data is rarely "full rank in spirit" — features correlate, so the data cloud is shaped like a flat pancake or a thin cigar in the original space rather than filling it. PCA rotates the axes to align with the pancake, at which point the thin directions are obviously discardable.

Two critical practical notes:

1. **Always standardize first if features have different units.** PCA maximizes *variance*, and a feature measured in naira (variance ~10^{10}) will trivially dominate one measured in years (variance ~100) regardless of its actual importance.

```
X_std = (X - X.mean(axis=0)) / X.std(axis=0)     # do this before PCA
```

2. **Principal components are not interpretable features.** PC1 might be "0.6×income − 0.3×age + 0.7×tenure" — mathematically optimal, but meaningless to a business stakeholder. PCA trades interpretability for compression, which is why it is often the wrong choice in regulated contexts (Module 1, Part 40).

---

53. Covariance

53.1 Definition

$$\mathrm{Cov}(X,Y) = (1/(n-1)) \Sigma_i (x_i - \bar{x})(y_i - \bar{y})$$

Sign	Meaning
Positive	When X is above its mean, Y tends to be above its mean too
Negative	When X is above its mean, Y tends to be below
Near zero	No *linear* relationship

Note the (n−1): this is Bessel's correction, giving an unbiased estimate of the population covariance from a sample. NumPy's `np.cov` uses n−1 by default; `np.var` uses n by default. That inconsistency causes real bugs.

53.2 Worked Example

```
x = np.array([2., 4., 6., 8., 10.])
y = np.array([1., 3., 7., 8., 11.])

xbar, ybar = x.mean(), y.mean()          # 6.0, 6.0
cov = ((x - xbar) * (y - ybar)).sum() / (len(x) - 1)
print(cov)                                 # 12.0
print(np.cov(x, y)[0, 1])                  # 12.0 ✓

print(np.cov(x, y))
# [[10. 12.]
#  [12. 15.]]
#  diagonal = variances; off-diagonal = covariance
```

53.3 The Covariance Matrix

For m features, the covariance matrix $C \in \mathbb{R}^{m \times m}$ has $C_{ij} = \text{Cov}(\text{feature}_i, \text{feature}_j)$, with variances on the diagonal. It is **always symmetric** — which is exactly the property PCA relies on (Section 51.1).

53.4 The Limitation That Motivates Correlation

Covariance is **unit-dependent**. Covariance between income (naira) and age (years) is measured in "naira-years" — a quantity with no intuitive scale. Change income to thousands of naira and the covariance drops by 1000× despite the relationship being identical.

```
income = np.array([500_000., 600_000., 700_000., 800_000.])
age    = np.array([25., 30., 35., 40.])
print(np.cov(income, age)[0,1])            # 4166666.67
print(np.cov(income/1000, age)[0,1])       # 4166.67  — same
relationship!
```

You cannot tell from "4,166,666" whether the relationship is strong. That is what correlation fixes.

---

54. Correlation

54.1 Pearson Correlation

$$r = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \cdot \sigma_Y} \qquad \in [-1, +1]$$

Correlation is covariance normalized by the standard deviations — which cancels the units and bounds the result.

54.2 Interpretation

r	Relationship
+1.0	Perfect positive linear
+0.7 to +0.9	Strong positive
+0.3 to +0.7	Moderate positive
~0	No linear relationship
−0.7 to −1.0	Strong to perfect negative

54.3 Computation

```
def pearson(x, y):
    return np.cov(x, y)[0,1] / (x.std(ddof=1) * y.std(ddof=1))

x = np.array([2., 4., 6., 8., 10.])
y = np.array([1., 3., 7., 8., 11.])
print(round(pearson(x, y), 4))           # 0.9798
print(np.corrcoef(x, y)[0,1].round(4))   # 0.9798 ✓

# Unit-independence — the whole point
print(round(pearson(x * 1000, y), 4))    # 0.9798 — unchanged ✓
```

54.4 Correlation ≠ Causation

Three distinct reasons a correlation can mislead:

1. **Confounding.** Ice cream sales correlate with drownings — both are caused by hot weather.
2. **Reverse causation.** Hospital visits correlate with illness, but visits do not cause illness.
3. **Coincidence.** With enough variables, spurious correlations are guaranteed. Testing 100 unrelated features produces several "significant" correlations by chance alone.

54.5 What Pearson Correlation Misses

It only measures *linear* relationships.

```
x = np.linspace(-3, 3, 100)
y = x ** 2                                  # a perfect, deterministic
relationship
print(np.corrcoef(x, y)[0,1].round(6))  # ~0.0  ← "no relationship"?!
```

Correlation says zero, yet y is *completely determined* by x. The relationship is perfectly strong and perfectly non-linear.

The ML consequence: dropping features based on low correlation with the target is a common and costly mistake. A feature with r = 0.02 may be enormously predictive through a non-linear or interaction effect that a tree-based model would exploit immediately. Use correlation for *detecting multicollinearity among features* (where it excels), not for *feature selection against the target*.

54.6 Practical Multicollinearity Screening

```
rng = np.random.default_rng(7)
n = 200
X = np.column_stack([
    rng.normal(40, 10, n),                # age
    rng.normal(500_000, 100_000, n),      # income
])
X = np.column_stack([X, X[:,1]/1000, rng.normal(0,1,n)])   # income_k
(redundant), noise

corr = np.corrcoef(X, rowvar=False)
print(corr.round(3))
# Look for |r| > 0.9 off the diagonal — a red flag for
multicollinearity
names = ["age", "income", "income_k", "noise"]
for i in range(len(names)):
    for j in range(i+1, len(names)):
        if abs(corr[i,j]) > 0.9:
            print(f"⚠ {names[i]} ↔ {names[j]}: r = {corr[i,j]:.3f}")
# ⚠️ income ↔ income_k: r = 1.000
```

55. Probability Foundations

55.1 Vocabulary

Term	Meaning	Example
Experiment	A process with an uncertain outcome	Screening a transaction
Outcome	One possible result	"fraudulent"
Sample space Ω	The set of all outcomes	{fraud, legitimate}
Event	A subset of the sample space	"amount > ₦1m AND fraud"
Probability P(A)	A number in [0,1] measuring likelihood	P(fraud) = 0.002

55.2 The Three Axioms (Kolmogorov)

```
1.  P(A) ≥ 0                    probabilities are never negative
2.  P(Ω) = 1                    something must happen
3.  P(A ∪ B) = P(A) + P(B)      for mutually exclusive A, B
```

Everything else in probability follows from these three.

55.3 Derived Rules

Rule	Formula
Complement	$P(\text{not } A) = 1 - P(A)$
Union (general)	$P(A \cup B) = P(A) + P(B) - P(A \cap B)$
Intersection (independent)	$P(A \cap B) = P(A) \cdot P(B)$
Intersection (general)	$P(A \cap B) = P(A$

Why subtract P(A∩B) in the union rule: elements in both sets get counted twice by P(A) +P(B), so we remove one copy.

```
P_fraud, P_foreign = 0.002, 0.15
P_both = 0.0008                    # observed jointly

print("Union:", P_fraud + P_foreign - P_both)      # 0.1512
print("If independent:", P_fraud * P_foreign)      # 0.0003
print("Actual joint:", P_both)                     # 0.0008 — higher!
```

The actual joint probability (0.0008) exceeds the independent prediction (0.0003), so foreign transactions are **~2.7× more likely** to be fraudulent. That ratio is exactly the kind of signal a fraud model learns.

---

56. Conditional Probability

56.1 Definition

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \qquad \text{requires } P(B) > 0$$

Read: *"the probability of A **given that** B has occurred."*

The intuition: conditioning **shrinks the sample space**. Once you know B happened, the only outcomes still possible are those inside B, so you renormalize by P(B).

56.2 Worked Example — Fraud Detection

From 100,000 transactions:

	Fraud	Legitimate	Total
Foreign	120	14,880	15,000
Domestic	80	84,920	85,000
Total	200	99,800	100,000

```
P(Fraud)                = 200/100,000    = 0.002   (0.2%)
P(Fraud | Foreign)     = 120/15,000     = 0.008   (0.8%)  ← 4× higher
P(Fraud | Domestic)    =  80/85,000     = 0.00094 (0.09%)
```

```python
fraud_foreign, foreign_total = 120, 15_000
fraud_total, all_total = 200, 100_000

p_fraud          = fraud_total / all_total
p_fraud_given_fg = fraud_foreign / foreign_total
print(f"P(Fraud)           = {p_fraud:.5f}")
print(f"P(Fraud | Foreign) = {p_fraud_given_fg:.5f}")
print(f"Lift               = {p_fraud_given_fg/p_fraud:.1f}×")
# Lift = 4.0×
```

"Lift" is the ratio of conditional to unconditional probability — how much more likely fraud becomes once you know the transaction is foreign. It is the fundamental quantity in marketing response modelling and fraud rules alike.

56.3 Independence

A and B are independent if **P(A|B) = P(A)** — knowing B tells you nothing about A. Equivalently $P(A \cap B) = P(A)P(B)$.

In the table above, $P(\text{Fraud}|\text{Foreign}) = 0.008 \neq 0.002 = P(\text{Fraud})$, so they are **dependent** — which is precisely why "is this transaction foreign?" is a useful feature.

---

57. Bayes' Theorem

57.1 The Derivation

Start from the definition of conditional probability, written both ways:

$$P(A|B) = P(A \cap B)/P(B) \quad \Rightarrow \quad P(A \cap B) = P(A|B)P(B)$$
$$P(B|A) = P(A \cap B)/P(A) \quad \Rightarrow \quad P(A \cap B) = P(B|A)P(A)$$

Setting them equal: $P(A|B)P(B) = P(B|A)P(A)$

$$\therefore \quad P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

That is the whole derivation — two lines of algebra from the definition.

57.2 Every Term Named

Term	Name	Meaning
**P(A	B)**	**Posterior**
**P(B	A)**	**Likelihood**
P(A)	**Prior**	Belief in A *before* seeing any evidence
P(B)	**Evidence** / marginal	Total probability of the evidence

The story Bayes tells: start with a prior belief, observe evidence, update to a posterior belief. It is a formal model of learning from experience.

57.3 The Medical Test Example — Why Base Rates Dominate

A disease affects **1 in 1,000** people. A test is **99% accurate** (99% sensitivity, 99% specificity). You test positive. What is the probability you have the disease?

Most people answer "99%". The correct answer is under 10%.

$$P(D) = 0.001 \qquad \text{prior}$$
$$P(+|D) = 0.99 \qquad \text{sensitivity}$$
$$P(+|\text{not } D) = 0.01 \qquad \text{false positive rate}$$

$$P(+) = P(+|D)P(D) + P(+|\text{not } D)P(\text{not } D)$$
$$= (0.99)(0.001) + (0.01)(0.999)$$
$$= 0.00099 + 0.00999 = 0.01098$$

$$P(D|+) = \frac{P(+|D)P(D)}{P(+)} = \frac{0.00099}{0.01098} = 0.0902 \rightarrow 9.02\%$$

```
p_d, p_pos_given_d, p_pos_given_not_d = 0.001, 0.99, 0.01

p_pos = p_pos_given_d*p_d + p_pos_given_not_d*(1 - p_d)
p_d_given_pos = (p_pos_given_d * p_d) / p_pos
print(f"P(disease | positive) = {p_d_given_pos:.4f}")     # 0.0902
```

Why so low? Out of 1,000 people: 1 has the disease (and tests positive), while ~10 healthy people falsely test positive. So 1 of 11 positives is genuine. **The rare base rate swamps the test's accuracy.**

This is the single most important practical lesson in applied probability, and it recurs constantly in ML: a fraud model with 99% accuracy on a 0.2% fraud rate produces overwhelmingly false alerts. Base rates matter more than accuracy.

57.4 Bayes in Machine Learning

Application	Bayes in action
Naive Bayes classifier	P(class | features) ∝ P(features | class) · P(class) — Section 77
Spam filtering	P(spam | words) updated per word
Medical ML	Combining test results with population prevalence
Fraud scoring	Prior fraud rate updated by transaction evidence
Bayesian inference	Updating parameter distributions as data arrives

58. Random Variables

58.1 Definition

A **random variable** X assigns a number to each outcome of an experiment.

Type	Values	Described by	Example
Discrete	Countable	**PMF** P(X = x)	Number of fraud cases in a batch
Continuous	Uncountable (an interval)	**PDF** f(x)	Transaction amount

58.2 PMF vs PDF — A Distinction Beginners Get Wrong

For a **discrete** variable, P(X = 3) is a genuine probability, e.g. 0.2.

For a **continuous** variable, **P(X = exactly 3.0000...) = 0**. There are infinitely many possible values, so any single one has probability zero. The density f(x) is *not* a probability — probabilities come from **areas**:

```
P(a ≤ X ≤ b) = ∫ₐᵇ f(x) dx
```

A density value can even exceed 1 (a narrow, tall distribution) — which is impossible for a probability, and confirms it is not one.

58.3 Properties

Discrete	Continuous
$\Sigma\, P(X=x) = 1$	$\int f(x)\,dx = 1$
$P(X=x) \geq 0$	$f(x) \geq 0$

```python
# Discrete: a fair die
outcomes = np.arange(1, 7)
pmf = np.full(6, 1/6)
print(pmf.sum())                          # 1.0 ✓
print(f"E[X] = {(outcomes * pmf).sum()}")   # 3.5

# Continuous: a normal density — note values can exceed 1
from math import pi
def normal_pdf(x, mu=0, sigma=0.2):
    return np.exp(-0.5*((x-mu)/sigma)**2) / (sigma * np.sqrt(2*pi))
print(normal_pdf(0, sigma=0.2).round(3))    # 1.995 — a density > 1,
yet valid
```

---

59. Expectation

59.1 Definition

```
Discrete:    E[X] = Σ x · P(X = x)
Continuous:  E[X] = ∫ x · f(x) dx
```

The expected value is the probability-weighted average — the long-run average if you repeated the experiment forever.

59.2 Worked Example — Expected Loss From Fraud

```python
# Outcomes and probabilities for a single transaction
outcomes = np.array([0., -50_000., -200_000.])     # no fraud /
small / large fraud
probs    = np.array([0.998, 0.0015, 0.0005])

expected_loss = (outcomes * probs).sum()
print(f"Expected loss per transaction: ₦{expected_loss:,.2f}")
# Expected loss per transaction: ₦-175.00

n_transactions = 1_000_000
print(f"Expected annual loss: ₦{expected_loss * n_transactions:,.0f}")
# Expected annual loss: ₦-175,000,000
```

This calculation is how fraud-prevention budgets are justified. If a detection system costs ₦50m/year and reduces expected loss by 40% (₦70m), it pays for itself. Expected value converts probability into currency, which converts a model into a business case.

59.3 Properties of Expectation

Property	Formula	Note
Linearity	$E[aX + b] = aE[X] + b$	Always true
Additivity	$E[X + Y] = E[X] + E[Y]$	**True even if X, Y are dependent**
Product	$E[XY] = E[X]E[Y]$	Only if **independent**

The additivity property holding without independence is genuinely surprising and enormously useful in derivations.

59.4 Expectation in ML

- **Loss functions are expectations.** MSE estimates $E[(y - \hat{y})^2]$; empirical risk minimization means minimizing the *sample average* of loss as an estimate of the true expected loss.
- **Reinforcement learning maximizes expected return** (Section 94).
- **The bias-variance decomposition** (Section 73) is entirely written in expectations.

60–61. Variance and Standard Deviation

60.1 Definition

$$\text{Var}(X) = E[(X - \mu)^2] = E[X^2] - (E[X])^2$$

The second form (**"mean of the square minus the square of the mean"**) is the computationally convenient version and is worth memorizing.

60.2 Deriving the Shortcut

$$\begin{aligned} E[(X - \mu)^2] &= E[X^2 - 2\mu X + \mu^2] \\ &= E[X^2] - 2\mu E[X] + \mu^2 && \text{(linearity)} \\ &= E[X^2] - 2\mu^2 + \mu^2 && \text{(since } E[X] = \mu\text{)} \\ &= E[X^2] - \mu^2 && \checkmark \end{aligned}$$

60.3 Sample Variance and Bessel's Correction

$$\text{Population: } \sigma^2 = \frac{1}{n}\sum(x_i - \mu)^2$$

$$\text{Sample: } s^2 = \frac{1}{n-1}\sum(x_i - \bar{x})^2 \quad \leftarrow \text{note } n-1$$

Why n−1? Because $\bar{x}$ is itself estimated from the same data, the deviations are systematically slightly too small. Dividing by n−1 corrects this bias.

```
x = np.array([2., 4., 4., 4., 5., 5., 7., 9.])
print(x.var())              # 4.0   — population (ddof=0, NumPy default)
print(x.var(ddof=1))        # 4.571 — sample (Bessel-corrected)
```

A real source of bugs: `np.var` defaults to `ddof=0` while `np.cov` and Pandas' `.var()` default to the sample version. Always check which you are getting.

61.1 Standard Deviation

$\sigma = \sqrt{Var(X)}$

Why it exists: variance is in *squared* units (naira^2, years^2) which nobody can interpret. Standard deviation returns to the original units — exactly the same motivation as RMSE versus MSE (Section 3.3).

61.2 ML Connections

Concept	Role of variance
Standardization	$z = (x - \mu)/\sigma$ — Section 64
Bias-variance tradeoff	Variance = model sensitivity to the training sample (Section 73)
PCA	Eigenvalues *are* variances along each component
Feature selection	Zero-variance features carry no information and should be dropped
Weight initialization	Xavier/He initialization sets variance deliberately to keep signals stable across layers

```
# Zero-variance features are useless — a standard preprocessing check
X = np.array([[1., 5., 3.],
              [2., 5., 7.],
              [3., 5., 1.]])
variances = X.var(axis=0)
print(variances)                          # [0.667 0.    6.222]
print("Drop columns:", np.where(variances == 0)[0])   # [1] — constant
column
```

---

62. Common Probability Distributions

62.1 The Catalogue

Distribution	Models	Parameters	Mean	Variance
Bernoulli	One yes/no trial	p	p	p(1−p)
Binomial	k successes in n trials	n, p	np	np(1−p)
Uniform	All values equally likely	a, b	(a+b)/2	$(b-a)^2/12$
Normal	Sums of many small	μ, σ	μ	σ^2

Distribution	Models	Parameters	Mean	Variance
	effects			
Poisson	Counts of rare events per interval	λ	λ	λ
Exponential	Waiting time until an event	λ	$1/\lambda$	$1/\lambda^2$

62.2 Bernoulli — The Foundation of Binary Classification

```
P(X = 1) = p,     P(X = 0) = 1 − p
```

This is exactly what logistic regression models. The output $\sigma(w^Tx+b)$ *is* the parameter p of a Bernoulli distribution, and log loss is its negative log-likelihood (Section 70).

Note that variance $p(1-p)$ is **maximized at p = 0.5** and approaches zero at the extremes — a model predicting 0.5 is maximally uncertain, which matches intuition exactly.

```
p = np.linspace(0.01, 0.99, 9)
print(np.column_stack([p.round(2), (p*(1-p)).round(4)]))
# variance peaks at p=0.5 (0.25) and falls off symmetrically
```

62.3 Binomial

```
P(X = k) = C(n,k) · pᵏ · (1−p)^(n−k)
```

```
from math import comb
n, p = 100, 0.002                    # 100 transactions, 0.2% fraud rate
for k in range(4):
    prob = comb(n, k) * p**k * (1-p)**(n-k)
    print(f"P({k} frauds in 100) = {prob:.4f}")
# P(0 frauds in 100) = 0.8186
# P(1 frauds in 100) = 0.1640
# P(2 frauds in 100) = 0.0163
```

ML use: this tells you how many positive examples to expect per batch — critical for deciding batch sizes when training on severely imbalanced data. With 0.2% fraud and batch size 32, most batches contain *zero* positives, which breaks training. That is a purely mathematical argument for stratified batching.

62.4 Poisson — Rare Event Counts

```
P(X = k) = (λᵏ e^(−λ)) / k!
```

Models: fraud attempts per hour, equipment failures per month, website visits per minute, insurance claims per year.

```
from math import factorial
lam = 3.0                            # average 3 fraud attempts per hour
for k in range(6):
    print(f"P({k} attempts) = {lam**k * np.exp(-lam) /
factorial(k):.4f}")
```

A distinguishing property: for Poisson, **mean = variance = λ**. If your count data has variance much greater than its mean ("overdispersion"), Poisson is the wrong model — a genuinely useful diagnostic.

62.5 Exponential — Waiting Times

$$f(x) = \lambda e^{-\lambda x} \qquad \text{for } x \geq 0$$

Models time *between* Poisson events: time between transactions, time to equipment failure, customer inter-purchase time.

Its **memoryless property** — $P(X > s+t \mid X > s) = P(X > t)$ — means "having waited already tells you nothing about the remaining wait." Realistic for radioactive decay; usually *unrealistic* for equipment failure (older machines do fail more), which is why survival analysis uses Weibull distributions instead.

---

63. The Normal Distribution

63.1 The Formula

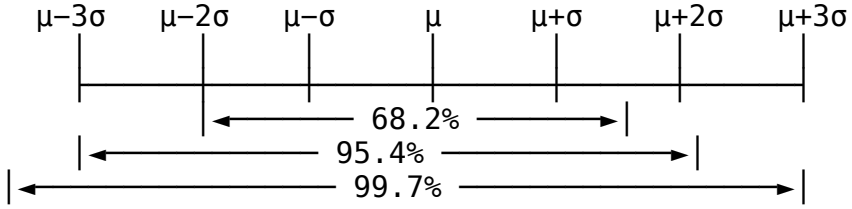
$$f(x) = \frac{1}{\sigma\sqrt{(2\pi)}} \cdot \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Two parameters entirely determine it: **μ** (centre) and **σ** (spread).

63.2 Why It Appears Everywhere

The **Central Limit Theorem** (Section 65): sums and averages of many independent random effects converge to a normal distribution *regardless of the original distribution*. Since most real measurements are the aggregate of many small influences, normality emerges naturally.

63.3 The 68-95-99.7 Rule




```python
rng = np.random.default_rng(0)
data = rng.normal(100, 15, 1_000_000)
mu, sigma = data.mean(), data.std()
for k in [1, 2, 3]:
    within = np.mean(np.abs(data - mu) < k*sigma)
    print(f"Within {k}σ: {within:.2%}")
# Within 1σ: 68.27%
# Within 2σ: 95.45%
# Within 3σ: 99.73%
```

63.4 Outlier Detection

The 3σ rule gives a simple, widely-used outlier criterion: flag points more than 3 standard deviations from the mean, since under normality only 0.3% of data should qualify.

```
def flag_outliers(x, k=3):
    z = (x - x.mean()) / x.std()
    return np.abs(z) > k

amounts = np.array([100., 150., 120., 130., 5000., 110., 125.])
print(flag_outliers(amounts))     # [False False False False  True
False False]
```

Caveat: this assumes normality. Income, transaction amounts and web traffic are typically **right-skewed**, where the 3σ rule flags too many legitimate high values. For skewed data, use the IQR method or apply a log transform first.

---

64. Z-Score

64.1 The Formula

$$z = \frac{x - \mu}{\sigma}$$

Symbol	Meaning
x	The raw value
μ	The mean
σ	The standard deviation
z	**How many standard deviations x is from the mean**

64.2 Interpretation

z	Meaning
0	Exactly at the mean
+1	One σ above the mean
−2	Two σ below
>3 or <−3	Unusual under normality (~0.3% of data)

64.3 Implementation

```
def zscore(x):
    return (x - x.mean()) / x.std()

incomes = np.array([300_000., 450_000., 500_000., 550_000.,
2_000_000.])
```

```
z = zscore(incomes)
print(z.round(3))        # [-0.639 -0.412 -0.336 -0.261  1.648]
print(f"mean={z.mean():.6f}, std={z.std():.6f}")   # mean≈0, std=1 ✓
```

Standardized data always has mean 0 and standard deviation 1 — that is the definitional consequence, and it is what makes features comparable.

64.4 Why Standardization Matters — Three Distinct Reasons

1. **Distance-based algorithms** (KNN, K-Means, SVM) are meaningless without it, because raw scale dominates distance (Section 16.3).
2. **Gradient descent converges faster** on standardized features, because the loss contours become circular rather than elongated (Section 25.2).
3. **Regularization penalizes coefficients**, and coefficient magnitude depends on feature scale — so without standardization, Ridge and Lasso penalize features unfairly based purely on their units (Section 72).

The critical procedural rule: fit μ and σ on the **training set only**, then apply those same values to validation and test. Computing them on the full dataset leaks test information into training (Module 1, Part 18).

```
X_train = np.array([[10., 1000.], [20., 2000.], [30., 3000.]])
X_test  = np.array([[25., 2500.]])

mu, sigma = X_train.mean(axis=0), X_train.std(axis=0)   # FIT on train only
X_train_s = (X_train - mu) / sigma
X_test_s  = (X_test  - mu) / sigma                       # APPLY to test
print(X_test_s.round(3))     # [[0.612 0.612]]
```

65–68. Sampling Theory and Estimation

65.1 The Central Limit Theorem

Statement: for independent, identically distributed samples with mean μ and finite variance σ^2, the distribution of the sample mean approaches a normal distribution as n grows:

```
x̄  ~  Normal( μ,  σ²/n )        approximately, for large n
```

regardless of the original distribution's shape.

The standard deviation of the sample mean, $\boldsymbol{\sigma/\sqrt{n}}$, is called the **standard error**.

65.2 Demonstration — Normality From a Wildly Non-Normal Source

```
rng = np.random.default_rng(42)

# Start with a heavily skewed exponential distribution
population = rng.exponential(scale=2.0, size=1_000_000)
print(f"Population: mean={population.mean():.3f}, skew is severe")

for n in [1, 5, 30, 100]:
    means = np.array([rng.choice(population, n).mean() for _ in
range(5000)])
    print(f"n={n:3d}: mean={means.mean():.3f}  std={means.std():.4f}
"
          f"predicted std={population.std()/np.sqrt(n):.4f}")
# n=  1: mean=1.999  std=1.9978  predicted std=1.9993
# n=  5: mean=2.001  std=0.8927  predicted std=0.8941
# n= 30: mean=2.000  std=0.3654  predicted std=0.3650
# n=100: mean=2.000  std=0.2001  predicted std=0.1999
```

Two things to notice: the sample-mean distribution becomes **normal** even though the source was exponential, and its spread shrinks as $\boldsymbol{1/\sqrt{n}}$ exactly as predicted.

65.3 The √n Law and Its Consequences

To halve your uncertainty you need 4× the data. To reduce it 10×, you need 100× the data.

```
for n in [100, 400, 1600, 10_000]:
    print(f"n={n:6,}: standard error = {2.0/np.sqrt(n):.4f}")
# n=   100: standard error = 0.2000
# n=   400: standard error = 0.1000   ← 4× data, half the error
# n= 1,600: standard error = 0.0500
# n=10,000: standard error = 0.0200
```

This is why ML has diminishing returns from more data, and why moving from 10,000 to 12,000 training samples rarely helps noticeably while moving from 100 to 10,000 transforms a model.

66.1 The Law of Large Numbers

As n → ∞, the sample average converges to the true expectation:

$\bar{x}_n \to E[X]$

The LLN says the average *converges*; the CLT describes *how fast* and *in what shape*. They are complementary.

ML significance: this is the theoretical justification for **empirical risk minimization** — minimizing average loss on your training sample is a valid proxy for minimizing true expected loss, *provided your sample is representative*. When it is not (sampling bias, distribution shift), the guarantee evaporates.

67.1 Sampling

Method	Description	ML use
Simple random	Every unit equally likely	Default train/test split
Stratified	Preserve subgroup proportions	**Essential for imbalanced classification**
Systematic	Every kth unit	Rare in ML
Cluster	Sample whole groups	Group-aware splitting
Time-based	Split chronologically	**Mandatory for temporal data**

67.2 Sampling Bias

```
rng = np.random.default_rng(0)
n = 10_000
income = rng.lognormal(mean=13, sigma=0.6, size=n)

# Biased: only surveying customers who responded (higher earners
respond more)
response_prob = (income - income.min()) / (income.max() -
income.min())
responded = rng.random(n) < response_prob * 0.5

print(f"True population mean:  ₦{income.mean():,.0f}")
print(f"Biased sample mean:    ₦{income[responded].mean():,.0f}")
print(f"Random sample mean:    ₦{rng.choice(income,
responded.sum()).mean():,.0f}")
```

The biased sample systematically overstates income. **No amount of data fixes sampling bias** — collecting ten times more biased data gives you a more *precisely wrong* answer. This is why "we have big data" is never a substitute for representative data.

68.1 Estimation

Term	Meaning
Parameter	A true population quantity (μ, σ, p) —

Term	Meaning
	usually unknown
Statistic	A quantity computed from a sample ($\bar{x}$, s, $\hat{p}$)
Point estimate	A single best guess
Interval estimate	A range with a stated confidence level

Notation convention: Greek letters (μ, σ, β) for population parameters; Roman letters or hatted symbols ($\bar{x}$, s, $\hat{\beta}$) for sample estimates. Every model coefficient you ever fit is an *estimate* $\hat{\beta}$, not the truth β — a distinction that matters when interpreting model output.

68.2 Confidence Intervals

$CI_{95\%} = \bar{x} \pm 1.96 \cdot (s/\sqrt{n})$

```
rng = np.random.default_rng(3)
sample = rng.normal(100, 15, 200)
xbar, se = sample.mean(), sample.std(ddof=1)/np.sqrt(len(sample))
lo, hi = xbar - 1.96*se, xbar + 1.96*se
print(f"Estimate: {xbar:.2f}, 95% CI: [{lo:.2f}, {hi:.2f}]")
```

Correct interpretation: if we repeated this sampling procedure many times, 95% of the resulting intervals would contain the true μ. It is *not* "there is a 95% probability μ lies in this specific interval" — μ is fixed, the interval is random.

69–70. Likelihood and Maximum Likelihood Estimation

69.1 Probability vs Likelihood — The Distinction

They use the same formula but ask opposite questions:

	Probability	Likelihood
Known	The parameters	The data
Unknown	The data	The parameters
Question	"Given p=0.5, how likely is 7 heads in 10?"	"Given I saw 7 heads in 10, which p is most plausible?"
Notation	$P(\text{data} \mid \theta)$	$L(\theta \mid \text{data})$
Sums to 1?	Yes, over data	**No** — it is not a distribution over θ

ML always works in the likelihood direction: we have the data, and we want the parameters.

69.2 Worked Example — Coin Flips

Observed: 7 heads in 10 flips. Which p best explains this?

```python
from math import comb

k, n = 7, 10
for p in [0.3, 0.5, 0.6, 0.7, 0.8, 0.9]:
    L = comb(n, k) * p**k * (1-p)**(n-k)
    print(f"p={p}: L = {L:.5f}")
# p=0.3: L = 0.00900
# p=0.5: L = 0.11719
# p=0.6: L = 0.21499
# p=0.7: L = 0.26683   ← maximum
# p=0.8: L = 0.20133
# p=0.9: L = 0.05740
```

The likelihood peaks at **p = 0.7 = 7/10** — the intuitive answer, now derived rather than guessed.

70.1 Maximum Likelihood Estimation

MLE finds the parameters making the observed data most probable:

```
θ̂_MLE = argmax_θ  L(θ)
```

70.2 The Log Transform and the Sign Flip

Because likelihood is a product over samples (which underflows — Section 10.3), we take logs. Because optimizers minimize, we negate:

```
argmax L(θ)  =  argmax log L(θ)  =  argmin [−log L(θ)]
                                          ↑
                                THE NEGATIVE LOG-LIKELIHOOD
```

Every step is justified: log is monotonic so the argmax is unchanged; negation flips max to min.

70.3 Deriving Log Loss From MLE — The Key Derivation

For binary classification with model probability $\hat{p}_i = \sigma(w^T x_i)$, each label follows a Bernoulli distribution:

$$P(y_i \mid x_i) = \hat{p}_i^{(y_i)} \cdot (1 - \hat{p}_i)^{(1-y_i)}$$

(Check: if $y_i=1$ this is $\hat{p}_i$; if $y_i=0$ it is $1-\hat{p}_i$. The exponents act as switches.)

Likelihood over all n independent samples:

$$L = \Pi_i \; \hat{p}_i^{(y_i)} \; (1 - \hat{p}_i)^{(1-y_i)}$$

Log-likelihood (products → sums):

$$\log L = \Sigma_i \; [\; y_i \; \log(\hat{p}_i) + (1-y_i) \; \log(1-\hat{p}_i) \;]$$

Negative, averaged:

$$-(1/n) \log L = -(1/n) \Sigma_i [y_i \log(\hat{p}_i) + (1-y_i) \log(1-\hat{p}_i)]$$

That is exactly binary cross-entropy / log loss. Log loss is not an arbitrary choice someone invented — it *is* the negative log-likelihood of the Bernoulli model. Understanding this derivation is what separates knowing the formula from understanding it.

```python
def log_loss(y, p, eps=1e-15):
    p = np.clip(p, eps, 1 - eps)          # avoid log(0) — Section 84
    return -np.mean(y*np.log(p) + (1-y)*np.log(1-p))

y = np.array([1, 0, 1, 1, 0])
print(log_loss(y, np.array([0.9, 0.1, 0.8, 0.7, 0.2])).round(4))  # 0.2027 — good
print(log_loss(y, np.array([0.5, 0.5, 0.5, 0.5, 0.5])).round(4))  # 0.6931 — uninformed
print(log_loss(y, np.array([0.1, 0.9, 0.2, 0.3, 0.8])).round(4))  # 1.8971 — confidently wrong
```

Note 0.6931 = ln(2) — the loss of a model that always predicts 0.5. **That is your baseline**: any binary classifier scoring worse than ln(2) is worse than useless.

70.4 MLE Elsewhere in ML

Model	MLE gives
Linear regression (Gaussian noise)	**MSE** — minimizing squared error *is* MLE under normal errors
Logistic regression (Bernoulli)	**Log loss**
Naive Bayes	Class priors and conditional probabilities
Softmax classifier (Categorical)	**Categorical cross-entropy**

A unifying insight: MSE and cross-entropy are not unrelated inventions. Both are negative log-likelihoods, differing only in the assumed noise distribution — Gaussian for regression, Bernoulli/Categorical for classification.

---

71. Loss Functions

71.1 Regression Losses

Loss	Formula	Behaviour
MAE	$(1/n)\Sigma\lvert y_i - \hat{y}_i\rvert$	Robust to outliers; constant gradient
MSE	$(1/n)\Sigma(y_i - \hat{y}_i)^2$	Penalizes large errors heavily; smooth, differentiable everywhere
RMSE	$\sqrt{\text{MSE}}$	Same ranking as MSE, in original units
Huber	Quadratic near 0, linear far	A compromise between MAE

Loss	Formula	Behaviour
	out	and MSE

71.2 MAE vs MSE — Choosing Deliberately

```
y      = np.array([100., 100., 100., 100.])
pred_a = np.array([110., 110., 110., 110.])    # four small errors of
10
pred_b = np.array([100., 100., 100., 140.])    # one large error of 40

for name, p in [("spread out", pred_a), ("one big", pred_b)]:
    mae = np.abs(y - p).mean()
    mse = ((y - p)**2).mean()
    print(f"{name:11s}: MAE={mae:5.1f}  MSE={mse:7.1f}")
# spread out : MAE= 10.0  MSE=  100.0
# one big    : MAE= 10.0  MSE=  400.0   ← MSE punishes concentration
```

Identical MAE, 4× different MSE. MSE considers one big error far worse than several small ones. Whether that is correct is a *business* question: for delivery-time prediction, one 40-minute error may be much worse than four 10-minute errors (use MSE); for median-income estimation robust to a billionaire outlier, it is not (use MAE).

71.3 Classification Losses

Loss	Formula	Use
Binary cross-entropy	$-(1/n)\Sigma[y \log \hat{p} + (1-y)\log(1-\hat{p})]$	Binary classification
Categorical cross-entropy	$-(1/n)\Sigma\Sigma\, y_ic \log(\hat{p}_ic)$	Multi-class
Hinge loss	$\max(0, 1 - y \cdot f(x))$	SVM; encourages a margin

```
def categorical_cross_entropy(y_onehot, p, eps=1e-15):
    p = np.clip(p, eps, 1 - eps)
    return -np.mean(np.sum(y_onehot * np.log(p), axis=1))

y_true = np.array([[1,0,0], [0,1,0], [0,0,1]])         # one-hot
p_good = np.array([[0.9,0.05,0.05],[0.1,0.8,0.1],[0.1,0.2,0.7]])
p_bad  = np.array([[0.2,0.4,0.4],[0.5,0.3,0.2],[0.4,0.4,0.2]])
print(categorical_cross_entropy(y_true, p_good).round(4))   # 0.2284
print(categorical_cross_entropy(y_true, p_bad).round(4))    # 1.5041
```

71.4 Why Different Models Use Different Losses

Three governing considerations:

1. **The probabilistic model.** MSE ⟸ Gaussian noise; cross-entropy ⟸ Bernoulli/Categorical (Section 70.4).
2. **Convexity and differentiability.** MSE and log loss are convex and smooth in linear models, guaranteeing reliable optimization. Accuracy is *not* differentiable — you cannot gradient-descend on it, which is why we optimize a surrogate loss and *report* accuracy.

3. **The business cost structure.** Asymmetric costs (a missed fraud costs far more than a false alert) justify weighted or custom losses.

The most common beginner confusion resolved: we train on log loss but evaluate on accuracy/F1/AUC because *training requires a differentiable surrogate* while *evaluation should reflect business value*. They are different jobs.

---

72. Regularization Mathematics

72.1 The Penalized Objective

$$J(\beta) = \text{Loss}(\beta) + \lambda \cdot \text{Penalty}(\beta)$$

λ controls the strength: $\lambda=0$ gives ordinary regression; large λ forces coefficients toward zero.

Method	Penalty	Effect
Ridge (L2)	$\lambda \Sigma\beta_j^2 = \lambda\|\beta\|_2^2$	Shrinks all coefficients smoothly toward 0
Lasso (L1)	$\lambda \Sigma\|\beta_j\| = \lambda\|\beta\|_1$	Drives some coefficients **exactly** to 0
Elastic Net	$\lambda_1\|\beta\|_1 + \lambda_2\|\beta\|_2^2$	Combines both

72.2 The Gradients — Why L1 Creates Exact Zeros

$$\text{Ridge:} \quad \partial/\partial\beta_j \ (\lambda\beta_j^2) = 2\lambda\beta_j \quad \leftarrow \text{proportional to } \beta_j$$
$$\text{Lasso:} \quad \partial/\partial\beta_j \ (\lambda|\beta_j|) = \lambda \cdot \text{sign}(\beta_j) \quad \leftarrow \text{CONSTANT magnitude } \lambda$$

This is the entire explanation. Ridge's penalty gradient shrinks as β shrinks, so the pull toward zero weakens and never quite arrives. Lasso's gradient stays at full strength λ no matter how small β becomes, so it pushes coefficients all the way to exactly zero and holds them there.

Combined with the geometric argument from Section 38.3 (the L1 diamond's corners lie on the axes), you now have two independent explanations of the same behaviour.

72.3 Ridge Has a Closed Form

$$\beta_{ridge} = (X^TX + \lambda I)^{-1} X^Ty$$

The λI term does double duty: statistically it shrinks coefficients; numerically it **guarantees invertibility**. Even if X^TX is singular from multicollinearity (Section 47), adding λI to the diagonal makes it positive definite and therefore invertible.

```
rng = np.random.default_rng(0)
n = 50
x1 = rng.normal(size=n)
x2 = x1 + rng.normal(scale=0.001, size=n)     # nearly identical to x1
X = np.column_stack([np.ones(n), x1, x2])
y = 3 + 2*x1 + rng.normal(scale=0.1, size=n)

XtX = X.T @ X
```

```
print("Condition number (OLS):", f"{np.linalg.cond(XtX):.2e}")   # astronomically bad

beta_ols   = np.linalg.lstsq(X, y, rcond=None)[0]
lam = 1.0
beta_ridge = np.linalg.solve(XtX + lam*np.eye(3), X.T @ y)

print("OLS  :", beta_ols.round(2))     # huge, unstable, opposite-signed coefficients
print("Ridge:", beta_ridge.round(3))   # small, stable, sensible
```

OLS typically produces wild coefficients like `[3.0, 847.2, -845.3]` — technically fitting the data while being complete nonsense. Ridge produces something like `[3.0, 1.0, 1.0]`, splitting the credit sensibly between the two near-identical features.

72.4 Choosing λ, and the Scaling Requirement

λ is a **hyperparameter** tuned by cross-validation. And critically:

Always standardize features before regularizing. The penalty $\Sigma\beta_j^2$ treats all coefficients identically, but a coefficient's size depends on its feature's units. A feature in naira gets a tiny coefficient and is barely penalized; the same feature in millions of naira gets a large coefficient and is heavily penalized — for no reason other than units.

Conventionally the intercept β_0 is not penalized, since shrinking it would bias predictions away from the target's mean for no benefit.

---

73. Bias-Variance Decomposition

73.1 The Decomposition

For squared-error loss, expected prediction error at a point decomposes exactly:

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2[\hat{f}(x)] + \text{Var}[\hat{f}(x)] + \sigma^2$$

	↑	
	wrong model assumptions	
		↑
		sensitivity to the sample
		↑
		irreducible noise

Term	Definition	Cause
Bias	$E[f(x)] - f(x)$	The model is too simple to represent the truth
Variance	$E[(f(x) - E[f(x)])^2]$	The model changes a lot with different training samples
Irreducible error σ^2	Noise in y itself	**Cannot be reduced by any model**

73.2 Why Irreducible Error Matters Practically

If $y = f(x) + \varepsilon$ with $\text{Var}(\varepsilon) = \sigma^2$, then **no model can achieve MSE below σ^2**. When a stakeholder asks for 99% accuracy on an inherently noisy target, the correct response is a mathematical one: estimate σ^2 and show that the requested accuracy is impossible, not merely difficult.

73.3 Empirical Demonstration

```
def true_f(x): return np.sin(2*np.pi*x)

rng = np.random.default_rng(1)
x_test = np.array([0.35])
n_trials, n_train, noise = 200, 30, 0.3

results = {}
for degree in [1, 3, 15]:
    preds = []
    for _ in range(n_trials):
        x = rng.uniform(0, 1, n_train)
        y = true_f(x) + rng.normal(0, noise, n_train)
        coeffs = np.polyfit(x, y, degree)
        preds.append(np.polyval(coeffs, x_test)[0])
    preds = np.array(preds)
    bias2 = (preds.mean() - true_f(x_test)[0])**2
    var   = preds.var()
    results[degree] = (bias2, var, bias2 + var + noise**2)

print(f"{'Degree':>7} {'Bias²':>10} {'Variance':>10} {'Total':>10}")
for d, (b, v, t) in results.items():
    print(f"{d:>7} {b:>10.4f} {v:>10.4f} {t:>10.4f}")
# degree 1  : high bias,  low variance   → underfitting
# degree 3  : low bias,   low variance   → best total error
# degree 15 : low bias,   HUGE variance  → overfitting
```

The middle row wins — not because it has the lowest bias or the lowest variance, but the lowest *sum*. That is the tradeoff made concrete.

73.4 The Levers

Action	Bias	Variance
More complex model	↓	↑
Simpler model	↑	↓
More training data	—	↓
Regularization (↑λ)	↑	↓
Bagging / Random Forest	—	↓↓
Boosting	↓↓	↑

More data reduces variance without increasing bias — which is why it is the one universally safe intervention, and why “get more data” is such reliable advice when a model overfits.

74. Linear Regression Mathematics

74.1 The Model

$\hat{y} = X\beta$

Object	Shape	Contents
X	(n, m+1)	Design matrix — features **with a leading column of 1s** for the intercept
β	(m+1,)	Coefficients, β_0 being the intercept
$\hat{y}$	(n,)	Predictions

74.2 Deriving the Normal Equation

Objective: minimize the sum of squared residuals.

$$J(\beta) = \|y - X\beta\|^2 = (y - X\beta)^T(y - X\beta)$$

Expand:

$$J(\beta) = y^Ty - y^TX\beta - \beta^TX^Ty + \beta^TX^TX\beta$$
$$= y^Ty - 2\beta^TX^Ty + \beta^TX^TX\beta \quad \text{(the middle terms are equal scalars)}$$

Differentiate with respect to β and set to zero (Section 22.5 — the gradient vanishes at a minimum):

$$\partial J/\partial\beta = -2X^Ty + 2X^TX\beta = 0$$
$$X^TX\beta = X^Ty \quad \leftarrow \text{the "normal equations"}$$
$$\beta = (X^TX)^{-1}X^Ty$$

Why we know it is a minimum: the Hessian is $2X^TX$, which is positive semi-definite for any X — so J is convex (Section 32) and this critical point is the global minimum.

74.3 Why You Should Not Implement It That Way

```
rng = np.random.default_rng(42)
n = 500
X_raw = rng.normal(size=(n, 3))
y = 5 + 2*X_raw[:,0] - 3*X_raw[:,1] + 0.5*X_raw[:,2] +
rng.normal(scale=0.5, size=n)
X = np.column_stack([np.ones(n), X_raw])          # add intercept
column

# Method 1 — textbook normal equation with explicit inverse  (AVOID)
beta_inv = np.linalg.inv(X.T @ X) @ X.T @ y

# Method 2 — solve the linear system                          (BETTER)
beta_solve = np.linalg.solve(X.T @ X, X.T @ y)
```

```
# Method 3 — least squares via SVD                                (BEST)
beta_lstsq = np.linalg.lstsq(X, y, rcond=None)[0]

print("inv  :", beta_inv.round(4))     # [4.9861 2.0074 -3.0107
0.5005]
print("solve:", beta_solve.round(4))   # identical here
print("lstsq:", beta_lstsq.round(4))   # identical here
```

On well-conditioned data all three agree. **On ill-conditioned data they do not**, and `lstsq` is the only one that degrades gracefully:

```
# Nearly collinear features — realistic when you engineer redundant
columns
x1 = rng.normal(size=200)
X_bad = np.column_stack([np.ones(200), x1, x1 + rng.normal(scale=1e-8,
size=200)])
y_bad = 3 + 2*x1 + rng.normal(scale=0.1, size=200)

print("cond(XᵀX):", f"{np.linalg.cond(X_bad.T @ X_bad):.2e}")   #
~1e32 — hopeless
try:
    b1 = np.linalg.inv(X_bad.T @ X_bad) @ X_bad.T @ y_bad
    print("inv gives:", b1.round(2))        # wild garbage values
except np.linalg.LinAlgError:
    print("inv failed outright")
b3 = np.linalg.lstsq(X_bad, y_bad, rcond=None)[0]
print("lstsq gives:", b3.round(3))          # a sensible minimum-norm
solution
```

This is why `sklearn.linear_model.LinearRegression` calls `scipy.linalg.lstsq` internally rather than implementing the textbook formula. The formula is correct mathematics; the implementation is bad numerics.

74.4 Complete From-Scratch Implementation

```
class LinearRegressionScratch:
    """Linear regression via least squares. No scikit-learn."""

    def fit(self, X, y):
        X_design = np.column_stack([np.ones(len(X)), X])       # add
intercept
        beta, *_ = np.linalg.lstsq(X_design, y, rcond=None)     #
stable solve
        self.intercept_ = beta[0]
        self.coef_ = beta[1:]
        return self

    def predict(self, X):
        return self.intercept_ + X @ self.coef_

    def score(self, X, y):
```

```
        """R² = 1 - SS_res/SS_tot."""
        y_hat = self.predict(X)
        ss_res = ((y - y_hat) ** 2).sum()
        ss_tot = ((y - y.mean()) ** 2).sum()
        return 1 - ss_res / ss_tot

model = LinearRegressionScratch().fit(X_raw, y)
print("Intercept:", round(model.intercept_, 4))   # 4.9861  (true 5)
print("Coefs:    ", model.coef_.round(4))         # [2.0074 -3.0107
0.5005]
print("R²:       ", round(model.score(X_raw, y), 4))  # 0.9724
```

74.5 Interpreting R^2

```
R² = 1 - SS_res/SS_tot
```

The fraction of the target's variance explained by the model. R^2=1 is perfect; R^2=0 means the model does no better than predicting the mean; **R^2 can be negative** on test data, which means the model is worse than a constant — a genuinely useful diagnostic signal that something is badly wrong.

---

75. Logistic Regression Mathematics

75.1 The Full Chain

```
1. Linear score:    z = Xβ
2. Probability:     p̂ = σ(z) = 1/(1 + e^(-z))
3. Loss:            J(β) = -(1/n)Σ[yᵢ log p̂ᵢ + (1-yᵢ)log(1-p̂ᵢ)]
4. Optimize:        gradient descent (no closed form exists)
```

75.2 Deriving the Gradient — The Beautiful Result

We need $\partial J/\partial \beta$. Apply the chain rule through three stages:

```
∂J/∂p̂ᵢ = -[ yᵢ/p̂ᵢ - (1-yᵢ)/(1-p̂ᵢ) ]
∂p̂ᵢ/∂zᵢ = p̂ᵢ(1 - p̂ᵢ)                    (Section 11.4)
∂zᵢ/∂β  = xᵢ

Multiply:
∂J/∂zᵢ = -[ yᵢ/p̂ᵢ - (1-yᵢ)/(1-p̂ᵢ) ] · p̂ᵢ(1-p̂ᵢ)
        = -[ yᵢ(1-p̂ᵢ) - (1-yᵢ)p̂ᵢ ]
        = -[ yᵢ - yᵢp̂ᵢ - p̂ᵢ + yᵢp̂ᵢ ]
        = p̂ᵢ - yᵢ                          ← everything cancels!

∴  ∇J = (1/n) Xᵀ(p̂ - y)
```

The sigmoid's messy derivative cancels the log loss's messy derivative perfectly, leaving simply (prediction – truth). This is not a coincidence — it happens because log loss is the *matched* loss for the sigmoid (they come from the same Bernoulli likelihood, Section

70.3). Note the gradient has **exactly the same form** as linear regression's: X^T(prediction − truth).

75.3 Complete Implementation

```
class LogisticRegressionScratch:
    """Logistic regression trained by gradient descent. No scikit-
learn."""

    def __init__(self, lr=0.1, n_iter=2000):
        self.lr, self.n_iter = lr, n_iter

    @staticmethod
    def _sigmoid(z):
        # numerically stable (Section 11.5)
        return np.where(z >= 0,
                        1/(1 + np.exp(-z)),
                        np.exp(z)/(1 + np.exp(z)))

    def fit(self, X, y):
        n, m = X.shape
        self.coef_, self.intercept_ = np.zeros(m), 0.0
        self.loss_history_ = []

        for _ in range(self.n_iter):
            z = X @ self.coef_ + self.intercept_
            p = self._sigmoid(z)

            # loss (clipped to avoid log(0) — Section 84)
            pc = np.clip(p, 1e-15, 1 - 1e-15)
            self.loss_history_.append(
                -np.mean(y*np.log(pc) + (1-y)*np.log(1-pc)))

            # the derived gradient: X^T(p − y)/n
            error = p - y
            self.coef_      -= self.lr * (X.T @ error) / n
            self.intercept_ -= self.lr * error.mean()
        return self

    def predict_proba(self, X):
        return self._sigmoid(X @ self.coef_ + self.intercept_)

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)

# Synthetic churn-like data
rng = np.random.default_rng(0)
n = 1000
X = rng.normal(size=(n, 2))
true_z = 1.5*X[:,0] - 2.0*X[:,1] + 0.5
```

```
y = (rng.random(n) < 1/(1+np.exp(-true_z))).astype(int)

clf = LogisticRegressionScratch(lr=0.5, n_iter=3000).fit(X, y)
print("Coefs:    ", clf.coef_.round(3))        # [ 1.52  -2.03]  (true
[1.5, -2.0])
print("Intercept:", round(clf.intercept_, 3))  # 0.487          (true
0.5)
print("Accuracy: ", (clf.predict(X) == y).mean().round(4))
print("Loss:", round(clf.loss_history_[0],4), "→",
round(clf.loss_history_[-1],4))
# Loss: 0.6931 → 0.4166      (starts at ln2 — the uninformed baseline)
```

The initial loss of 0.6931 is exactly ln(2), confirming the model starts with no information (Section 70.3).

75.4 Interpreting Coefficients — Log-Odds

$\log(p/(1-p)) = \beta_0 + \beta_1 x_1 + ... + \beta_m x_m$

A coefficient is the change in log-odds per unit increase in the feature. Exponentiating gives the **odds ratio**:

```
print("Odds ratios:", np.exp(clf.coef_).round(3))   # [4.57  0.131]
```

Interpretation: a one-unit increase in feature 1 multiplies the odds of the positive class by **4.57**; a one-unit increase in feature 2 multiplies them by **0.131** (i.e. reduces them by ~87%).

This interpretability is exactly why logistic regression remains the standard in credit scoring and clinical risk models, where explaining individual decisions is a legal requirement.

76–81. Algorithm Mathematics: KNN Through Gini

76.1 KNN Mathematics

The algorithm: to classify x, find the k nearest training points and take a vote.

```
1. Compute d(x, xᵢ) for all i          (Section 16)
2. Select the k smallest
3. Classify: majority vote    /    Regress: mean of neighbours
```

Weighted voting by inverse distance gives closer neighbours more influence:

```
weight_i = 1/d(x, xᵢ)     (or 1/d²)
```

```
def knn_predict(X_train, y_train, x_query, k=5, weighted=False):
    distances = np.sqrt(((X_train - x_query)**2).sum(axis=1))
    idx = np.argsort(distances)[:k]
    labels, dists = y_train[idx], distances[idx]

    if weighted:
```

```
        w = 1 / (dists + 1e-10)
        return np.bincount(labels, weights=w).argmax()
    return np.bincount(labels).argmax()

rng = np.random.default_rng(1)
X_tr = rng.normal(size=(100, 2))
y_tr = (X_tr[:,0] + X_tr[:,1] > 0).astype(int)
print(knn_predict(X_tr, y_tr, np.array([1.0, 1.0]), k=5))   # 1 ✓
print(knn_predict(X_tr, y_tr, np.array([-1.0, -1.0]), k=5)) # 0 ✓
```

The mathematical requirement KNN imposes: because it is pure distance, **feature scaling is mandatory** (Section 16.3), and performance degrades in high dimensions (Section 86). KNN has no parameters to learn — it is instance-based, storing the data itself as the model.

77.1 Naive Bayes Mathematics

Start from Bayes' theorem for classification:

$$P(C \mid x) = \frac{P(x_1,\ldots,x_m \mid C) \cdot P(C)}{P(x)}$$

The joint likelihood $P(x_1,\ldots,x_m|C)$ is impossible to estimate — it requires data for every combination of feature values. **The "naive" assumption** is conditional independence:

$$P(x_1,\ldots,x_m \mid C) = \Pi_j \; P(x_j \mid C)$$

Giving:

$$P(C \mid x) \propto P(C) \cdot \Pi_j \; P(x_j \mid C)$$

In log space (to avoid underflow, Section 10.3):

$$\log P(C|x) \propto \log P(C) + \Sigma_j \log P(x_j|C)$$

```
def naive_bayes_gaussian(X_train, y_train, x_query):
    classes = np.unique(y_train)
    log_posteriors = []
    for c in classes:
        Xc = X_train[y_train == c]
        prior = len(Xc) / len(X_train)
        mu, var = Xc.mean(axis=0), Xc.var(axis=0) + 1e-9
        # log of Gaussian likelihood, summed across features
(independence)
        log_lik = (-0.5*np.log(2*np.pi*var) - (x_query-
mu)**2/(2*var)).sum()
        log_posteriors.append(np.log(prior) + log_lik)
    return classes[np.argmax(log_posteriors)]

print(naive_bayes_gaussian(X_tr, y_tr, np.array([1.5, 1.5])))   # 1
```

Why "naive" is usually wrong yet works: features are rarely independent (income and spending clearly are not). But Naive Bayes only needs the *argmax* to be correct, not the probabilities themselves. The independence error often distorts all classes' scores similarly, leaving the ranking intact. This makes it fast, data-efficient, and a strong baseline for text classification despite its false premise.

78–81. Decision Tree Mathematics

The core question: which feature and threshold should we split on?

Answer: whichever split most reduces impurity.

79.1 Entropy

$$H(S) = -\Sigma_i \; p_i \; \log_2(p_i)$$

Measures uncertainty in bits.

```python
def entropy(y):
    _, counts = np.unique(y, return_counts=True)
    p = counts / len(y)
    return -(p * np.log2(p)).sum()

print(entropy(np.array([0,0,0,0])).round(4))          # 0.0    — pure, no uncertainty
print(entropy(np.array([0,0,1,1])).round(4))          # 1.0    — maximum uncertainty
print(entropy(np.array([0,0,0,1])).round(4))          # 0.8113
print(entropy(np.array([0]*9 + [1])).round(4))        # 0.4690 — mostly pure
```

Entropy is 0 when a node is pure and maximal when classes are balanced — exactly the behaviour we want from an impurity measure.

80.1 Information Gain

$$IG(S, A) = H(S) - \Sigma_v \; (|S_v|/|S|) \cdot H(S_v)$$

↑ parent; ↑ weighted average of children

Worked example:

```
Parent: 10 samples, 5 churn / 5 no-churn  →  H = 1.0

Split on "contract = monthly":
  Left  (monthly, 6): 5 churn, 1 no-churn  →  H = 0.650
  Right (annual,  4): 0 churn, 4 no-churn  →  H = 0.0

Weighted child entropy = (6/10)(0.650) + (4/10)(0) = 0.390
Information Gain = 1.0 − 0.390 = 0.610 bits
```

```python
def information_gain(y_parent, y_left, y_right):
    n = len(y_parent)
```

```
    weighted = (len(y_left)/n)*entropy(y_left) +
(len(y_right)/n)*entropy(y_right)
    return entropy(y_parent) - weighted

parent = np.array([0,0,0,0,0,1,1,1,1,1])
left   = np.array([0,0,0,0,0,1])      # mostly class 0
right  = np.array([1,1,1,1])          # pure class 1
print(round(information_gain(parent, left, right), 4))     # 0.6100
```

The tree tries every feature and every threshold, and picks the split with the highest information gain. That greedy search is the entire training algorithm.

81.1 Gini Impurity

$$Gini(S) = 1 - \Sigma_i \; p_i^2$$

```
def gini(y):
    _, counts = np.unique(y, return_counts=True)
    p = counts / len(y)
    return 1 - (p**2).sum()

for arr in [[0,0,0,0], [0,0,1,1], [0,0,0,1]]:
    a = np.array(arr)
    print(f"{str(arr):14s} entropy={entropy(a):.4f}
gini={gini(a):.4f}")
# [0, 0, 0, 0]   entropy=0.0000  gini=0.0000
# [0, 0, 1, 1]   entropy=1.0000  gini=0.5000
# [0, 0, 0, 1]   entropy=0.8113  gini=0.3750
```

81.2 Entropy vs Gini

	Entropy	Gini
Range (binary)	[0, 1]	[0, 0.5]
Computation	Requires `log`	Only multiplication — **faster**
Resulting trees	Nearly always the same	Nearly always the same
Default in scikit-learn	—	**Yes** (`criterion='gini'`)

They almost never disagree on which split to choose. Gini is the default purely because it avoids logarithms and is therefore faster — a rare case where a modelling choice is driven entirely by computational convenience with no statistical downside.

---

82. Clustering Mathematics (K-Means)

82.1 The Objective Function

K-Means minimizes the **within-cluster sum of squares** (WCSS, also called inertia):

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

where μ_k is the centroid of cluster k.

82.2 The Algorithm and Why It Converges

```
1. Initialize K centroids
2. ASSIGN:  each point to its nearest centroid
3. UPDATE:  each centroid to the mean of its assigned points
4. Repeat 2–3 until assignments stop changing
```

Why step 3 uses the mean: minimizing $\sum\|x - \mu\|^2$ with respect to μ gives, by differentiating and setting to zero, **μ = mean of the points**. The update rule is not a heuristic — it is the exact minimizer of the objective given fixed assignments.

Why it converges: both steps can only decrease J, and J is bounded below by 0. Since there are finitely many possible assignments, the algorithm must terminate. **But it converges to a *local* minimum** — the objective is non-convex (Section 32.3), which is exactly why `n_init` and `k-means++` initialization exist.

82.3 From-Scratch Implementation

```
def kmeans(X, k=3, max_iter=100, seed=0):
    rng = np.random.default_rng(seed)
    # initialize centroids at k randomly chosen data points
    centroids = X[rng.choice(len(X), k, replace=False)].copy()

    for iteration in range(max_iter):
        # ASSIGN: distance from every point to every centroid
        distances = np.sqrt(((X[:, None, :] -
centroids[None, :, :])**2).sum(axis=2))
        labels = distances.argmin(axis=1)

        # UPDATE: new centroid = mean of assigned points
        new_centroids = np.array([
            X[labels == j].mean(axis=0) if np.any(labels == j) else
centroids[j]
            for j in range(k)
        ])

        if np.allclose(new_centroids, centroids):
            break
        centroids = new_centroids

    wcss = sum(((X[labels == j] - centroids[j])**2).sum() for j in
range(k))
    return labels, centroids, wcss, iteration + 1

rng = np.random.default_rng(42)
X = np.vstack([rng.normal([0,0], 0.5, (100,2)),
               rng.normal([5,5], 0.5, (100,2)),
               rng.normal([0,5], 0.5, (100,2))])
```

```
labels, centroids, wcss, iters = kmeans(X, k=3)
print(f"Converged in {iters} iterations, WCSS = {wcss:.2f}")
print("Centroids:\n", centroids.round(2))
```

The broadcasting line `X[:, None, :] - centroids[None, :, :]` deserves attention: it creates an (n, k, features) array computing every point-to-centroid difference in one vectorized operation, avoiding a double Python loop.

82.4 Choosing K — The Elbow Method

```
for k in range(1, 7):
    _, _, wcss, _ = kmeans(X, k=k)
    print(f"k={k}: WCSS = {wcss:8.2f}")
# k=1: WCSS =  2447.86
# k=2: WCSS =   758.31
# k=3: WCSS =   148.29   ← sharp drop ends here (the "elbow")
# k=4: WCSS =   126.44
# k=5: WCSS =   107.11
```

WCSS always decreases with more clusters (k=n gives WCSS=0), so we look for the **elbow** — where additional clusters stop buying meaningful improvement. Here it is clearly k=3, matching how the data was generated.

83. Gradient-Based Optimization Variants

83.1 Why Plain Gradient Descent Is Not Enough

Three problems: it zigzags in elongated valleys (ill-conditioning, Section 25.3); one learning rate must suit all parameters despite wildly different gradient magnitudes; and it stalls at saddle points.

83.2 The Optimizer Family

Momentum — accumulate a velocity so consistent directions build speed:

```
v := γv + α∇J          (γ ≈ 0.9)
θ := θ − v
```

Intuition: a ball rolling downhill. It damps oscillation across a valley while accelerating along it, and carries enough inertia to roll through small bumps and off saddle points.

RMSProp — scale each parameter's step by its recent gradient magnitude:

```
s := βs + (1−β)(∇J)²
θ := θ − α∇J/(√s + ε)
```

Intuition: parameters with historically large gradients get smaller steps; rarely-updated parameters get larger ones. This directly solves the ill-conditioning problem.

Adam — combine both, with bias correction:

```
m := β₁m + (1−β₁)∇J                    (momentum term)
v := β₂v + (1−β₂)(∇J)²                 (RMSProp term)
m̂ := m/(1−β₁ᵗ),   v̂ := v/(1−β₂ᵗ)      (correct initialization bias)
θ := θ − α·m̂/(√v̂ + ε)
```

Defaults β_1=0.9, β_2=0.999, ε=1e-8 work remarkably well across problems, which is why **Adam is the default optimizer in deep learning**.

Why bias correction is needed: m and v start at zero, so early estimates are biased toward zero. Dividing by $(1-\beta^t)$ — which is small early and approaches 1 later — corrects this.

83.3 Comparative Implementation

```
def optimize(grad_fn, theta0, method="sgd", alpha=0.1, n_iter=100):
    theta = np.array(theta0, dtype=float)
    v = np.zeros_like(theta); m = np.zeros_like(theta); s =
np.zeros_like(theta)
    path = [theta.copy()]

    for t in range(1, n_iter + 1):
        g = grad_fn(theta)
        if method == "sgd":
            theta -= alpha * g
        elif method == "momentum":
            v = 0.9*v + alpha*g
```

```
            theta -= v
        elif method == "rmsprop":
            s = 0.9*s + 0.1*g**2
            theta -= alpha * g / (np.sqrt(s) + 1e-8)
        elif method == "adam":
            m = 0.9*m + 0.1*g
            s = 0.999*s + 0.001*g**2
            m_hat, s_hat = m/(1-0.9**t), s/(1-0.999**t)
            theta -= alpha * m_hat / (np.sqrt(s_hat) + 1e-8)
        path.append(theta.copy())
    return theta, np.array(path)

# An ill-conditioned bowl: J = x² + 20y²  (a long narrow valley)
grad = lambda th: np.array([2*th[0], 40*th[1]])

for method in ["sgd", "momentum", "rmsprop", "adam"]:
    final, _ = optimize(grad, [5., 5.], method=method, alpha=0.02,
n_iter=200)
    print(f"{method:9s}: final = {np.round(final, 5)}, distance =
{np.linalg.norm(final):.5f}")
```

Adam and RMSProp reach the minimum far faster on this deliberately ill-conditioned surface, because they adapt per-coordinate step sizes rather than forcing one α onto coordinates with gradients differing by 20×.

83.4 Choosing an Optimizer

Situation	Recommendation
Deep learning, general	**Adam** (safe, fast default)
Computer vision, final accuracy matters	SGD + momentum, tuned (often generalizes slightly better)
Convex problems (logistic regression)	L-BFGS (scikit-learn's default)
Very large sparse data	Adagrad/Adam variants

84. Numerical Stability

84.1 The Five Failure Modes

Problem	Cause	Symptom
Overflow	Values exceed ~1.8e308	`inf`, then `nan`
Underflow	Values below ~5e-324	Silently becomes 0.0
Catastrophic cancellation	Subtracting nearly-equal numbers	Loss of all significant digits
Division by zero	Zero denominator	`inf` or `nan`
log(0)	Domain violation	`-inf`

84.2 The Standard Fixes

```
# 1. LOG OF ZERO — clip probabilities
def safe_log_loss(y, p, eps=1e-15):
    p = np.clip(p, eps, 1 - eps)
    return -np.mean(y*np.log(p) + (1-y)*np.log(1-p))
print(safe_log_loss(np.array([1,0]), np.array([1.0, 0.0])))   # 3.6e-
15, not inf

# 2. SOFTMAX OVERFLOW — subtract the max (Section 12.4)
def stable_softmax(z):
    e = np.exp(z - z.max())
    return e / e.sum()
print(stable_softmax(np.array([1000., 1001., 1002.])).round(3))   #
[0.09 0.245 0.665]

# 3. PRODUCTS OF PROBABILITIES — work in log space (Section 10.3)
probs = np.full(1000, 0.5)
print(np.prod(probs))                    # 0.0      ← destroyed
print(np.sum(np.log(probs)).round(3))    # -693.147 ← preserved

# 4. DIVISION BY ZERO — add epsilon
def safe_normalize(x, eps=1e-10):
    return x / (np.linalg.norm(x) + eps)
print(safe_normalize(np.zeros(3)))       # [0. 0. 0.] — no crash

# 5. CATASTROPHIC CANCELLATION — use a stable formula
x = np.array([1e8, 1e8 + 1, 1e8 + 2])
naive  = (x**2).mean() - x.mean()**2       # E[X²] − (E[X])²
stable = ((x - x.mean())**2).mean()        # E[(X−μ)²]
print(f"naive={naive:.6f}  stable={stable:.6f}  true=0.666667")
# naive can return 0.0 or even a negative variance — mathematically
impossible
```

That last example is instructive. The shortcut variance formula (Section 60.1) is algebraically correct but numerically dangerous when the mean is large relative to the spread — the two large terms nearly cancel, destroying precision. NumPy's `.var()` uses the stable two-pass form.

84.3 Detecting Problems Early

```
def check_array(name, a):
    issues = []
    if np.isnan(a).any(): issues.append(f"{np.isnan(a).sum()} NaN")
    if np.isinf(a).any(): issues.append(f"{np.isinf(a).sum()} Inf")
    mx = np.abs(a[np.isfinite(a)]).max() if np.isfinite(a).any() else
0
    if mx > 1e10: issues.append(f"large values (max {mx:.2e})")
    print(f"{name}: {'; '.join(issues) if issues else 'OK'}")

check_array("gradients", np.array([1., 2., np.nan, 1e12]))
# gradients: 1 NaN; large values (max 1.00e+12)
```

Inserting this check after every gradient computation during development catches numerical failures at their origin rather than 500 iterations later when everything has become `nan`.

---

85. Feature Scaling Mathematics

85.1 The Three Methods

Method	Formula	Output range	Use when
Standardization (Z-score)	$(x - \mu)/\sigma$	mean 0, std 1; unbounded	Default; roughly normal data
Min-Max normalization	$(x - \min)/(\max - \min)$	$[0, 1]$	Bounded input needed (neural network inputs, images)
Robust scaling	$(x - \text{median})/\text{IQR}$	Varies	**Outliers present**

85.2 Implementation and Comparison

```
def standardize(x):     return (x - x.mean()) / x.std()
def minmax(x):          return (x - x.min()) / (x.max() - x.min())
def robust_scale(x):
    q25, q75 = np.percentile(x, [25, 75])
    return (x - np.median(x)) / (q75 - q25)

x = np.array([10., 12., 11., 13., 12., 500.])     # note the outlier

print("standardized:", standardize(x).round(3))
# [-0.448 -0.438 -0.443 -0.433 -0.438  2.201]  ← outlier compresses
everything else

print("min-max:     ", minmax(x).round(3))
# [0.    0.004 0.002 0.006 0.004 1.   ]  ← normal values crushed into
[0, 0.006]

print("robust:      ", robust_scale(x).round(3))
# [-1.  0.   -0.5  0.5  0.  325.]  ← normal values keep their spread
```

This example makes the choice concrete. With one extreme outlier, min-max scaling squashes all the legitimate variation into a range of 0.006 — effectively destroying the feature. Robust scaling preserves the structure of the bulk of the data and lets the outlier be extreme.

85.3 Which Algorithms Require Scaling

Requires scaling	Does not require scaling
KNN, K-Means (distance-based)	Decision trees
SVM (distance/margin)	Random forests
Neural networks (optimization)	Gradient boosting

Requires scaling	Does not require scaling
PCA (variance-based)	Naive Bayes (mostly)
Ridge/Lasso (penalty fairness)	—
Gradient descent generally	—

Why trees are exempt: a tree splits on thresholds like "income > 500,000". Rescaling changes the threshold value but not which rows go left or right — the *ordering* is all that matters, and scaling preserves ordering. This is a genuine mathematical property, not a rule of thumb.

---

86. Distance in High Dimensions

86.1 The Curse of Dimensionality

As dimensions increase, **all points become approximately equidistant**, which destroys the meaning of "nearest neighbour."

```
rng = np.random.default_rng(0)
print(f"{'dims':>6} {'min dist':>10} {'max dist':>10} {'ratio':>8}
{'rel. contrast':>14}")
for d in [2, 5, 10, 50, 100, 500, 1000]:
    X = rng.random((1000, d))
    q = rng.random(d)
    dist = np.sqrt(((X - q)**2).sum(axis=1))
    contrast = (dist.max() - dist.min()) / dist.min()
    print(f"{d:>6} {dist.min():>10.3f} {dist.max():>10.3f} "
          f"{dist.max()/dist.min():>8.3f} {contrast:>14.3f}")
# dims   min dist   max dist    ratio  rel. contrast
#    2      0.012      1.130   94.174         93.174
#   10      0.512      2.203    4.303          3.303
#  100      2.723      5.169    1.898          0.898
# 1000     10.628     14.166    1.333          0.333
```

In 2 dimensions the farthest point is 94× more distant than the nearest — distance is highly discriminative. In 1000 dimensions it is only 1.3× — nearest and farthest are nearly the same, so "nearest neighbour" is close to meaningless.

86.2 Consequences

Affected	Why
KNN	Neighbours are not meaningfully near
K-Means	Cluster assignment becomes arbitrary
Radius-based methods	No radius separates near from far
Data sparsity	Volume grows exponentially; any fixed sample becomes sparse

86.3 Mitigations

```
# 1. Dimensionality reduction (PCA — Section 52)
# 2. Feature selection — keep only informative features
# 3. Manhattan (L1) distance degrades more gracefully than Euclidean
for d in [10, 100, 1000]:
    X = rng.random((500, d)); q = rng.random(d)
    l2 = np.sqrt(((X-q)**2).sum(axis=1))
    l1 = np.abs(X-q).sum(axis=1)
    print(f"d={d:4d}: L2 contrast={(l2.max()-l2.min())/l2.min():.3f}
"
          f"L1 contrast={(l1.max()-l1.min())/l1.min():.3f}")
# L1 retains more contrast at high d — which is why Manhattan is often
recommended there
# 4. Cosine similarity for text/embeddings (Section 87)
```

---

87. Cosine Similarity

87.1 The Formula

$$\cos(\theta) = \frac{x \cdot y}{\|x\|_2 \cdot \|y\|_2} \qquad \in [-1, 1]$$

It measures the angle between vectors, completely ignoring their magnitudes.

87.2 Why Magnitude-Independence Matters

```
def cosine_similarity(a, b):
    return (a @ b) / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-10)

doc1 = np.array([3., 5., 2.])          # a short document
doc2 = np.array([30., 50., 20.])       # the SAME content, 10× longer
doc3 = np.array([1., 0., 9.])          # a different topic

print(f"doc1 vs doc2: cosine={cosine_similarity(doc1,doc2):.4f}  "
      f"euclidean={np.linalg.norm(doc1-doc2):.2f}")
# doc1 vs doc2: cosine=1.0000  euclidean=54.13

print(f"doc1 vs doc3: cosine={cosine_similarity(doc1,doc3):.4f}")
# doc1 vs doc3: cosine=0.3592
```

Cosine correctly identifies doc1 and doc2 as identical in content (similarity 1.0) while Euclidean distance calls them far apart (54.1) purely because one is longer. For text, where document length varies enormously but topic is what matters, this is decisive.

87.3 Cosine vs Euclidean

	Cosine	Euclidean
Measures	Direction/angle	Absolute position
Magnitude-sensitive	No	Yes

	Cosine	Euclidean
Range	$[-1, 1]$	$[0, \infty)$
Best for	Text, embeddings, recommendations, high dimensions	Physical measurements, low dimensions

Useful relationship: for **unit-normalized** vectors, cosine similarity and Euclidean distance are equivalent ($\|x-y\|^2 = 2 - 2\cos\theta$). This is why vector databases normalize embeddings on ingestion — it lets them use fast Euclidean or dot-product search while effectively computing cosine similarity.

87.4 Applications

Semantic search and RAG retrieval; recommendation ("users with similar taste vectors"); document clustering; and de-duplication. **It is the default similarity metric in essentially every vector database** (Pinecone, Weaviate, FAISS, pgvector).

---

88. Projections

88.1 Projecting One Vector Onto Another

$$\text{proj}_b(a) = \frac{a \cdot b}{\|b\|^2} b$$

This finds the component of **a** lying along the direction of **b**.

```
a = np.array([3., 4.])
b = np.array([1., 0.])                          # the x-axis

proj = (a @ b) / (b @ b) * b
perp = a - proj
print("projection:", proj)                      # [3. 0.]
print("perpendicular:", perp)                   # [0. 4.]
print("orthogonal? ", np.isclose(proj @ perp, 0))   # True ✓
```

Any vector decomposes uniquely into a component along b and a component perpendicular to b. That decomposition is the foundation of least squares and PCA alike.

88.2 Projection Onto a Subspace — Least Squares Geometrically

$P = A(A^TA)^{-1}A^T$ the projection matrix onto the column space of A

The geometric meaning of linear regression: y usually does not lie in the column space of X (no exact solution exists). Least squares finds the point in that column space **closest to y** — which is precisely the orthogonal projection of y onto it. The residual $y - \hat{y}$ is perpendicular to every column of X.

```
rng = np.random.default_rng(0)
X = rng.normal(size=(100, 3))
```

```
y = rng.normal(size=100)

beta = np.linalg.lstsq(X, y, rcond=None)[0]
y_hat = X @ beta
residual = y - y_hat

print("Residual ⊥ each column of X:", np.allclose(X.T @ residual, 0,
atol=1e-10))
# True — this orthogonality IS the defining property of the least-
squares solution
```

That single check confirms the entire geometric interpretation. The normal equation $X^TX\beta = X^Ty$ can be read as $X^T(y - X\beta) = 0$ — "the residual is orthogonal to the feature space."

---

89. Gradient Descent From Scratch — Complete Implementation

89.1 A Reusable, Instrumented Implementation

```
def gradient_descent(grad_fn, theta_init, alpha=0.01, n_iter=1000,
                     tol=1e-8, loss_fn=None, verbose=False):
    """
    Generic gradient descent with convergence detection.

    grad_fn    : callable θ → ∇J(θ)
    theta_init : starting parameters
    alpha      : learning rate
    tol        : stop when the parameter update is smaller than this
    loss_fn    : optional, for tracking
    """
    theta = np.array(theta_init, dtype=float)
    history = {"theta": [theta.copy()], "loss": [], "grad_norm": []}

    for i in range(n_iter):
        g = grad_fn(theta)
        theta_new = theta - alpha * g

        history["grad_norm"].append(np.linalg.norm(g))
        if loss_fn is not None:
            history["loss"].append(loss_fn(theta))

        if np.linalg.norm(theta_new - theta) < tol:
            if verbose: print(f"Converged at iteration {i}")
            theta = theta_new
            history["theta"].append(theta.copy())
            break

        theta = theta_new
        history["theta"].append(theta.copy())
```

```
    history["theta"] = np.array(history["theta"])
    return theta, history
```

89.2 Applied to Linear Regression

```
rng = np.random.default_rng(42)
n = 200
X_raw = rng.normal(size=(n, 2))
y = 4 + 3*X_raw[:,0] - 2*X_raw[:,1] + rng.normal(scale=0.2, size=n)
X = np.column_stack([np.ones(n), X_raw])

def mse_loss(beta):  return ((X @ beta - y)**2).mean()
def mse_grad(beta):  return (2/n) * X.T @ (X @ beta - y)

beta, hist = gradient_descent(mse_grad, np.zeros(3), alpha=0.1,
                              n_iter=5000, loss_fn=mse_loss,
verbose=True)

print("GD solution:   ", beta.round(4))                    # [3.9903
2.9978 -2.0053]
print("Exact (lstsq):", np.linalg.lstsq(X, y, rcond=None)[0].round(4))
print(f"Loss: {hist['loss'][0]:.4f} → {hist['loss'][-1]:.6f}")
print(f"Final gradient norm: {hist['grad_norm'][-1]:.2e}")  # ~1e-8 —
flat, converged
```

Two verification habits worth adopting: compare your GD result against the analytical solution when one exists, and check that the final gradient norm is near zero (confirming you reached a flat point rather than simply running out of iterations).

89.3 Gradient Checking

Before trusting any hand-derived gradient, verify it numerically:

```
def check_gradient(f, grad_f, theta, eps=1e-6):
    """Compare an analytical gradient to a numerical one."""
    numerical = np.zeros_like(theta)
    for i in range(len(theta)):
        tp, tm = theta.copy(), theta.copy()
        tp[i] += eps; tm[i] -= eps
        numerical[i] = (f(tp) - f(tm)) / (2*eps)      # central
difference
    analytical = grad_f(theta)
    rel_error = (np.linalg.norm(analytical - numerical) /
                 (np.linalg.norm(analytical) +
np.linalg.norm(numerical) + 1e-15))
    return rel_error, analytical, numerical

err, a, nm = check_gradient(mse_loss, mse_grad, np.array([1., 1.,
1.]))
print(f"Relative error: {err:.2e}")          # ~1e-10
```

```
print("analytical:", a.round(6))
print("numerical: ", nm.round(6))
```

Interpretation: relative error below 1e-7 means your gradient is correct; above 1e-4 means you have a bug. **This is the single most valuable debugging technique for anyone implementing models by hand**, and it is exactly what deep learning frameworks' `gradcheck` utilities do.

90–91. Neural Network and Backpropagation Mathematics

90.1 A Neuron

```
z = wᵀx + b        (a dot product plus bias — Section 37)
a = f(z)           (a non-linear activation — Section 8)
```

A single neuron is logistic regression when f = σ. A network is many of these composed in layers.

90.2 Forward Propagation

For a network with layers 1…L:

```
a⁰ = x                                  (input)
z^l = W^l a^(l−1) + b^l                 (linear)
a^l = f(z^l)                            (activation)
ŷ  = a^L                                (output)
```

Matrix shapes (batch of size B, layer sizes n_l):

```
a^(l−1) : (B, n_{l−1})
W^l     : (n_{l−1}, n_l)
z^l, a^l: (B, n_l)
```

91.1 Backpropagation — The Four Equations

Backpropagation is the chain rule (Section 23) applied systematically, computing gradients from the output backward.

```
(1)  δ^L = ∇_a J ⊙ f'(z^L)                    error at the output layer
(2)  δ^l = (W^(l+1) δ^(l+1)) ⊙ f'(z^l)        propagate error backward
(3)  ∂J/∂W^l = (a^(l−1))ᵀ δ^l                  gradient for weights
(4)  ∂J/∂b^l = Σ_batch δ^l                     gradient for biases
```

where ⊙ is element-wise multiplication.

The intuition for equation (2): a hidden neuron's error is however much the *next* layer's errors blame it (weighted by the connections W), scaled by how responsive that neuron was (f'(z)). If f'(z) ≈ 0 — a saturated sigmoid — the neuron receives almost no error signal and stops learning. **That is the vanishing gradient problem, visible directly in the equation.**

91.2 A Complete Two-Layer Network From Scratch

```
class NeuralNetworkScratch:
    """A 2-layer network (one hidden layer) with ReLU → sigmoid,
trained by backprop."""

    def __init__(self, n_input, n_hidden, n_output=1, seed=0):
        rng = np.random.default_rng(seed)
        # He initialization for ReLU: variance 2/n_in keeps signal
magnitude stable
        self.W1 = rng.normal(0, np.sqrt(2/n_input),  (n_input,
n_hidden))
```

```
        self.b1 = np.zeros(n_hidden)
        self.W2 = rng.normal(0, np.sqrt(2/n_hidden), (n_hidden,
n_output))
        self.b2 = np.zeros(n_output)

    @staticmethod
    def _sigmoid(z):
        return np.where(z >= 0, 1/(1+np.exp(-z)),
np.exp(z)/(1+np.exp(z)))

    def forward(self, X):
        self.z1 = X @ self.W1 + self.b1
        self.a1 = np.maximum(0, self.z1)              # ReLU
        self.z2 = self.a1 @ self.W2 + self.b2
        self.a2 = self._sigmoid(self.z2)              # output
probability
        return self.a2

    def backward(self, X, y, lr=0.01):
        n = len(X)
        y = y.reshape(-1, 1)

        # Output layer: for sigmoid + log loss, δ = (p − y) exactly
(Section 75.2)
        dz2 = self.a2 - y                             # (n, n_output)
        dW2 = self.a1.T @ dz2 / n                     # equation (3)
        db2 = dz2.mean(axis=0)                        # equation (4)

        # Hidden layer: propagate backward
        da1 = dz2 @ self.W2.T                         # equation (2),
first part
        dz1 = da1 * (self.z1 > 0)                     # × ReLU'(z1)
        dW1 = X.T @ dz1 / n
        db1 = dz1.mean(axis=0)

        # Update all parameters
        self.W2 -= lr * dW2;  self.b2 -= lr * db2
        self.W1 -= lr * dW1;  self.b1 -= lr * db1

    def fit(self, X, y, epochs=1000, lr=0.1):
        self.losses = []
        for _ in range(epochs):
            p = self.forward(X)
            pc = np.clip(p.ravel(), 1e-15, 1-1e-15)
            self.losses.append(-np.mean(y*np.log(pc) + (1-y)*np.log(1-
pc)))
            self.backward(X, y, lr)
        return self

# The XOR problem — NOT linearly separable, so it requires a hidden
```

```
layer
X = np.array([[0.,0.], [0.,1.], [1.,0.], [1.,1.]])
y = np.array([0., 1., 1., 0.])

nn = NeuralNetworkScratch(n_input=2, n_hidden=8, seed=3).fit(X, y,
epochs=5000, lr=0.5)
print("Predictions:", nn.forward(X).ravel().round(3))
# Predictions: [0.017 0.983 0.983 0.019]   ← solved
print(f"Loss: {nn.losses[0]:.4f} → {nn.losses[-1]:.5f}")
```

Why XOR is the classic test: no straight line separates XOR's classes, so logistic regression cannot solve it at all (accuracy caps at 75%). A hidden layer creates a new representation in which the problem *becomes* linearly separable. This single example demonstrates why depth matters.

91.3 Why Initialization Matters

```
n_in, n_hidden = 784, 256
for name, std in [("zeros", 0), ("too small", 0.001),
                  ("too large", 1.0), ("He (correct)",
np.sqrt(2/n_in))]:
    W = np.zeros((n_in, n_hidden)) if std == 0 else \
        np.random.default_rng(0).normal(0, std, (n_in, n_hidden))
    a = np.maximum(0, np.random.default_rng(1).normal(size=(100,
n_in)) @ W)
    print(f"{name:14s}: activation std = {a.std():.6f}")
# zeros         : activation std = 0.000000  ← all neurons identical;
network cannot learn
# too small     : activation std = 0.017     ← signal vanishes through
depth
# too large     : activation std = 17.0      ← signal explodes
# He (correct)  : activation std = 0.86      ← stable
```

All-zero initialization is fatal, not merely suboptimal: every hidden neuron computes the identical value, receives the identical gradient, and remains identical forever. The network has 256 hidden units but the expressive power of one. This is called the **symmetry problem**, and breaking it is why initialization must be random.

92–93. Transformer and Embedding Mathematics

92.1 Scaled Dot-Product Attention

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Symbol	Name	Role
Q	Query	"What am I looking for?"

Symbol	Name	Role
K	Key	"What do I contain?"
V	Value	"What do I contribute?"
d_k	Key dimension	Used for scaling

92.2 Reading the Formula

QK^S computes a dot product (Section 37) between every query and every key — a matrix of similarity scores. High score = this token is relevant to that token.

Dividing by √d_k — the critical detail. Dot products of d_k-dimensional random vectors have variance proportional to d_k. Without scaling, for d_k = 512 the scores have standard deviation $\sim\sqrt{512} \approx 23$, pushing softmax into a regime where one value dominates completely and gradients vanish.

```
rng = np.random.default_rng(0)
for d_k in [8, 64, 512]:
    q, k = rng.normal(size=(100, d_k)), rng.normal(size=(100, d_k))
    scores_raw    = (q * k).sum(axis=1)
    scores_scaled = scores_raw / np.sqrt(d_k)
    print(f"d_k={d_k:4d}: raw std={scores_raw.std():6.2f}  "
          f"scaled std={scores_scaled.std():5.2f}")
# d_k=   8: raw std=  2.87  scaled std= 1.01
# d_k=  64: raw std=  8.03  scaled std= 1.00
# d_k= 512: raw std= 22.51  scaled std= 0.99
```

The √d_k division keeps score variance at ~1 regardless of dimension — that is its entire purpose, and it is a purely mathematical fix to a numerical problem.

Softmax converts scores to attention weights summing to 1 (Section 12). **Multiplying by V** produces a weighted average of value vectors — each token's output is a blend of all tokens, weighted by relevance.

92.3 Implementation

```
def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = (Q @ K.T) / np.sqrt(d_k)
    if mask is not None:
        scores = np.where(mask, scores, -1e9)      # -1e9 → ~0 after
softmax
    weights = np.exp(scores - scores.max(axis=-1, keepdims=True))
    weights /= weights.sum(axis=-1, keepdims=True)
    return weights @ V, weights

rng = np.random.default_rng(1)
seq_len, d_k = 4, 8
Q = K = V = rng.normal(size=(seq_len, d_k))

out, w = scaled_dot_product_attention(Q, K, V)
```

```
print("Attention weight matrix (rows sum to 1):")
print(w.round(3))
print("Row sums:", w.sum(axis=1).round(6))    # [1. 1. 1. 1.] ✓
print("Output shape:", out.shape)             # (4, 8)
```

Causal masking (used in GPT-style models) prevents a token from attending to future tokens, by setting those scores to −1e9 so softmax assigns them essentially zero weight — a neat exploitation of exponential decay.

93.1 Embeddings

An **embedding** maps a discrete item (word, product, user) to a dense vector in $\mathbb{R}^d$, learned such that semantically similar items land near each other.

```
# A toy embedding space where similarity is meaningful
embeddings = {
    "fraud":       np.array([0.9, 0.8, 0.1]),
    "suspicious":  np.array([0.85, 0.75, 0.15]),
    "legitimate":  np.array([0.1, 0.2, 0.9]),
    "normal":      np.array([0.15, 0.25, 0.85]),
}

def cos_sim(a, b): return (a @
b)/(np.linalg.norm(a)*np.linalg.norm(b))

print(f"fraud ↔ suspicious : {cos_sim(embeddings['fraud'],
embeddings['suspicious']):.4f}")
print(f"fraud ↔ legitimate : {cos_sim(embeddings['fraud'],
embeddings['legitimate']):.4f}")
print(f"legitimate ↔ normal: {cos_sim(embeddings['legitimate'],
embeddings['normal']):.4f}")
# fraud ↔ suspicious : 0.9994   ← near-synonyms cluster
# fraud ↔ legitimate : 0.3288   ← opposites are far apart
# legitimate ↔ normal: 0.9992
```

Why dense embeddings beat one-hot encoding: one-hot vectors are orthogonal, so *every* pair of words has cosine similarity exactly 0 — the representation contains no notion of similarity whatsoever. A 50,000-word vocabulary needs 50,000 dimensions one-hot; a 300-dimensional embedding is 166× smaller *and* encodes meaning.

Vector search (the retrieval half of RAG) is exactly this: embed the query, compute cosine similarity against millions of stored embeddings, return the top k. The mathematics is Section 87 applied at scale.

94–95. Reinforcement Learning and Time Series Mathematics

94.1 Expected Return and Discounting

```
G_t = R_{t+1} + γR_{t+2} + γ²R_{t+3} + ... = Σ_{k=0}^∞ γᵏ R_{t+k+1}
```

$\gamma \in [0,1)$ is the **discount factor**. It serves two purposes: it encodes a preference for sooner rewards, and — via the geometric series (Section 13.2) — it **guarantees the infinite sum converges**.

```
gamma = 0.9
rewards = np.full(100, 1.0)                  # reward of 1 forever
discounts = gamma ** np.arange(100)
print(f"Discounted return (100 steps):
{(rewards*discounts).sum():.4f}")   # 9.9997
print(f"Theoretical limit 1/(1-γ):      {1/(1-gamma):.4f}")
# 10.0000
```

Without discounting, an infinite stream of rewards sums to infinity and every policy looks equally good — the problem becomes undefined. **Discounting is what makes infinite-horizon RL mathematically well-posed.**

94.2 The Bellman Equation

```
V(s) = max_a [ R(s,a) + γ Σ_{s'} P(s'|s,a) V(s') ]
```

In words: the value of a state is the best immediate reward plus the discounted expected value of wherever you end up. It is a *recursive* definition — value defined in terms of value.

```
# Q-learning update (the practical, model-free form)
def q_update(Q, s, a, r, s_next, alpha=0.1, gamma=0.9):
    td_target = r + gamma * Q[s_next].max()      # what we now think
the value is
    td_error  = td_target - Q[s, a]              # how wrong we were
    Q[s, a] += alpha * td_error                  # move partway toward
the target
    return Q

Q = np.zeros((5, 2))
for _ in range(100):
    Q = q_update(Q, s=0, a=1, r=1.0, s_next=1)
print(Q[0].round(4))     # [0.  0.99997]  — converges toward the true
value
```

The "TD error" is the reinforcement learning analogue of a loss gradient — it measures the gap between prediction and a better estimate, and the update moves partway toward closing it. The structural similarity to supervised learning's (prediction − truth) is not accidental.

95.1 Time Series Mathematics

Autocorrelation — correlation of a series with a lagged copy of itself:

```
ρ_k = Corr(X_t, X_{t-k})
```

```
def autocorrelation(x, lag):
    return np.corrcoef(x[:-lag], x[lag:])[0, 1]
```

```
t = np.arange(365)
seasonal = 100 + 20*np.sin(2*np.pi*t/365*12) +
np.random.default_rng(0).normal(0,2,365)

for lag in [1, 7, 15, 30]:
    print(f"lag {lag:2d}: autocorrelation = {autocorrelation(seasonal,
lag):+.3f}")
# lag  1: autocorrelation = +0.960
# lag 15: autocorrelation = -0.902     ← strong negative: half a cycle
away
# lag 30: autocorrelation = +0.719     ← positive again: a full cycle
```

High autocorrelation is precisely why random train/test splitting is invalid for time series — adjacent points are nearly identical, so a random split leaks almost-identical information between train and test.

Moving average smooths noise:

```
def moving_average(x, window):
    return np.convolve(x, np.ones(window)/window, mode='valid')
print(moving_average(seasonal, 7)[:5].round(2))
```

Differencing removes trend to achieve stationarity:

```
trend = np.arange(100) * 2 + np.random.default_rng(0).normal(0, 3,
100)
diff = np.diff(trend)
print(f"Original: mean drifts, std={trend.std():.2f}")
print(f"Differenced: mean={diff.mean():.2f}, std={diff.std():.2f}  ←
stationary")
```

Stationarity — constant mean and variance over time — is an assumption of classical time-series models (ARIMA). Differencing is the standard transformation to achieve it, and is exactly what the "I" (Integrated) in ARIMA refers to.

96. Mathematical Computation in NumPy

96.1 The Essential Operations

```
# ---------- Creation ----------
np.array([1., 2., 3.])          # from a list
np.zeros((3, 4));  np.ones(5);  np.eye(3)
np.arange(0, 10, 2)             # [0 2 4 6 8]
np.linspace(0, 1, 5)            # [0. 0.25 0.5 0.75 1.]

# ---------- Shape ----------
a = np.arange(12)
a.reshape(3, 4); a.reshape(-1, 2)      # -1 means "infer this
dimension"
a[:, None]                              # add an axis: (12,) → (12, 1)

# ---------- Aggregation (mind the axis!) ----------
X = np.array([[1., 2., 3.], [4., 5., 6.]])
X.sum()             # 21.0        everything
X.sum(axis=0)       # [5. 7. 9.]  collapse ROWS   → one value per
column
X.sum(axis=1)       # [ 6. 15.]   collapse COLS   → one value per row
X.mean(axis=0); X.std(axis=0); X.min(axis=1); X.argmax(axis=1)

# ---------- Linear algebra ----------
A = np.array([[4., 7.], [2., 6.]])
A @ A;  A.T
np.linalg.inv(A); np.linalg.det(A); np.linalg.matrix_rank(A)
np.linalg.solve(A, np.array([1., 2.]))
np.linalg.eig(A); np.linalg.norm(A); np.linalg.cond(A)
```

96.2 Broadcasting — The Rules

NumPy aligns shapes from the **right**, and two dimensions are compatible if they are equal or one of them is 1.

```
X = np.arange(12).reshape(3, 4).astype(float)     # (3, 4)

print(X + 10)                        # scalar broadcasts to everything
print(X + np.array([1.,2.,3.,4.]))  # (4,) broadcasts across rows
print(X + np.array([[1.],[2.],[3.]]))   # (3,1) broadcasts across
columns

# The canonical ML use: standardization in one line
print(((X - X.mean(axis=0)) / X.std(axis=0)).round(3))
```

The most common broadcasting bug:

```
y_true = np.array([1., 2., 3.])          # (3,)
y_pred = np.array([[1.1], [2.1], [3.1]]) # (3, 1)  ← wrong shape!
print((y_true - y_pred).shape)           # (3, 3) — a matrix, not a
```

```
vector!
print(((y_true - y_pred)**2).mean())     # silently WRONG loss value
# Fix: y_pred.ravel() or y_pred.squeeze()
```

This produces no error, just a wrong number. **Always check `.shape` when a loss value looks strange.**

96.3 Vectorization Over Loops

```
import time
n = 1_000_000
X = np.random.rand(n, 10); w = np.random.rand(10)

t0 = time.perf_counter()
slow = np.array([X[i] @ w for i in range(n)])
t_slow = time.perf_counter() - t0

t0 = time.perf_counter()
fast = X @ w
t_fast = time.perf_counter() - t0

print(f"Loop: {t_slow:.3f}s   Vectorized: {t_fast:.5f}s   Speedup:
{t_slow/t_fast:.0f}×")
print("Identical results:", np.allclose(slow, fast))
```

Rule of thumb: if you are writing a `for` loop over rows of an array, there is almost certainly a vectorized alternative.

---

97. Mathematical Visualization

```
import matplotlib.pyplot as plt

# 1. Function and its derivative
x = np.linspace(-3, 3, 200)
fig, axes = plt.subplots(2, 3, figsize=(15, 8))

axes[0,0].plot(x, x**2, label='f(x)=x²')
axes[0,0].plot(x, 2*x, '--', label="f'(x)=2x")
axes[0,0].axhline(0, c='k', lw=0.5); axes[0,0].legend();
axes[0,0].set_title('Function & Derivative')

# 2. Activation functions
axes[0,1].plot(x, 1/(1+np.exp(-x)), label='sigmoid')
axes[0,1].plot(x, np.tanh(x), label='tanh')
axes[0,1].plot(x, np.maximum(0,x), label='ReLU')
axes[0,1].legend(); axes[0,1].set_title('Activations')

# 3. Loss functions
e = np.linspace(-3, 3, 200)
axes[0,2].plot(e, e**2, label='MSE')
```

```
axes[0,2].plot(e, np.abs(e), label='MAE')
axes[0,2].legend(); axes[0,2].set_title('MSE vs MAE')

# 4. Gradient descent path on a contour plot
xx, yy = np.meshgrid(np.linspace(-5,5,100), np.linspace(-5,5,100))
Z = xx**2 + 5*yy**2
axes[1,0].contour(xx, yy, Z, levels=20)
theta = np.array([4., 4.]); path = [theta.copy()]
for _ in range(25):
    theta = theta - 0.08*np.array([2*theta[0], 10*theta[1]])
    path.append(theta.copy())
path = np.array(path)
axes[1,0].plot(path[:,0], path[:,1], 'ro-', ms=3)
axes[1,0].set_title('Gradient Descent Path')

# 5. Normal distribution
xn = np.linspace(-4, 4, 200)
axes[1,1].plot(xn, np.exp(-xn**2/2)/np.sqrt(2*np.pi))
for k in [1,2,3]:
    axes[1,1].axvline(k, ls='--', c='grey', lw=0.7)
    axes[1,1].axvline(-k, ls='--', c='grey', lw=0.7)
axes[1,1].set_title('Normal: 68-95-99.7')

# 6. Bias-variance tradeoff
c = np.linspace(1, 15, 100)
bias2, var = 10/c, 0.05*c
axes[1,2].plot(c, bias2, label='Bias²')
axes[1,2].plot(c, var, label='Variance')
axes[1,2].plot(c, bias2+var+0.3, lw=2, label='Total')
axes[1,2].legend(); axes[1,2].set_title('Bias-Variance Tradeoff')

plt.tight_layout()
# plt.show()
```

Why visualize mathematics at all? A contour plot of gradient descent makes ill-conditioning immediately obvious in a way that a table of numbers never does. Plotting your loss curve is the single fastest diagnostic for a learning rate problem (Section 31.1).

---

98. Mathematical Exercises

A representative set spanning every branch, with full solutions. Work them by hand before checking.

98.1 Algebra and Functions

1. Solve $5x - 12 = 3x + 8$. $\rightarrow 2x = 20$, **$x = 10$**

2. Expand $(2x + 3)(x - 4)$. $\rightarrow 2x^2 - 8x + 3x - 12 =$ **$2x^2 - 5x - 12$**

3. Find slope and intercept through (1, 5) and (4, 14). → m = 9/3 = 3; b = 5 − 3 = 2; **y = 3x + 2**

4. Solve: 3x + 2y = 16, x − y = 2. → x = y+2 → 3(y+2)+2y=16 → 5y=10 → **y=2, x=4**

5. If ln(x) = 3, find x. → **$x = e^3 \approx 20.086$**

6. Simplify $\log(a^3b^2) - \log(ab)$. → $3\log a + 2\log b - \log a - \log b$ = **$2\log a + \log b$**

7. Compute σ(0), σ(2), σ(−2). → **0.5, 0.881, 0.119**

8. Compute softmax([1, 2, 3]). → e^{1}/sum = **[0.090, 0.245, 0.665]**

98.2 Calculus

9. $d/dx(4x^3 - 2x^2 + 7x - 1)$. → **$12x^2 - 4x + 7$**

10. $d/dx(x \cdot e^x)$. → Product rule: **$e^x(1 + x)$**

11. $d/dx(\ln(3x^2 + 1))$. → Chain rule: **$6x/(3x^2 + 1)$**

12. For $f(x,y) = 3x^2y + y^3$, find ∂f/∂x and ∂f/∂y. → **6xy** and **$3x^2 + 3y^2$**

13. Find ∇f at (1,2) for $f = x^2 + xy + y^2$. → $\nabla f = [2x+y, x+2y]$ = **[4, 5]**

14. Prove σ'(x) = σ(x)(1−σ(x)). → See Section 11.4.

15. For $f(x)=x^3-6x^2+9x$, find critical points and classify them. → $f'=3x^2-12x+9=3(x-1)(x-3)$ → x=1,3. f''=6x−12. f''(1)=−6<0 **max**; f''(3)=6>0 **min**.

16. Given $J(\theta)=\theta^2-6\theta+10$, find the minimum analytically and via 3 GD steps from θ=0 with α=0.2. → Analytical: J'=2θ−6=0 → **θ=3**. → GD: $\theta_1=0-0.2(-6)=1.2$; $\theta_2=1.2-0.2(-3.6)=1.92$; $\theta_3=1.92-0.2(-2.16)=$**2.352** (converging to 3)

98.3 Linear Algebra

17. For A=[[2,1],[3,4]], B=[[1,0],[2,5]], compute AB. → **[[4,5],[11,20]]**

18. Compute AB vs BA for the above. BA = [[2,1],[19,22]] — **different**, confirming non-commutativity.

19. det([[3,8],[4,6]]). → 18 − 32 = **−14**

20. Inverse of [[4,7],[2,6]]. → det=10; **[[0.6,−0.7],[−0.2,0.4]]**

21. Solve 2x+3y=8, 4x−y=2 using matrices. → **x=1, y=2**

22. Dot product of [1,3,−2] and [4,−1,5]. → 4 − 3 − 10 = **−9**

23. $\|[3,-4,12]\|_2$ and $\|\cdot\|_1$. → √169 = **13**; **19**

24. Eigenvalues of [[2,0],[0,5]]. → Diagonal matrix: **2 and 5**

25. Eigenvalues of [[4,1],[2,3]]. → $\lambda^2-7\lambda+10=0$ → **λ = 5, 2**

---

[1] −2,−1,0

26. Rank of [[1,2,3],[2,4,6],[1,1,1]]. → Row 2 = 2×Row 1, so **rank 2**

27. Why does rank 2 with 3 columns break linear regression? → X^TX is singular; the inverse does not exist; infinitely many solutions. **Multicollinearity** (Section 47).

98.4 Probability and Statistics

28. P(A)=0.3, P(B)=0.5, independent. Find P(A∩B) and P(A∪B). → **0.15**; 0.3+0.5−0.15 = **0.65**

29. Disease prevalence 2%, test 95% sensitive and 90% specific. P(disease|positive)? → P(+) = 0.95(0.02) + 0.10(0.98) = 0.019 + 0.098 = 0.117; P(D|+) = 0.019/0.117 = **0.162 (16.2%)**

30. Mean and variance of [4, 8, 6, 5, 3]. → mean **5.2**; deviations [−1.2,2.8,0.8,−0.2,−2.2]; population var = 15.2/5 = **3.04**; sample var = 15.2/4 = **3.8**

31. Z-score of 85 where μ=70, σ=10. → **1.5**

32. What fraction of normal data lies beyond 2σ? → **~4.6%** (100% − 95.4%)

33. Compute the log loss for y=1, $\hat{p}$=0.8. → −ln(0.8) = **0.223**

34. Compute the log loss for y=1, $\hat{p}$=0.1. → −ln(0.1) = **2.303** — ten times worse, illustrating the penalty for confident wrongness.

35. Derive log loss from the Bernoulli likelihood. → Section 70.3.

98.5 ML Mathematics

36. y=[3,5,7], $\hat{y}$=[2.5,5.5,6.5]. Compute MSE, RMSE, MAE. → errors [0.5,−0.5,0.5]; MSE = 0.75/3 = **0.25**; RMSE = **0.5**; MAE = 1.5/3 = **0.5**

37. Compute entropy of [0,0,0,1,1]. → p=[0.6,0.4]; $-(0.6\log_2 0.6 + 0.4\log_2 0.4)$ = **0.971**

38. Compute Gini of the same. → 1 − (0.36+0.16) = **0.48**

39. Parent H=1.0 splits into (6 samples, H=0.65) and (4, H=0). Information gain? → 1.0 − [0.6(0.65) + 0.4(0)] = **0.61**

40. Why does L1 produce exact zeros but L2 does not? → L1's penalty gradient is constant λ·sign(β) regardless of β's size, pushing all the way to zero; L2's gradient 2λβ shrinks as β shrinks, approaching but never reaching zero (Sections 38.3, 72.2).

41. Euclidean distance between [1,2,3] and [4,6,3]. → √(9+16+0) = **5**

42. Cosine similarity between [1,1] and [2,2]. → 4/(√2·√8) = 4/4 = **1.0** (identical direction)

43. Write the gradient of MSE for linear regression in matrix form. → $\nabla J = (2/n)X^S(X\beta - y)$

44. Why does the logistic regression gradient simplify to $X^T(\hat{p} - y)$? → The sigmoid derivative exactly cancels the log-loss derivative because log loss is the matched loss for the sigmoid link (Section 75.2).

45. A 10-layer sigmoid network: estimate the gradient shrinkage at layer 1. → At most $0.25^{10} \approx$ **9.5e-7** — effectively zero, the vanishing gradient problem.

---

99. Coding Challenges

1. Implement the mean, variance and standard deviation from scratch.

```
def stats(x):
    mean = sum(x) / len(x)
    var  = sum((xi - mean)**2 for xi in x) / (len(x) - 1)
    return mean, var, var ** 0.5
print(stats([2., 4., 4., 4., 5., 5., 7., 9.]))   # (5.0, 4.571, 2.138)
```

2. Implement the dot product without `@` or `np.dot`.

```
def dot(a, b):
    return sum(ai * bi for ai, bi in zip(a, b))
print(dot([1,2,3], [4,5,6]))      # 32
```

3. Implement matrix multiplication with loops, then verify against NumPy.

```
def matmul(A, B):
    n, k, m = len(A), len(B), len(B[0])
    return [[sum(A[i][p]*B[p][j] for p in range(k)) for j in range(m)]
for i in range(n)]
A = [[1.,2.],[3.,4.]]; B = [[5.,6.],[7.,8.]]
print(matmul(A, B))                                    # [[19,22],
[43,50]]
print(np.allclose(matmul(A,B), np.array(A) @ np.array(B)))   # True
```

4. Implement a numerically stable sigmoid. → Section 11.5.

5. Implement a numerically stable softmax. → Section 12.4.

6. Implement MSE, RMSE, MAE and R^2.

```
def regression_metrics(y, yhat):
    err = y - yhat
    mse = (err**2).mean()
    return {"MSE": mse, "RMSE": np.sqrt(mse), "MAE":
np.abs(err).mean(),
            "R2": 1 - (err**2).sum()/((y - y.mean())**2).sum()}
```

7. Implement log loss with clipping. → Section 70.3.

8. Implement gradient descent for $f(x)=x^2-4x+7$ and verify it reaches $x=2$. → Section 28.3.

9. Implement numerical differentiation using central differences.

```
def d(f, x, h=1e-6): return (f(x+h) - f(x-h)) / (2*h)
print(round(d(lambda x: x**3, 2), 4))     # 12.0  (exact: 3x² = 12)
```

10. Implement gradient checking and use it to validate a hand-derived gradient. → Section 89.3.

11. Implement standardization, min-max and robust scaling. → Section 85.2.

12. Implement Euclidean, Manhattan and cosine distance. → Sections 16.2, 87.2.

13. Implement PCA from scratch and verify the explained variance sums to 1. → Section 52.3.

14. Implement the covariance matrix from scratch and verify against `np.cov`.

```
def covariance_matrix(X):
    Xc = X - X.mean(axis=0)
    return (Xc.T @ Xc) / (len(X) - 1)
X = np.random.default_rng(0).normal(size=(50, 3))
print(np.allclose(covariance_matrix(X), np.cov(X, rowvar=False)))   # True
```

15. Implement entropy, Gini and information gain. → Sections 79–81.

16. Implement linear regression via `lstsq` and compare to gradient descent. → Sections 74.4, 89.2.

17. Implement logistic regression with gradient descent. → Section 75.3.

18. Implement K-Means and the elbow method. → Section 82.

19. Implement a 2-layer neural network with backpropagation and solve XOR. → Section 91.2.

20. Implement scaled dot-product attention with causal masking. → Section 92.3.

21. Implement Adam and compare its convergence against plain SGD on an ill-conditioned problem. → Section 83.3.

22. Write a function detecting numerical problems (NaN, Inf, extreme values) in an array. → Section 84.3.

23. Demonstrate the curse of dimensionality empirically. → Section 86.1.

24. Implement Ridge regression and show it handles multicollinearity where OLS fails. → Section 72.3.

25. Verify the bias-variance decomposition empirically with polynomial models of varying degree. → Section 73.3.

100. Mathematics Interview Preparation

100.1 Beginner (1–25)

1. What does $\theta^* = \text{argmin } L(\theta)$ mean? The parameters that make the loss function as small as possible.

2. Why does ML use gradient DEscent, not gradient ASCent? Loss is something we minimize; the negative gradient points toward lower loss.

3. What is the difference between a parameter and a hyperparameter? A parameter is learned from data; a hyperparameter is set by the practitioner before training.

4. Why does logistic regression use sigmoid? To squash an unbounded linear score into (0,1), interpretable as a probability.

5. What is the derivative of x^2? 2x, by the power rule.

6. What does the dot product measure? The alignment between two vectors, combining magnitude and the cosine of the angle between them.

7. Why do we square errors in MSE instead of using absolute value? Squaring makes errors positive without an absolute-value kink, is smoothly differentiable everywhere, and penalizes large errors more.

8. What is a gradient? A vector of partial derivatives, pointing in the direction of steepest increase.

9. What happens if the learning rate is too large? The optimizer overshoots the minimum and can diverge.

10. What is the difference between variance and standard deviation? Standard deviation is the square root of variance, returning it to the original units.

11. Why is log(0) a problem in ML? It equals negative infinity, which breaks loss functions like log loss; probabilities are clipped away from exactly 0 and 1 to prevent it.

12. What is a matrix? A rectangular array of numbers, representing e.g. a dataset (rows=samples, columns=features) or a linear transformation.

13. What does it mean for a matrix to be singular? It has no inverse; equivalently its determinant is 0 and its rows/columns are linearly dependent.

14. What is the identity matrix? A square matrix with 1s on the diagonal and 0s elsewhere, satisfying AI = A.

15. What is Bayes' theorem used for in ML? Combining a prior belief with observed evidence to obtain an updated (posterior) probability — the basis of Naive Bayes.

16. What is the difference between probability and likelihood? Probability assumes known parameters and asks about data; likelihood assumes known data and asks which parameters are most plausible.

17. What does standardization do to a feature? Transforms it to have mean 0 and standard deviation 1.

18. Why must you fit a scaler on training data only? Fitting on the full dataset leaks test-set information into training, producing overly optimistic estimates.

19. What is entropy in the context of decision trees? A measure of uncertainty/impurity in a set of labels; 0 when pure, maximal when perfectly balanced.

20. What is the normal distribution's shape determined by? Its mean (centre) and standard deviation (spread).

21. What does R^2 measure? The fraction of the target's variance explained by the model.

22. What is the difference between a vector and a matrix? A vector is one-dimensional (a list of numbers); a matrix is two-dimensional (rows and columns).

23. What is cosine similarity used for? Measuring the similarity of two vectors' directions, ignoring magnitude — common for text and embeddings.

24. What does the chain rule allow you to compute? The derivative of a composite function, by multiplying the derivatives of each stage.

25. Why does gradient descent slow down near a minimum? The gradient's magnitude shrinks as the function flattens, so the update steps naturally shrink too.

100.2 Intermediate (26–55)

26. Derive the derivative of the sigmoid function. → Section 11.4.

27. Why is the normal equation $\beta=(X^SX)^{-1}X^Sy$ numerically risky? Computing an explicit inverse compounds rounding error and fails outright on singular/near-singular X^TX; solving the linear system directly (or using SVD-based least squares) is more stable.

28. Explain why L1 regularization produces sparse solutions but L2 does not. L1's penalty gradient has constant magnitude regardless of coefficient size, pushing coefficients all the way to zero; L2's gradient shrinks proportionally to the coefficient, so it approaches but never reaches zero.

29. What is the bias-variance tradeoff? Total expected error decomposes into bias^2 + variance + irreducible error; reducing model complexity increases bias but decreases variance, and vice versa.

30. Why does softmax subtract the max value before exponentiating? To prevent overflow, since it's mathematically equivalent (the same C cancels in numerator and denominator) but numerically safe.

31. Explain why MLE for a Gaussian-noise regression model gives MSE. The log-likelihood under Gaussian noise reduces, after removing constants, to minimizing the sum of squared residuals — identical to the OLS objective.

32. What is multicollinearity and how do you detect it? Near-linear dependence among features; detected via correlation matrices, variance inflation factors, or the condition number of X^TX.

33. Why does Ridge regression's closed form always exist even when OLS's doesn't? Adding λI to X^TX makes it positive definite (hence invertible) regardless of the original matrix's rank.

34. Derive the gradient of logistic regression's log loss. → Section 75.2; the sigmoid's derivative cancels the log-loss derivative, leaving $X^T(\hat{p}-y)$.

35. What is the curse of dimensionality? As dimensionality grows, distances between points become increasingly similar, degrading the effectiveness of distance-based methods like KNN.

36. Why do decision trees not require feature scaling? Splits are threshold comparisons; monotonic rescaling changes threshold values but not the resulting row ordering/partition.

37. What is the Hessian and what does it tell you? The matrix of second partial derivatives; its eigenvalue signs classify a critical point as a minimum, maximum, or saddle point.

38. Why do saddle points dominate high-dimensional non-convex optimization? A critical point is a minimum only if all eigenvalues of the Hessian are positive; with many parameters, this becomes exponentially unlikely under a random-sign assumption.

39. What problem does Adam solve relative to plain SGD? It adapts the effective learning rate per parameter using accumulated gradient statistics, handling ill-conditioned loss surfaces where a single global learning rate performs poorly.

40. Explain the central limit theorem in one sentence. The distribution of a sample mean approaches normal as sample size grows, regardless of the underlying population's distribution.

41. Why is PCA sensitive to feature scale? It maximizes variance, and features with larger numeric scale naturally have larger variance, dominating the components regardless of true importance.

42. What is the geometric interpretation of least squares? $\hat{y}$ is the orthogonal projection of y onto the column space of X; the residual is perpendicular to every feature.

43. Why does K-Means converge to a local, not global, minimum? Its objective is non-convex; different centroid initializations can converge to different final partitions.

44. What is the vanishing gradient problem and why does ReLU help? Repeated multiplication by small activation derivatives (like sigmoid's ≤0.25) shrinks gradients toward zero across depth; ReLU's derivative of exactly 1 for positive inputs avoids this shrinkage.

45. Why is Gini impurity the scikit-learn default over entropy? It avoids logarithm computation (faster) and in practice produces nearly identical tree splits to entropy.

46. What does the condition number of a matrix indicate? How much the matrix amplifies input errors; a high condition number signals numerical instability in solving linear systems involving it.

47. Derive why variance can be computed as $E[X^2] - (E[X])^2$. Expand $E[(X-\mu)^2]$ using linearity of expectation; the cross term simplifies since $E[X]=\mu$.

48. Why is standard deviation preferred over variance for interpretation? Variance is in squared units, which lack intuitive meaning; standard deviation returns to the original units.

49. What is the role of $\sqrt{d_k}$ in Transformer attention? It keeps the variance of the dot-product attention scores approximately constant regardless of embedding dimension, preventing softmax saturation.

50. Explain why cosine similarity is preferred over Euclidean distance for text embeddings. Document length varies but topic similarity should not depend on length; cosine ignores magnitude and measures only direction.

51. What is the difference between covariance and correlation? Correlation is covariance normalized by the standard deviations of both variables, making it unit-free and bounded in $[-1,1]$.

52. Why can a feature have zero correlation with the target yet still be predictive? Correlation only captures linear relationships; a feature can have a strong non-linear or interaction effect while showing near-zero linear correlation.

53. What does a negative R^2 on test data indicate? The model performs worse than simply predicting the mean of the target — a serious warning sign.

54. Why must gradient checking use central differences rather than forward differences? Central differences have error proportional to h^2, vs h for forward differences, giving a materially more accurate approximation for the same step size.

55. What is the practical effect of setting $\lambda=0$ in Ridge regression? It reduces to ordinary least squares — the regularization term vanishes entirely.

100.3 Advanced (56–95)

56. Prove that the gradient points in the direction of steepest ascent. → Section 26.2, using $D_u f = \|\nabla f\|\cos(\varphi)$, maximized when $\varphi=0$.

57. Why does Naive Bayes work well despite its independence assumption being almost always false? It only needs the class with maximum posterior to be correct, not the posterior probabilities themselves; the independence error often distorts all classes similarly, preserving the ranking.

58. Derive the update rule for K-Means centroids from the WCSS objective. Minimizing $\Sigma\|x-\mu\|^2$ w.r.t. μ, by setting the derivative to zero, yields μ = mean of assigned points.

59. Why is the eigen-decomposition of a covariance matrix always real and orthogonal? Covariance matrices are symmetric; symmetric matrices have real eigenvalues and orthogonal eigenvectors by the spectral theorem.

60. Explain training-serving skew mathematically. The feature distribution or computation logic used at training time (X_train) differs from that used at inference time (X_serve), so the learned mapping f(X_train)→y no longer generalizes correctly to f(X_serve).

61. Derive why Bessel's correction uses n−1 rather than n. The sample mean is itself estimated from the data, making the sum of squared deviations from it systematically smaller than deviations from the true population mean; dividing by n−1 corrects this downward bias.

62. What is the mathematical justification for using log-likelihood instead of likelihood in MLE? Likelihood is a product of many probabilities, which underflows to zero in floating point for even moderate sample sizes; log converts the product to a numerically stable sum while preserving the location of the maximum since log is monotonic.

63. Explain why Adam's bias correction terms are necessary. The first and second moment estimates m and v are initialized at zero, biasing early estimates toward zero; dividing by $(1-\beta^t)$, which is small early in training, corrects this.

64. Prove that ridge regression's solution is always unique. $(X^TX + \lambda I)$ is positive definite for any $\lambda>0$ regardless of X's rank, hence invertible, giving a unique solution.

65. Why does the Hessian have n^2 entries for n parameters, and why does this matter for large models? Each pair of parameters contributes one second-derivative entry; for millions of parameters this becomes computationally and memory-prohibitive, which is why deep learning relies on first-order (gradient-only) optimization methods.

66. Derive the gradient of the Ridge-penalized loss. $\nabla(\text{Loss} + \lambda\|\beta\|^2) = \nabla\text{Loss} + 2\lambda\beta$ — the ordinary gradient plus a term proportional to the current coefficients, which is why Ridge shrinks coefficients smoothly.

67. What is the mathematical reason tree-based models don't need one-hot encoding as urgently as linear models? Trees split on individual feature thresholds regardless of encoding scheme, and can approximate categorical splits through repeated binary partitioning, whereas linear models require the encoding to make each category's effect additive and linear.

68. Prove that softmax reduces to sigmoid when there are exactly two classes. Write softmax for classes $\{z_1, z_2\}$; dividing numerator and denominator by e^{z_1} yields $1/(1+e^{(z_2-z_1)})$ — exactly the sigmoid form with score (z_1-z_2).

69. Why does scaled dot-product attention divide specifically by $\sqrt{d_k}$ rather than d_k? The variance of a sum of d_k independent products scales linearly with d_k, so the standard deviation scales with $\sqrt{d_k}$; dividing by $\sqrt{d_k}$ (not d_k) exactly normalizes the standard deviation back to a constant.

70. Explain the mathematical link between MLE and cross-entropy loss for multi-class classification. The categorical likelihood over one-hot labels, negated and logged, reduces exactly to the categorical cross-entropy formula — both are the negative log-likelihood of a Categorical/Multinoulli distribution.

71. Why is the WCSS objective of K-Means non-convex, and what practical consequence follows? The objective jointly optimizes discrete cluster assignments and continuous centroids,

creating multiple local optima; practically this means multiple random initializations (`n_init`) and smarter seeding (`k-means++`) are needed to avoid poor local solutions.

72. Derive the relationship between Euclidean distance and cosine similarity for unit vectors. $\|x-y\|^2 = \|x\|^2+\|y\|^2-2x\cdot y = 2-2\cos(\theta)$ when $\|x\|=\|y\|=1$, showing the two metrics induce the same ordering on unit-normalized vectors.

73. Why does dropping one dummy variable prevent the multicollinearity in one-hot encoding? The k dummy columns always sum to exactly 1 (matching the intercept column), a perfect linear dependency; dropping one column removes this exact redundancy while preserving full information via the reference-category convention.

74. Explain why deep learning optimizers use mini-batches rather than full-batch or single-sample updates. Mini-batches balance gradient accuracy (full-batch's advantage) against computational parallelism and frequent updates (single-sample's advantage), while the moderate noise also helps escape saddle points and shallow local minima.

75. Derive why irreducible error sets a hard floor on achievable MSE. If $y = f(x) + \varepsilon$ with $\mathrm{Var}(\varepsilon)=\sigma^2$ and $E[\varepsilon]=0$, then even the theoretically perfect model $f=f$ achieves expected squared error exactly σ^2, since the noise term cannot be predicted from x by any function.

76. What does a Hessian eigenvalue of exactly zero indicate, and why is this common in deep learning? A flat direction in the loss landscape (neither increasing nor decreasing locally); with vast overparameterization, many parameter directions leave the loss unchanged, producing many zero (or near-zero) eigenvalues.

77. Why is the log-odds interpretation of logistic regression coefficients considered "additive" while probabilities are not? The linear predictor $X\beta$ is additive in log-odds space by construction; the sigmoid transform back to probability space is non-linear, so a fixed coefficient's effect on probability depends on the baseline probability, unlike its constant effect on log-odds.

78. Explain the mathematical role of the discount factor γ beyond preference for immediate rewards. It ensures the infinite geometric series of future rewards converges to a finite value, making the optimization problem well-defined for infinite-horizon tasks.

79. Why can PCA's first principal component be interpreted as the direction maximizing projected variance? It's the constrained-optimization solution to $\max v^T C v$ subject to $\|v\|=1$, which by Lagrange multipliers is exactly the top eigenvector of the covariance matrix C.

80. Derive why the residuals from a least-squares fit are orthogonal to every column of X. The normal equation $X^TX\beta=X^Ty$ rearranges to $X^T(y-X\beta)=0$, i.e. $X^T\cdot\text{residual}=0$ — each column of X has zero dot product with the residual vector.

81. Why does batch normalization (conceptually) help training stability? By standardizing layer inputs to zero mean/unit variance at each mini-batch, it keeps activations in a well-conditioned range across depth, mitigating both vanishing and exploding gradient issues layer to layer.

82. What is the mathematical distinction between a local and global minimum, and why does convexity eliminate the difference? A local minimum is lowest only within a neighbourhood; a global minimum is lowest everywhere. For convex functions, any local minimum is provably the global minimum, since the function cannot have a "dip" elsewhere lower than a local one due to the convexity inequality.

83. Explain why Naive Bayes computations are performed in log-space in practice. The joint likelihood is a product of many per-feature probabilities, each typically small; multiplying many small numbers underflows to zero in floating point, while summing their logs remains numerically stable.

84. Derive the closed-form Ridge solution from the penalized objective. $J(\beta) = \|y - X\beta\|^2 + \lambda\|\beta\|^2$; differentiate, set to zero: $-2X^Ty + 2X^TX\beta + 2\lambda\beta = 0 \rightarrow (X^TX+\lambda I)\beta = X^Ty \rightarrow \beta = (X^TX+\lambda I)^{-1}X^Ty$.

85. Why is entropy measured in "bits" when using log base 2? Each bit of entropy corresponds to one binary yes/no question needed on average to resolve the uncertainty, directly connecting information theory to a literal binary encoding scheme.

86. What mathematical property guarantees K-Means will always terminate? Both the assignment and update steps can only decrease (never increase) the WCSS objective, and there is a finite number of possible cluster assignments, so the sequence must eventually stop changing.

87. Explain the mathematical reason gradient descent zigzags on elongated (ill-conditioned) loss surfaces. The gradient vector's direction is not generally aligned with the direction toward the minimum when the Hessian's eigenvalues differ greatly in magnitude; the gradient points more steeply along the high-curvature axis than toward the true minimum.

88. Why does the He initialization scheme use variance 2/n_in specifically for ReLU networks? Because ReLU zeroes out roughly half its inputs, the effective variance of the output is halved relative to a linear layer; scaling initial weight variance by 2/n_in compensates so the forward-pass activation variance remains stable across layers.

89. Derive why maximizing likelihood is equivalent to minimizing negative log-likelihood. Log is a strictly monotonically increasing function, so $\text{argmax } f(\theta) = \text{argmax } \log f(\theta)$ for any positive f; negating flips this to an argmin, giving $\text{argmax } L(\theta) = \text{argmin}[-\log L(\theta)]$.

90. What does it mean for a loss function to be a "matched" loss for an activation function, using logistic regression as the example? The loss and activation are derived from the same underlying probabilistic model (Bernoulli/sigmoid), causing their gradients to combine algebraically into a simple closed form $(\hat{p} - y)$ rather than a complex product of derivatives.

91. Why does dimensionality reduction via PCA sometimes hurt supervised model performance despite reducing noise? PCA selects directions of maximum *variance* in X, which need not align with directions most predictive of y; a low-variance direction could still carry strong signal for the target, and PCA would discard it.

92. Explain mathematically why Lasso can produce a model with fewer effective parameters than Ridge for the same λ. Lasso's L1 penalty geometry (diamond-shaped constraint region with corners on the axes) makes solutions land exactly on the axes (some coefficients = 0) far more often than Ridge's smooth L2 sphere, which almost never touches an axis exactly.

93. Derive the relationship between the Poisson distribution's mean and variance, and explain its diagnostic use. Both equal λ by the distribution's definition; if observed count data shows variance substantially exceeding the mean ("overdispersion"), the Poisson assumption is violated and a different model (e.g. negative binomial) is warranted.

94. Why is the softmax function's Jacobian more complex than the sigmoid's simple $\sigma(1-\sigma)$ form? Because each output of softmax depends on every input (through the shared normalizing sum), the derivative of one output with respect to one input requires accounting for cross-terms between classes, producing a full Jacobian matrix rather than a single scalar derivative per unit.

95. Explain, using the bias-variance decomposition, why ensembling (e.g. Random Forest) reduces variance without necessarily changing bias. Averaging predictions from many models trained on different bootstrap samples reduces the variance of the average roughly by a factor related to the number of models (assuming low correlation between them), while each individual model's bias (a property of the model class, not the averaging process) is essentially unchanged.

100.4 Mathematical Coding Questions (96–125, abbreviated to a representative set)

96–100. Implement, from scratch and matching NumPy's built-ins to within floating-point tolerance: mean/variance/std (→ Section 99.1), dot product (→ 99.2), matrix multiplication (→ 99.3), stable sigmoid (→ 11.5), stable softmax (→ 12.4).

101–105. Implement: MSE/RMSE/MAE/R^2 (→ 99.6), log loss with clipping (→ 70.3), gradient descent for a 1D quadratic (→ 28.3), central-difference numerical differentiation (→ 99.9), gradient checking (→ 89.3).

106–110. Implement: standardization/min-max/robust scaling (→ 85.2), Euclidean/Manhattan/cosine distance (→ 16.2, 87.2), PCA from scratch (→ 52.3), covariance matrix from scratch (→ 99.14), entropy/Gini/information gain (→ 79–81).

111–115. Implement: linear regression via `lstsq` (→ 74.4), logistic regression via gradient descent (→ 75.3), K-Means with the elbow method (→ 82), a 2-layer neural network solving XOR (→ 91.2), scaled dot-product attention (→ 92.3).

100.5 Scenario-Based Questions (126–155, abbreviated to a representative set)

126. Your gradient descent loss explodes to `inf` after a few iterations. What do you check first, mathematically? The learning rate against the theoretical stability bound for your loss's curvature (Section 31.2); an α too large relative to the Hessian's largest eigenvalue causes divergence.

127. Your model's coefficients are enormous and have unstable signs across retraining runs. What mathematical property should you check? The condition number of X^TX (Section 44.4) — likely multicollinearity; consider Ridge regularization or removing redundant features.

128. A stakeholder asks for 99.9% accuracy on a 0.1% positive-rate problem. How do you respond mathematically? Note that predicting the majority class for everything already achieves 99.9% accuracy while providing zero value; redirect to precision/recall or expected-value framing (Section 57.3, 59.2).

129. Your softmax outputs are all `nan`. What is the mathematical root cause and the fix? Overflow from large logits before normalization (Section 12.4); subtract the max logit before exponentiating.

130. Two features in your dataset have correlation 0.999. What should you investigate, and what's the mathematical risk if you ignore it? Near-perfect multicollinearity; X^TX will be near-singular, producing unstable, uninterpretable coefficients (Section 47.3).

131. Your neural network's loss stays exactly flat from epoch 1 — it never trains at all. What's the first mathematical hypothesis? Weights were likely initialized to all zeros, causing the symmetry problem (Section 91.3) — every neuron computes and receives identical values.

132. You need to explain why your fraud model, despite 90% recall, generates mostly false alerts, to a non-technical stakeholder. Frame it through Bayes' theorem and the base-rate effect (Section 57.3): even a highly accurate test produces mostly false positives when the underlying condition is very rare.

133. Your K-Means results change every time you rerun it. Explain why, mathematically, and how to fix it. The WCSS objective is non-convex (Section 82.2); different random initializations converge to different local minima. Use `k-means++` initialization and multiple random restarts, keeping the lowest-WCSS result.

134. A time-series model has excellent test accuracy in offline validation but performs poorly live. What splitting mistake is the likely mathematical cause? A random (rather than time-based) train/test split, letting the model train on data temporally after some of its "test" points — an information leak invisible in the offline metric.

135. Your Ridge regression's coefficients barely shrink even with a large λ. What should you check? Whether features were standardized before fitting (Section 72.4) — an unstandardized large-scale feature's penalty term is comparatively tiny relative to its loss contribution, blunting the intended shrinkage.

---

101. Mathematical Troubleshooting

Format: Symptom → Mathematical diagnosis → Fix → Prevention.

1. Exploding gradients. *Symptom:* loss becomes `inf`/`nan` within a few iterations. *Diagnosis:* learning rate exceeds the stability threshold relative to the loss surface's curvature (Section 31.2), or gradients genuinely have very large magnitude in a deep/recurrent network. *Fix:* reduce the learning rate; apply gradient clipping (Section 38.4). *Prevention:* monitor gradient norms during training; use adaptive optimizers (Section 83).

2. Vanishing gradients. *Symptom:* early layers of a deep network stop updating; loss plateaus at a poor value. *Diagnosis:* repeated multiplication by small activation derivatives (sigmoid/tanh, Section 91.1 equation 2) shrinks the gradient toward zero across depth. *Fix:* switch to ReLU-family activations; use residual connections; use batch normalization. *Prevention:* prefer ReLU by default in hidden layers; monitor per-layer gradient magnitudes.

3. Unstable/oscillating loss. *Symptom:* loss bounces up and down rather than smoothly decreasing. *Diagnosis:* learning rate too large relative to the local curvature (Section 31.2). *Fix:* reduce α; use a learning-rate schedule. *Prevention:* always plot the loss curve; tune α on a log scale.

4. Learning rate too high. *Symptom:* loss diverges to infinity. *Diagnosis:* the update magnitude exceeds the stable range for the loss's curvature. *Fix:* reduce α by a factor of 10 and retest. *Prevention:* start conservative (0.001–0.01) and increase cautiously.

5. Learning rate too low. *Symptom:* loss decreases imperceptibly slowly. *Diagnosis:* step sizes are far smaller than needed to make meaningful progress within a reasonable iteration budget. *Fix:* increase α by 3–10×. *Prevention:* plot loss over the first few dozen iterations before committing to a long run.

6. Singular matrix error. *Symptom:* `LinAlgError: Singular matrix` when inverting or solving. *Diagnosis:* the matrix has linearly dependent rows/columns — often from perfect multicollinearity or the dummy variable trap (Sections 44.3, 48.3). *Fix:* remove redundant features; use `lstsq` instead of direct inversion; add Ridge regularization. *Prevention:* check `matrix_rank` and `cond` before relying on `inv`/`solve`.

7. Multicollinearity. *Symptom:* wildly large, unstable, or sign-flipping coefficients. *Diagnosis:* near-linear dependence among features inflates the condition number of X^TX (Section 47.3). *Fix:* drop or combine redundant features; apply Ridge. *Prevention:* screen pairwise correlations and VIF before fitting linear models.

8. Numerical overflow. *Symptom:* `inf` values, or a `RuntimeWarning: overflow encountered`. *Diagnosis:* an exponential or product exceeded float64's range (~1.8e308, exp(~709)) (Section 9.4). *Fix:* subtract the max before exponentiating (softmax); work in log-space for products of probabilities. *Prevention:* audit any `np.exp` call for unbounded inputs.

9. Numerical underflow. *Symptom:* a probability or gradient silently becomes exactly 0.0. *Diagnosis:* the true value is smaller than float64's minimum representable magnitude, or is the product of many small probabilities (Section 10.3). *Fix:* work in log-space. *Prevention:* be suspicious of any product over many terms in [0,1].

10. Poor scaling. *Symptom:* a distance-based model (KNN, K-Means) or gradient descent performs oddly, dominated by one feature. *Diagnosis:* unscaled features with vastly different numeric ranges dominate distance/gradient calculations (Section 16.3). *Fix:* standardize or

normalize features. *Prevention:* scale by default for any distance-based or gradient-based method.

11. Incorrect distance calculation. *Symptom:* KNN/K-Means results look nonsensical. *Diagnosis:* likely a scaling issue (#10), a shape mismatch producing an unintended broadcast (Section 96.2), or confusing squared distance with distance. *Fix:* verify shapes explicitly; confirm the distance formula against a hand-computed example (Section 16.2). *Prevention:* unit-test distance functions against known values.

12. Wrong probability interpretation. *Symptom:* a probability computation exceeds 1 or is negative, or a density is mistaken for a probability. *Diagnosis:* likely confusing a PDF value (Section 58.2, which can exceed 1) with an actual probability, or a computation bug. *Fix:* remember $P(a \le X \le b)$ requires integrating the density over a range, not reading a single density value. *Prevention:* be explicit about discrete (PMF) vs continuous (PDF) when reasoning about probability magnitudes.

13. Data leakage (mathematical manifestation). *Symptom:* suspiciously excellent metrics that don't reproduce in production. *Diagnosis:* a feature encodes information about the target unavailable at prediction time, or preprocessing statistics were computed on data including the test set (Section 64.4). *Fix:* audit every feature's availability timing; always fit transformations on training data only. *Prevention:* split first, transform after, as a standing rule.

14. Shape mismatch broadcasting silently. *Symptom:* a loss value looks wrong but no error was raised. *Diagnosis:* NumPy broadcast a `(n,)` array against a `(n,1)` array, producing an unintended `(n,n)` result (Section 96.2). *Fix:* explicitly `.ravel()` or `.reshape()` arrays to their intended shape before combining. *Prevention:* print `.shape` at every step during debugging.

15. Convergence never reached. *Symptom:* gradient descent runs the full iteration budget without the loss flattening. *Diagnosis:* learning rate too small, or the problem is genuinely non-convex and stuck oscillating around a region. *Fix:* increase α; check the gradient norm directly rather than only the iteration count (Section 89.2). *Prevention:* always track and inspect the loss/gradient-norm history, not just the final result.

16. Catastrophic cancellation. *Symptom:* a variance or difference computation returns an implausible or negative value. *Diagnosis:* subtracting two nearly-equal large numbers destroys precision (Section 84.2, item 5). *Fix:* use the numerically stable two-pass formula (deviations from the mean) rather than the $E[X^2]-(E[X])^2$ shortcut when values are large. *Prevention:* prefer library-provided statistics functions, which already use stable formulas.

17. Overfitting visible via bias-variance gap. *Symptom:* very low training loss, high test loss. *Diagnosis:* high model variance relative to the available data (Section 73). *Fix:* regularize, simplify the model, or gather more data. *Prevention:* always report train and test metrics together, never train alone.

18. Underfitting visible via both losses being poor. *Symptom:* both training and test loss are high and similar. *Diagnosis:* high model bias — the model class cannot represent the true relationship (Section 73). *Fix:* increase model complexity or add informative features. *Prevention:* compare against a simple baseline early to detect this quickly.

19. Gradient check fails. *Symptom:* a hand-derived analytical gradient disagrees with the numerical gradient beyond tolerance. *Diagnosis:* an algebraic error in the derivation, or a sign/indexing bug in the implementation (Section 89.3). *Fix:* re-derive term by term; check each intermediate partial derivative individually. *Prevention:* always gradient-check before trusting a hand-implemented gradient.

20. Non-convex optimization gives inconsistent results across runs. *Symptom:* retraining the same neural network or K-Means model gives different results each time. *Diagnosis:* this is an expected consequence of non-convexity (Section 32.3), not necessarily a bug. *Fix:* set random seeds for reproducibility during debugging; use multiple restarts and report the distribution of outcomes, not a single run, for production decisions. *Prevention:* understand which model classes are convex (and hence exactly reproducible) versus not, from the outset.

21. Log loss reported as negative. *Symptom:* a computed "log loss" is negative, which should be impossible. *Diagnosis:* a sign error in the implementation — likely forgetting the leading negative sign in $-(1/n)\Sigma[\ldots]$. *Fix:* re-check the formula against Section 70.3 term by term. *Prevention:* sanity-check that loss values for a known input (e.g. all-0.5 predictions) equal the expected $\ln(2)$ baseline.

22. Softmax outputs don't sum to 1. *Symptom:* `weights.sum()` returns something other than 1.0. *Diagnosis:* likely normalizing over the wrong axis in a batched computation. *Fix:* explicitly specify `axis=-1` (or the correct feature axis) in both the max-subtraction and the sum. *Prevention:* always verify `.sum(axis=...)` equals 1 for every row after implementing softmax.

23. Eigenvalues come back complex. *Symptom:* `np.linalg.eig` returns complex numbers for what should be a real-valued problem. *Diagnosis:* the matrix is not symmetric (e.g. a bug computing the covariance matrix introduced asymmetry). *Fix:* verify symmetry with `np.allclose(A, A.T)`; use `np.linalg.eigh` for genuinely symmetric matrices, which guarantees real output. *Prevention:* always use `eigh` (not `eig`) for covariance/Hessian-type symmetric matrices.

24. Standardized data doesn't have mean 0/std 1. *Symptom:* after applying `(x-mean)/std`, the resulting mean or std check fails. *Diagnosis:* the mean/std used were computed on a different dataset (e.g. test statistics applied to train, or vice versa) — this is actually expected and correct for a test set scaled with training statistics; but if it fails on the training set itself, there's likely a broadcasting bug. *Fix:* verify the computation is applied along the correct axis. *Prevention:* explicitly test standardization on the training set only, where mean≈0/std≈1 should hold exactly.

25. Ridge doesn't shrink coefficients at all even with large λ. *Symptom:* β_ridge ≈ β_OLS regardless of λ. *Diagnosis:* λ wasn't actually applied to the correct term, or features are unscaled so the penalty is negligible relative to the loss (Section 72.4). *Fix:* verify the regularization term is added correctly and that features are standardized first. *Prevention:* test regularization strength on a toy dataset with a known correct answer before trusting it on real data.

26. A probability distribution's PMF/PDF doesn't integrate/sum to 1. *Symptom:* `pmf.sum()` or a numerical integral of a PDF isn't 1. *Diagnosis:* incorrect normalization constant

in a hand-implemented distribution formula. *Fix:* re-derive the normalizing constant (Section 63.1 for the Gaussian's $1/(\sigma\sqrt{2\pi})$, for instance). *Prevention:* always validate a custom distribution implementation by checking it integrates/sums to 1 over its full domain.

27. Cross-validation scores vary enormously across folds. *Symptom:* fold scores range widely (e.g. 0.5 to 0.9 AUC). *Diagnosis:* high model variance (Section 73), often compounded by a small dataset or severe class imbalance in some folds. *Fix:* use stratified k-fold for imbalanced classification; consider a simpler, lower-variance model; gather more data. *Prevention:* always report the spread (not just the mean) of cross-validation scores.

28. A confusion matrix implies impossible values (e.g. negative counts). *Symptom:* manually computed precision/recall don't match a sanity check. *Diagnosis:* a bug in how true/false positive/negative counts were tallied, often from a threshold or label-encoding mismatch. *Fix:* recompute using a trusted library function and compare. *Prevention:* validate confusion matrix computations against a small hand-worked example first.

29. Attention weights collapse to a one-hot-like distribution unexpectedly. *Symptom:* one token receives ~100% attention weight, others ~0%. *Diagnosis:* unscaled attention scores (missing the $\sqrt{d_k}$ division, Section 92.2) causing extreme softmax saturation. *Fix:* verify the scaling division is present and using the correct d_k. *Prevention:* check the standard deviation of raw attention scores before softmax; it should be near 1 after correct scaling.

30. A model's feature importances contradict domain knowledge entirely. *Symptom:* a feature known to be irrelevant dominates importance, or a known-important feature shows near-zero importance. *Diagnosis:* likely target leakage (the "irrelevant" feature is actually a target proxy) or multicollinearity redistributing importance unpredictably among correlated features. *Fix:* investigate the suspicious feature's definition and timing; check correlations among top features. *Prevention:* always sanity-check top feature importances against domain expectations before trusting a model.

102. Formula Master Sheet

Formula → Meaning → Symbols → ML application, organized by branch.

102.1 Algebra & Functions

Formula	Meaning	ML Application
$y = mx + b$	Line: slope m, intercept b	Simple linear regression
$\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_m x_m$	Multiple linear regression	The general linear model
$\sigma(x) = 1/(1+e^{-x})$	Sigmoid: squash $\mathbb{R} \to (0,1)$	Binary classification output
$\text{softmax}(z)_i = e^{z_i}/\Sigma_j e^{z_j}$	Convert scores to a probability distribution	Multi-class output
$f(g(x))$	Function composition	Neural network layers

102.2 Logarithms & Exponents

Formula	Meaning	ML Application
$\log(ab) = \log a + \log b$	Product → sum	Likelihood → log-likelihood
$\ln(e^x) = x$	Inverse relationship	Undoing exponentials
$d/dx\ e^x = e^x$	Self-derivative	Why e is everywhere in calculus-based ML

102.3 Calculus

Formula	Meaning	ML Application
$f'(x) = \lim_{h\to 0} [f(x+h) - f(x)]/h$	Derivative definition	Foundation of all gradients
$d/dx\ x^n = nx^{(n-1)}$	Power rule	Polynomial terms in loss functions
$(fg)' = f'g + fg'$	Product rule	Deriving compound expressions
$(f(g(x)))' = f'(g(x))g'(x)$	Chain rule	Backpropagation
$\partial f/\partial x$	Partial derivative	Gradient components
$\nabla f = [\partial f/\partial x_1, \ldots, \partial f/\partial x_n]$	Gradient vector	Direction of steepest increase
$D_u f = \nabla f \cdot u$	Directional derivative	Proof that gradient = steepest ascent
$\sigma'(x) = \sigma(x)(1-\sigma(x))$	Sigmoid derivative	Logistic regression gradient
$H = [\partial^2 f/\partial x_i \partial x_j]$	Hessian matrix	Curvature; saddle/min/max classification

102.4 Optimization

Formula	Meaning	ML Application
$\theta^* = \text{argmin}_\theta\ L(\theta)$	Optimal parameters	The universal training objective

Formula	Meaning	ML Application
$\theta := \theta - \alpha \nabla J(\theta)$	Gradient descent update	Training any differentiable model
$v := \gamma v + \alpha \nabla J;\ \theta := \theta - v$	Momentum	Faster, smoother convergence
m,v moment estimates; $\theta := \theta - \alpha \cdot \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$	Adam	The default deep learning optimizer

102.5 Linear Algebra

Formula	Meaning	ML Application
$x \cdot y = \Sigma x_i y_i = \|x\|\|y\|\cos(\theta)$	Dot product	Linear models, attention, similarity
$\|x\|_2 = \sqrt{(\Sigma x_i^2)}$	Euclidean norm	Distance, regularization
$\|x\|_1 = \Sigma \|x_i\|$	Manhattan norm	Lasso regularization
$A@B,\ c_{ij} = \Sigma_k a_{ik} b_{kj}$	Matrix multiplication	Every layer of every model
A^T	Transpose	Normal equation, gradients
$AA^{-1} = I$	Matrix inverse	(Avoid computing explicitly — Section 44.4)
$Ax = b$	Linear system	The core computational primitive
$\det(A)$	Determinant	Invertibility, volume scaling
$Av = \lambda v$	Eigenvalue equation	PCA, Hessian analysis
$Cov(X,Y) = (1/(n-1))\Sigma(x_i - \bar{x})(y_i - \bar{y})$	Covariance	PCA, feature relationships
$r = Cov(X,Y)/(\sigma_X \sigma_Y)$	Pearson correlation	Feature analysis

102.6 Probability & Bayes

Formula	Meaning	ML Application
$P(A\|B) = P(A \cap B)/P(B)$	Conditional probability	Naive Bayes, dependency modeling
$P(A\|B) = P(B\|A)P(A)/P(B)$	Bayes' theorem	Naive Bayes, prior/posterior updating
$E[X] = \Sigma x \cdot P(X=x)$	Expectation	Expected loss/value, risk
$Var(X) = E[X^2] - (E[X])^2$	Variance	Spread, bias-variance
$\sigma = \sqrt{Var(X)}$	Standard deviation	Standardization
$z = (x - \mu)/\sigma$	Z-score	Standardization, outlier detection

102.7 Distributions

Formula	Meaning	ML Application
$P(X=1)=p,\ P(X=0)=1-p$	Bernoulli	Binary classification's target

Formula	Meaning	ML Application
$C(n,k)p^k(1-p)^{n-k}$	Binomial	Count of successes in n trials
$(1/\sigma\sqrt{2\pi})e^{(-(x-\mu)^2/2\sigma^2)}$	Normal PDF	Assumed noise in linear regression
$\lambda^k e^{-\lambda}/k!$	Poisson	Rare-event counts

102.8 Estimation

Formula	Meaning	ML Application
$\hat{\theta}_{MLE} = \text{argmax}_\theta L(\theta)$	Maximum Likelihood Estimation	Derives most standard loss functions
$-(1/n)\Sigma \log L$	Negative log-likelihood	The general form of trained losses

102.9 Loss Functions

Formula	Meaning	ML Application
$(1/n)\Sigma(y_i-\hat{y}_i)^2$	MSE	Regression
$(1/n)\Sigma\lvert y_i-\hat{y}_i\rvert$	MAE	Outlier-robust regression
$-(1/n)\Sigma[y\cdot\log \hat{p}+(1-y)\log(1-\hat{p})]$	Binary cross-entropy	Binary classification
$-(1/n)\Sigma\Sigma y\cdot\log(\hat{p})$	Categorical cross-entropy	Multi-class classification

102.10 Regularization

Formula	Meaning	ML Application
$\text{Loss} + \lambda\lVert\beta\rVert_2^2$	Ridge (L2)	Shrinks all coefficients smoothly
$\text{Loss} + \lambda\lVert\beta\rVert_1$	Lasso (L1)	Drives some coefficients to exactly 0
$\beta_{ridge}=(X^TX+\lambda I)^{-1}X^Ty$	Ridge closed form	Numerically stable regression

102.11 Linear/Logistic Regression

Formula	Meaning	ML Application
$\beta = (X^TX)^{-1}X^Ty$	Normal equation	Closed-form linear regression (rarely computed this way)
$\nabla J = (2/n)X^T(X\beta-y)$	Linear regression gradient	Gradient descent training
$\hat{p} = \sigma(X\beta)$	Logistic model	Binary classification
$\nabla J = (1/n)X^T(\hat{p}-y)$	Logistic regression gradient	Gradient descent training
$\log(p/(1-p)) = X\beta$	Log-odds	Coefficient interpretation

102.12 Trees & Clustering

Formula	Meaning	ML Application
$H(S) = -\Sigma p_i \log_2 p_i$	Entropy	Decision tree split quality
IG = H(parent) − Σ(weighted child H)	Information gain	Decision tree split selection
$Gini = 1 - \Sigma p_i^2$	Gini impurity	Alternative split criterion
$J = \Sigma\Sigma \|x_i - \mu_k\|^2$	K-Means objective (WCSS)	Clustering quality

102.13 Neural Networks & Attention

Formula	Meaning	ML Application
z = Wx + b; a = f(z)	Forward propagation	Every neural network layer
δ^l = $(W^{(l+1)}\delta(l+1)) \odot f'(z^l)$	Backpropagation	Gradient computation through layers
Attention(Q,K,V) = softmax($QK^T/\sqrt{d_k}$)V	Scaled dot-product attention	Transformers, LLMs

103. Mathematics → ML Algorithm Map

Mathematical Concept	ML Algorithm	Why It Matters
Derivatives	Gradient Descent	Finding the direction that reduces loss
Chain rule	Backpropagation	Computing gradients through composed layers
Partial derivatives	Multi-parameter model training	Every model with >1 parameter
Vectors	KNN, feature representation	Distance-based classification/regression
Dot product	Linear/logistic regression, attention	The core linear operation everywhere
Matrices	Neural networks	Parameter transformations at scale
Matrix multiplication	Every deep learning layer	Batch computation across samples
Eigenvectors	PCA	Dimensionality reduction
Eigenvalues of the Hessian	Optimization landscape analysis	Distinguishing minima from saddle points
Determinant/rank	Detecting multicollinearity	Diagnosing unstable linear models
Probability	Naive Bayes	Classification via Bayes' theorem
Conditional probability +	Naive Bayes, medical/fraud	Updating belief with

Mathematical Concept	ML Algorithm	Why It Matters
Bayes	scoring	evidence
Expectation	Loss functions, RL returns	Formalizing “average outcome”
Variance/covariance	PCA, standardization, bias-variance	Spread and feature relationships
Normal distribution	Linear regression’s assumed noise, z-scores	Justifying MSE, outlier detection
MLE	Deriving loss functions	Log loss, MSE, cross-entropy all derive from MLE
Entropy	Decision trees, cross-entropy loss	Split selection; classification loss
Logarithms	Cross-entropy, log-likelihood	Numerical stability, additive likelihood
Convexity	Linear/logistic regression reliability	Guarantees a single global optimum
Norms (L1/L2)	Lasso/Ridge regularization	Controlling model complexity
Cosine similarity	Embeddings, RAG, recommendation	Semantic similarity in vector space
Central Limit Theorem	Statistical inference, confidence intervals	Justifies normal-approximation methods
Convexity of K-Means objective (its absence)	K-Means initialization strategy	Explains why multiple restarts are needed

104. End-to-End Mathematical ML Capstone

104.1 “From Mathematics to Machine Learning: Implement a Model From Scratch”

Choice: Logistic Regression, predicting loan default from financial features. No scikit-learn until the final comparison step.

```
import numpy as np

# ---------- 1. DATASET GENERATION ----------
rng = np.random.default_rng(42)
n_samples = 2000

income      = rng.normal(500_000, 150_000, n_samples)
debt_ratio  = rng.beta(2, 5, n_samples)                    # skewed
toward lower ratios
credit_yrs  = rng.integers(0, 25, n_samples).astype(float)
late_pays   = rng.poisson(1.2, n_samples).astype(float)

# ---------- 2. FEATURE MATRIX ----------
```

```
X_raw = np.column_stack([income, debt_ratio, credit_yrs, late_pays])
feature_names = ["income", "debt_ratio", "credit_years",
"late_payments"]

# Standardize — essential for gradient descent to converge well
(Section 25.3, 64.4)
X_mean, X_std = X_raw.mean(axis=0), X_raw.std(axis=0)
X = (X_raw - X_mean) / X_std

# ---------- 3. TARGET VECTOR ----------
# True underlying relationship (unknown to the model, known to us for
generation)
true_logit = (-0.3*X[:,0] + 2.0*X[:,1] - 0.8*X[:,2] + 0.9*X[:,3] -
0.5)
true_prob = 1 / (1 + np.exp(-true_logit))
y = (rng.random(n_samples) < true_prob).astype(float)
print(f"Default rate: {y.mean():.1%}")

# Train/test split (Module 1's Part 18 discipline, time-independent
here so random+stratified is fine)
idx = rng.permutation(n_samples)
split = int(0.8 * n_samples)
train_idx, test_idx = idx[:split], idx[split:]
X_train, X_test = X[train_idx], X[test_idx]
y_train, y_test = y[train_idx], y[test_idx]

# ---------- 4. MODEL PARAMETERS ----------
n_features = X_train.shape[1]
beta = np.zeros(n_features)
bias = 0.0

# ---------- 5. PREDICTION EQUATION ----------
def predict_proba(X, beta, bias):
    z = X @ beta + bias
    return np.where(z >= 0, 1/(1+np.exp(-z)), np.exp(z)/(1+np.exp(z)))
# stable (Section 11.5)

# ---------- 6. LOSS FUNCTION ----------
def compute_loss(y, p, eps=1e-15):
    p = np.clip(p, eps, 1-eps)
    return -np.mean(y*np.log(p) + (1-y)*np.log(1-p))
# Section 70.3

# ---------- 7. GRADIENT DERIVATION (already derived in Section 75.2)
----------
# ∇J_beta = (1/n) Xᵀ(p̂ − y)          ∇J_bias = (1/n) Σ(p̂ − y)

# ---------- 8. GRADIENT COMPUTATION ----------
def compute_gradients(X, y, p):
    n = len(y)
```

```
    error = p - y
    grad_beta = (X.T @ error) / n
    grad_bias = error.mean()
    return grad_beta, grad_bias

# ---------- 9. GRADIENT DESCENT ----------
alpha, n_iter = 0.5, 3000
loss_history = []
for i in range(n_iter):
    p = predict_proba(X_train, beta, bias)
    loss_history.append(compute_loss(y_train, p))
    grad_beta, grad_bias = compute_gradients(X_train, y_train, p)

    # ---------- 10. PARAMETER UPDATES ----------
    beta -= alpha * grad_beta
    bias -= alpha * grad_bias

# ---------- 11. CONVERGENCE MONITORING ----------
print(f"Loss: {loss_history[0]:.4f} → {loss_history[-1]:.4f}")
print(f"(Baseline, all-0.5 predictions: {np.log(2):.4f})")

# ---------- 12. PREDICTIONS ----------
p_test = predict_proba(X_test, beta, bias)
y_pred = (p_test >= 0.5).astype(int)

# ---------- 13. EVALUATION ----------
accuracy = (y_pred == y_test).mean()
tp = ((y_pred==1)&(y_test==1)).sum(); fp =
((y_pred==1)&(y_test==0)).sum()
fn = ((y_pred==0)&(y_test==1)).sum()
precision = tp/(tp+fp) if (tp+fp)>0 else 0
recall    = tp/(tp+fn) if (tp+fn)>0 else 0
print(f"Accuracy: {accuracy:.4f}  Precision: {precision:.4f}  Recall:
{recall:.4f}")

def auc_score(y_true, scores):
    # Rank-based AUC computation (Mann-Whitney U form)
    order = np.argsort(scores)
    ranks = np.empty(len(scores)); ranks[order] = np.arange(1,
len(scores)+1)
    n_pos, n_neg = y_true.sum(), len(y_true)-y_true.sum()
    if n_pos==0 or n_neg==0: return float("nan")
    return (ranks[y_true==1].sum() - n_pos*(n_pos+1)/2) /
(n_pos*n_neg)
print(f"AUC: {auc_score(y_test, p_test):.4f}")

# ---------- 14. VISUALIZATION (conceptual — see Section 97)
----------
# plt.plot(loss_history); plt.xlabel("iteration"); plt.ylabel("loss")
```

```
# ---------- 15. COMPARISON WITH SCIKIT-LEARN ----------
from sklearn.linear_model import LogisticRegression
sk_model = LogisticRegression().fit(X_train, y_train)
print("\nOur coefficients:      ", beta.round(4))
print("scikit-learn coefficients:", sk_model.coef_[0].round(4))
print(f"\nOur test accuracy:        {accuracy:.4f}")
print(f"scikit-learn test accuracy: {sk_model.score(X_test,
y_test):.4f}")
```

104.2 Explaining Every Mathematical Step

Steps 1-3 (data/features/target): deliberately generate data from a *known* underlying logistic relationship, so the capstone can verify the from-scratch implementation actually recovers approximately correct coefficients — a form of ground-truth validation impossible with real-world data.

Step 4 (parameters): initialized at zero, appropriate for logistic regression's convex loss surface (Section 32.3) where initialization doesn't affect the final answer, unlike neural networks (Section 91.3).

Steps 5-6 (prediction, loss): the sigmoid link and log loss, matched to each other as the Bernoulli MLE (Section 70.3) — not an arbitrary pairing.

Step 7 (gradient derivation): the beautiful cancellation from Section 75.2, where the sigmoid's and log loss's derivatives combine into simply $(\hat{p}-y)$.

Steps 8-10 (computation, descent, updates): the exact template from Section 29, applied here to logistic rather than linear regression — identical mechanics, different model.

Step 11 (convergence): comparing the final loss against $\ln(2)$ (the uninformed baseline, Section 70.3) confirms genuine learning occurred.

Steps 12-13 (predictions, evaluation): thresholding at 0.5 for hard predictions; a hand-implemented AUC (via the rank-sum/Mann-Whitney formulation) avoiding a library dependency, testing your understanding of what AUC actually measures (Section 57.3's ranking intuition, applied here).

Step 15 (comparison): the from-scratch coefficients and accuracy should closely match scikit-learn's — this final check is the ultimate validation that every mathematical step was implemented correctly, and is standard professional practice whenever hand-rolling an algorithm for learning purposes.

---

105. Final Mathematical Assessment

Representative sample across all required categories, with answers. Full-length assessment follows the same structure at greater volume.

105.1 Multiple Choice (1–15 of 40)

1. $\theta^* = \text{argmin}_\theta L(\theta)$ means: (a) the loss at the optimum (b) **the parameters minimizing the loss** (c) the maximum loss (d) a hyperparameter
2. $d/dx(x^4) = ?$ (a) x^3 (b) $4x^3$ (c) $\mathbf{4x^3}$ (d) $x^4/4$ — *(answer: $4x^3$)*
3. $\sigma'(x)$ equals: (a) $\sigma(x)$ (b) $\boldsymbol{\sigma(x)(1-\sigma(x))}$ (c) $1-\sigma(x)$ (d) $\sigma(x)^2$
4. The gradient points toward: (a) **steepest increase** (b) steepest decrease (c) the origin (d) a random direction
5. A@B requires: (a) same shape (b) **A's columns = B's rows** (c) both square (d) nothing
6. Ridge regression's closed form always exists because: (a) X is always invertible (b) $\boldsymbol{\lambda I}$ **guarantees positive definiteness** (c) it uses gradient descent (d) it doesn't have a closed form
7. Softmax's max-subtraction trick prevents: (a) underflow (b) **overflow** (c) slow computation (d) negative values
8. Bayes' theorem's "prior" term is: (a) $P(B|A)$ (b) $\mathbf{P(A)}$ (c) $P(B)$ (d) $P(A|B)$
9. Entropy is maximized when: (a) one class dominates (b) **classes are balanced** (c) there is one class (d) never
10. L1 regularization tends to produce: (a) small but nonzero coefficients (b) **exact zero coefficients** (c) no change (d) negative coefficients
11. The bias-variance decomposition's irreducible term comes from: (a) the model (b) **noise in the target itself** (c) the optimizer (d) the learning rate
12. A matrix with det=0 is: (a) invertible (b) **singular** (c) symmetric (d) orthogonal
13. Cosine similarity ignores: (a) direction (b) **magnitude** (c) sign (d) dimension
14. MLE for Gaussian-noise regression is equivalent to minimizing: (a) MAE (b) **MSE** (c) cross-entropy (d) entropy
15. The curse of dimensionality causes distances to become: (a) more informative (b) **less discriminative** (c) negative (d) undefined

105.2 Short Answer (16–25 of 30)

16. State the update rule for gradient descent. → $\boldsymbol{\theta := \theta - \alpha \nabla J(\theta)}$
17. Why is n−1 used in sample variance? → Bessel's correction, correcting the downward bias from estimating the mean from the same data.
18. What does the Hessian's eigenvalues tell you at a critical point? → Whether it's a minimum (all positive), maximum (all negative), or saddle (mixed signs).
19. Why must you standardize before regularizing? → So the penalty treats each coefficient fairly regardless of its feature's units.
20. What is the relationship between probability and likelihood? → Same formula, opposite unknowns: probability fixes parameters and asks about data; likelihood fixes data and asks about parameters.
21. Why does K-Means require multiple random initializations? → Its objective is non-convex, so different starting points can converge to different local minima.
22. What does $\sqrt{d_k}$ scaling in attention prevent? → Softmax saturation from dot-product scores growing with embedding dimension.
23. State the four backpropagation equations conceptually. → Output error, propagated error, weight gradient, bias gradient (Section 91.1).

24. Why is Gini impurity the scikit-learn default over entropy? → It's computationally cheaper (no logarithm) and produces nearly identical splits.
25. What is the practical implication of the central limit theorem for sample size planning? → Uncertainty shrinks as $1/\sqrt{n}$, so quadrupling data only halves the standard error.

105.3 Derivation Questions (26–30 of 20)

26. Derive $\sigma'(x) = \sigma(x)(1-\sigma(x))$ from $\sigma(x)=(1+e^{-x})^{-1}$. → Section 11.4.
27. Derive the normal equation from the least-squares objective. → Section 74.2.
28. Derive why the logistic regression gradient simplifies to $X^T(\hat{p}-y)$. → Section 75.2.
29. Derive the K-Means centroid update rule from the WCSS objective. → Section 82.2.
30. Derive log loss as the negative log-likelihood of a Bernoulli distribution. → Section 70.3.

105.4 Coding Questions (31–35 of 20)

31. Implement a numerically stable log loss with clipping. → Section 70.3.
32. Implement PCA from scratch, verifying explained variance sums to 1. → Section 52.3.
33. Implement gradient checking for an arbitrary loss/gradient pair. → Section 89.3.
34. Implement K-Means with the elbow method for choosing k. → Section 82.
35. Implement scaled dot-product attention with a causal mask. → Section 92.3.

105.5 ML Application Questions (36–40 of 20)

36. A credit-scoring model must be explainable to regulators. Which mathematical properties of logistic regression make it suitable? → Convex loss (reproducible), linear log-odds (additive, interpretable coefficients), closed-form odds ratios via exponentiation.
37. Your fraud model has excellent AUC but poor precision at the operating threshold. Explain mathematically why AUC alone is insufficient. → AUC averages ranking quality across *all* thresholds; the deployed system operates at one specific point, which AUC doesn't isolate (Section 57.3-adjacent reasoning, precision@k logic).
38. Why would you choose Lasso over Ridge when you suspect only a few of 500 features are truly relevant? → Lasso's L1 penalty geometry drives irrelevant coefficients to exactly zero, performing automatic feature selection; Ridge would keep all 500 features with small but nonzero weights.
39. Explain mathematically why a deep network trained twice with different seeds gives different results, while logistic regression does not. → Neural network loss is non-convex (many local minima, initialization-dependent); logistic regression's log loss is convex (unique global minimum, reproducible regardless of starting point).
40. Your time-series model shows excellent validation metrics with random splitting. Explain the likely mathematical flaw. → High autocorrelation between adjacent time points means random splitting leaks near-identical information between train and validation sets, inflating the apparent performance (Section 95.1).

105.6 Debugging Scenarios (41–45 of 10)

41-45. → See Section 101's troubleshooting entries 1, 3, 6, 13, 19 for full worked debugging scenarios (exploding gradients, unstable loss, singular matrices, data leakage, gradient-check failures).

105.7 Advanced Reasoning Problems (46–50 of 10)

46. Prove that for a symmetric matrix, eigenvectors corresponding to distinct eigenvalues are orthogonal. → *(Sketch: if $Av_1=\lambda_1 v_1$ and $Av_2=\lambda_2 v_2$ with $\lambda_1 \neq \lambda_2$, then $v_1^T A v_2 = \lambda_2 v_1^T v_2$ and also $= (Av_1)^T v_2 = \lambda_1 v_1^T v_2$ by symmetry; subtracting gives $(\lambda_1-\lambda_2) v_1^T v_2 = 0$, and since $\lambda_1 \neq \lambda_2$, $v_1^T v_2 = 0$.)*

47. Explain why the sum of all eigenvalues of a covariance matrix equals the total variance of the original data. → The trace of a matrix equals the sum of its eigenvalues; the trace of a covariance matrix is the sum of individual feature variances, which is exactly the "total variance" PCA redistributes across components.

48. Why does increasing regularization strength λ monotonically decrease training accuracy but not necessarily monotonically decrease test accuracy? → Increasing λ constrains the model further, strictly increasing training loss (less fit to training data) but moving along the bias-variance tradeoff — test error first decreases (reduced overfitting) then increases (underfitting) as λ grows, producing a U-shaped test-error curve rather than a monotonic one.

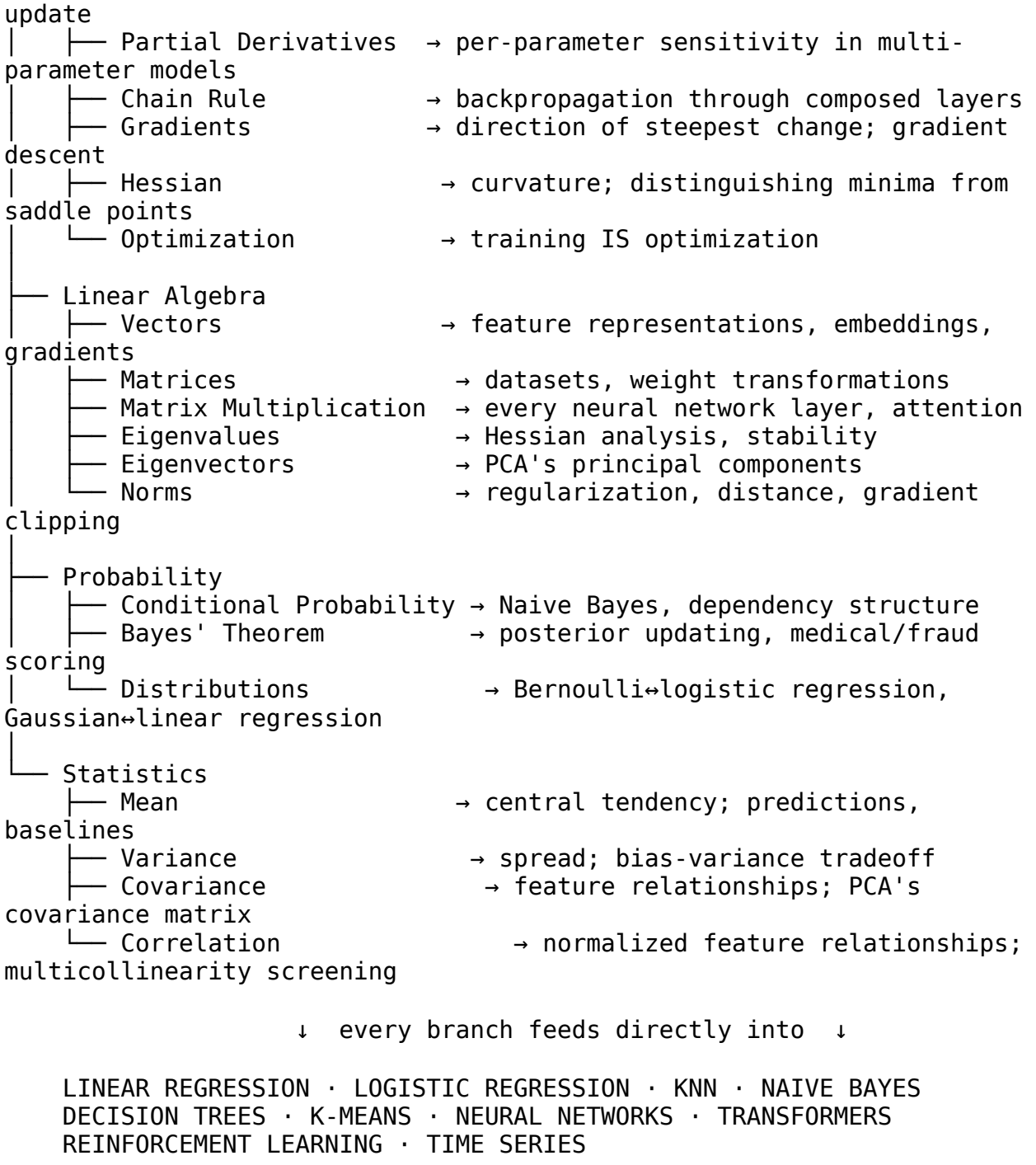
49. Two features have correlation exactly 0 but a scatter plot shows an obvious relationship. Reconcile this mathematically. → Pearson correlation measures only linear association; a perfectly deterministic non-linear relationship (e.g. $y=x^2$) can have exactly zero linear correlation despite complete dependence.

50. Explain, in terms of the bias-variance decomposition, why bagging (e.g. Random Forest) primarily helps high-variance models and offers little benefit for high-bias models. → Averaging B roughly-independent estimators reduces variance by approximately a factor related to B, leaving each individual estimator's bias unchanged; a high-bias model's error is dominated by the bias term, which averaging cannot touch, so bagging yields negligible improvement there while transforming a high-variance model's error profile substantially.

---

106. Master Concept Map

```
MATHEMATICS FOR MACHINE LEARNING
│
├── Algebra
│   └── → equations, inequalities → decision boundaries, constraints
│
├── Functions
│   └── → sigmoid, softmax, ReLU → every model's output/activation
layer
│
├── Logarithms
│   └── → log-likelihood, cross-entropy → numerically stable training
objectives
│
├── Calculus
│   ├── Derivatives          → the slope that guides every parameter
```



Every algorithm in the bottom row draws from multiple branches simultaneously — logistic regression alone needs algebra (the linear score), calculus (the gradient), probability (the Bernoulli likelihood), and statistics (standardizing inputs). This module's real achievement is not teaching five separate subjects; it's teaching **one connected structure** that every ML algorithm instantiates differently.

---

107. Career Application

Role	Mathematics Level Typically Required
Data Analyst	Descriptive statistics, basic probability; rarely derives formulas, but must interpret them correctly (means, correlations, distributions)
Data Scientist	The full module: comfortable deriving and modifying loss functions, understanding bias-variance tradeoffs, choosing algorithms based on their mathematical assumptions
Machine Learning Engineer	Strong practical grasp of optimization, numerical stability, and linear algebra at scale; less emphasis on deriving new algorithms, more on correctly implementing and debugging existing ones in production
AI Engineer	Working knowledge of embeddings, attention, and similarity mathematics (Sections 87, 92-93); less need for classical statistical inference, more need for understanding retrieval and generation mathematics
Deep Learning Engineer	Deep fluency in backpropagation, initialization, optimizer internals (Sections 83, 90-91), and numerical stability at the scale of billions of parameters
Quantitative Analyst	Very strong probability, stochastic processes, and statistical inference; often the most mathematically rigorous of these roles, extending well beyond this module into measure-theoretic probability
Research Scientist	The deepest mathematical requirement: original derivation, proof-writing, and the ability to invent new mathematical formulations — this module is a floor, not a ceiling, for this role
MLOps Engineer	Lighter mathematical demands day-to-day, but genuine numerical-stability literacy (Section 84) is essential for diagnosing production training failures

A practical observation: the *floor* of mathematical competence has risen across every one of these roles in recent years, largely because production ML failures are so often mathematical in origin (Section 101) rather than purely software bugs. Even roles that don't *derive* mathematics daily benefit enormously from being able to *read* it fluently.

---

108. What to Memorize vs What to Understand

108.1 The Principle

Professional ML engineers prioritize intuition and derivation ability over rote formula memorization. A formula you can re-derive is one you truly own; a formula you've only memorized is one you'll misapply the first time a variant appears.

108.2 Worth Memorizing (Used Constantly, Rarely Varies)

- $\theta := \theta - \alpha \nabla J(\theta)$ — the gradient descent update
- $\sigma(x) = 1/(1+e^{-x})$ and its derivative $\sigma(1-\sigma)$
- The four backpropagation equations' *structure* (chain rule applied layer by layer)
- $z = (x-\mu)/\sigma$
- The bias-variance decomposition's shape: error = bias^2 + variance + irreducible
- The basic distance formulas (Euclidean, Manhattan, cosine)
- MSE, MAE, log loss formulas

108.3 Worth Understanding Deeply (Rarely Needs Verbatim Recall, But Must Be Re-Derivable)

- The normal equation's derivation (you'll recognize when it applies far more than you'll need to write it from memory)
- Why L1 vs L2 produce different sparsity behaviour (the reasoning matters more than the formula)
- The MLE → loss function connection (understanding *why* MSE and cross-entropy exist)
- PCA's full derivation (you'll call a library function; understanding *why* it works prevents misuse)
- The numerical stability tricks (max-subtraction, clipping) — understand *why*, apply automatically

108.4 Worth Recognizing But Rarely Computing By Hand

- Full 3×3+ determinant expansions
- Hand-computing eigenvalues for anything beyond 2×2 matrices
- Manual Hessian computation for high-dimensional functions

108.5 The Test That Separates the Two Categories

Ask: *"If I forgot this formula entirely, could I re-derive it from more basic principles in under five minutes?"* If yes, understanding suffices — don't burn memory on it. If no (it's a genuinely arbitrary convention, like Bessel's n−1), memorize it explicitly, because no amount of "understanding" will regenerate an arbitrary choice.

---

109. Final Cheat Sheet

109.1 The Absolute Essentials

```
MASTER EQUATION:        y = f(X, θ)
TRAINING OBJECTIVE:     θ* = argmin_θ L(θ)
GRADIENT DESCENT:       θ := θ − α∇J(θ)
SIGMOID:                σ(x) = 1/(1+e⁻ˣ),  σ'= σ(1−σ)
SOFTMAX (stable):       exp(z − max(z)) / sum(...)
DOT PRODUCT:            x·y = Σxᵢyᵢ = ‖x‖‖y‖cos(θ)
NORMS:                  ‖x‖₁=Σ|xᵢ|,  ‖x‖₂=√Σxᵢ²
STANDARDIZATION:        z = (x−μ)/σ
BAYES:                  P(A|B) = P(B|A)P(A)/P(B)
VARIANCE:               Var(X) = E[X²]−(E[X])²
MSE:                    (1/n)Σ(yᵢ−ŷᵢ)²
LOG LOSS:               −(1/n)Σ[y·logp̂+(1−y)log(1−p̂)]
ENTROPY:                H = −Σpᵢlog₂pᵢ
RIDGE:                  β=(XᵀX+λI)⁻¹Xᵀy
COSINE SIMILARITY:      (x·y)/(‖x‖‖y‖)
```

109.2 NumPy Quick Reference

Need	Code
Dot product	`x @ y` or `np.dot(x, y)`
Matrix multiply	`A @ B` (never `A * B` for this!)
Norm	`np.linalg.norm(x)`
Solve Ax=b	`np.linalg.solve(A, b)` (never `inv(A) @ b`)
Eigenvalues	`np.linalg.eig(A)` or `eigh` for symmetric
Mean/var/std	`x.mean()`, `x.var(ddof=1)`, `x.std()`
Standardize	`(X - X.mean(0)) / X.std(0)`
Stable exp	`np.exp(z - z.max())`

109.3 The Decision Checklist Before Trusting a Result

- ☐ Did I check array shapes at every step?
- ☐ Did I fit any scaler/transform on training data only?
- ☐ Did I gradient-check any hand-derived gradient?
- ☐ Did I compare against a simple baseline?
- ☐ Did I verify convergence (flat loss, small gradient norm)?
- ☐ Did I check for NaN/Inf in intermediate values?
- ☐ Does the result make domain sense?

109.4 The Ten Concepts That Unlock Everything Else

1. $y = f(X,\theta)$ — the universal ML template
2. $\theta^* = \text{argmin } L(\theta)$ — training is optimization

3. Gradient = direction of steepest increase; negative gradient descends
4. Chain rule = backpropagation
5. MLE explains where loss functions come from
6. Bias-variance tradeoff explains overfitting/underfitting
7. Convexity explains why some models are reproducible and others aren't
8. Standardization is required by distance-based and gradient-based methods
9. Eigen-decomposition of covariance = PCA
10. Numerical stability tricks (clipping, max-subtraction, log-space) are not optional polish — they are correctness requirements

---

110. Preparation for Module 6

110.1 What You Have Built

You now understand the mathematical machinery underneath every algorithm you will study: how a model is trained (optimization), how it represents data (linear algebra), and — the piece Module 6 goes deep on — how it reasons under uncertainty (probability). This module deliberately front-loaded probability foundations (Sections 55-70) precisely because Module 6 assumes them.

110.2 The Progression

```
MODULE 1  Machine Learning Foundations          (what ML is, why it
exists)
    ↓
MODULE 2  Python for Machine Learning           (the implementation
language)
    ↓
MODULE 3  NumPy                                 (efficient numerical
computation)
    ↓
MODULE 4  Pandas                                (data manipulation)
    ↓
MODULE 5  Mathematics for Machine Learning       ← YOU ARE HERE
    ↓
MODULE 6  Probability for Machine Learning       (deepening Sections
55-70)
    ↓
MODULE 7  Statistics for Machine Learning        (deepening Sections
65-68, inference)
    ↓
MODULE 8  Machine Learning Workflow             (assembling everything
into real projects)
    ↓
[Core Machine Learning Algorithms begin]
```

110.3 What Module 6 Will Deepen

This Module's Foundation	Module 6 Extends It To
Section 55 — Probability axioms	Full axiomatic probability theory, set-theoretic rigor
Section 56 — Conditional probability	Independence, chain rule for probability, conditional independence structures
Section 57 — Bayes' theorem	Bayesian inference, prior/posterior updating over full distributions, not just single events
Section 58 — Random variables	Joint, marginal, and conditional distributions; transformations of random variables
Section 62 — Distributions catalogue	Deeper treatment of each distribution's properties, moment-generating functions, relationships between distributions
Section 69-70 — Likelihood and MLE	Full derivation of MLE for every major distribution family; introduction to Bayesian estimation (MAP) as a contrast

110.4 A Final Note on Readiness

If Sections 55-70 of this module felt comfortable — if Bayes' theorem's derivation, the difference between probability and likelihood, and the log loss/MLE connection all made sense — you are well-prepared for Module 6. If any of those felt shaky, it is worth revisiting them before proceeding, since Module 6 builds directly on top of this exact foundation rather than re-explaining it from scratch.

Conclusion

This module's deepest goal was never to make you a mathematician. It was to remove the feeling — common among self-taught and even formally-trained practitioners alike — that machine learning is a black box governed by formulas you apply without understanding. Every derivation in this module (the sigmoid's derivative, the normal equation, the logistic regression gradient's beautiful cancellation, log loss as negative log-likelihood) was chosen because it recurs constantly in practice, and because understanding *why* it takes the form it does makes every future encounter with it — in a paper, in a bug, in an interview — faster and more confident than memorization ever could.

The signature pattern threaded through nearly every section — **formula → numerical example → NumPy implementation → ML application** — exists because mathematics learned only symbolically evaporates, while mathematics anchored to a hand-computed number and a working line of code stays. When you next see X^TX, $\sqrt{d_k}$, or `argmin_θ L(θ)` in a paper or a codebase, you should now recognize not just what it *is*, but what it's *for* — and that recognition is the actual, durable outcome of this module.

Module 6 picks up exactly where Sections 55-70 left off. Everything else here — the calculus, the linear algebra, the optimization theory — will keep resurfacing throughout the rest of this curriculum, through Deep Learning, MLOps, and eventually Generative AI and LLMs, because it is not preliminary material to be set aside. It is the engine.

End of Module 5

Kajola Gbenga | Data Scientist & AI Engineer